From Search to Expressions: Trees and Traversals

Operator combinations, unique printing, Make 24 Task B, and expression trees

CSCI 235 – Data Structures

Hunter College, Fall 2026

Class 17 Tuesday, November 3, 2026

Today

Last two weeks you met the ideas we need today. We recall them, **break out the key ideas**, and then **you write the code**.

1. **Operator combinations** (Class 15): a recursive search over a *string*, with choose, explore, undo.
2. **Printing unique strings** (Classes 15–16): `printAll` versus `printUnique`, and skipping repeated branches.
3. **Project 3, Task B**: how to return *one expression*, and how Task B grows out of Task A.
4. **Expression trees**: pre-order, in-order, and post-order traversals.

- **Exercise 7** (Permutation Trace) is due today.
- Today's lecture slip has a short **survey** about the pace and learning activities (including an optional mini-quiz). Please answer it honestly: it tells me whether to slow down.

Recall: Operator Combinations

Problem. The numbers `1 2 3` are in this order. Fill the **two gaps** with `+`, `-`, `*`, `/`, and print every expression.

Question. (a) How many lines will be printed? (b) What are the first four?

Recall: Operator Combinations

Problem. The numbers `1 2 3` are in this order. Fill the **two gaps** with `+`, `-`, `*`, `/`, and print every expression.

Question. (a) How many lines will be printed? (b) What are the first four?

(a) 16: four choices for the first gap times four for the second. **(b)** `1+2+3`, `1+2-3`, `1+2*3`, `1+2/3`. The last gap changes fastest.

Picture. Start from `1`. Each expression **grows** into four longer ones, one for each operator. The next slide draws it.

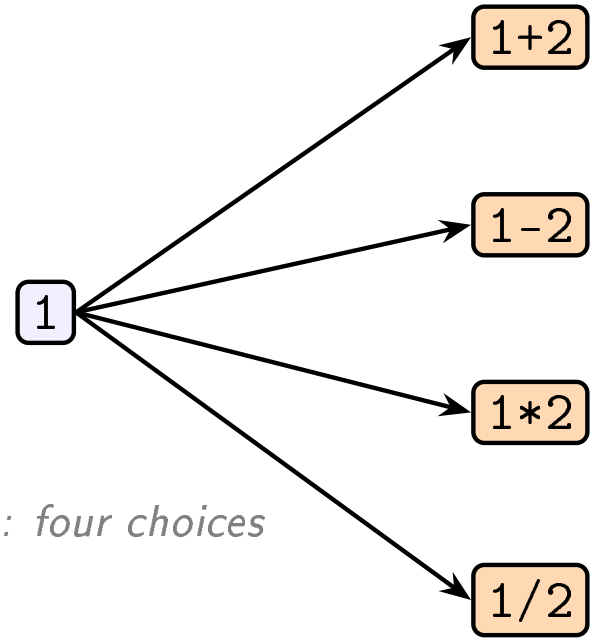
Watch the Expression Grow

Each step appends **an operator and the next number**: the string `expr` grows, and `k` counts how many numbers it uses.

1

Watch the Expression Grow

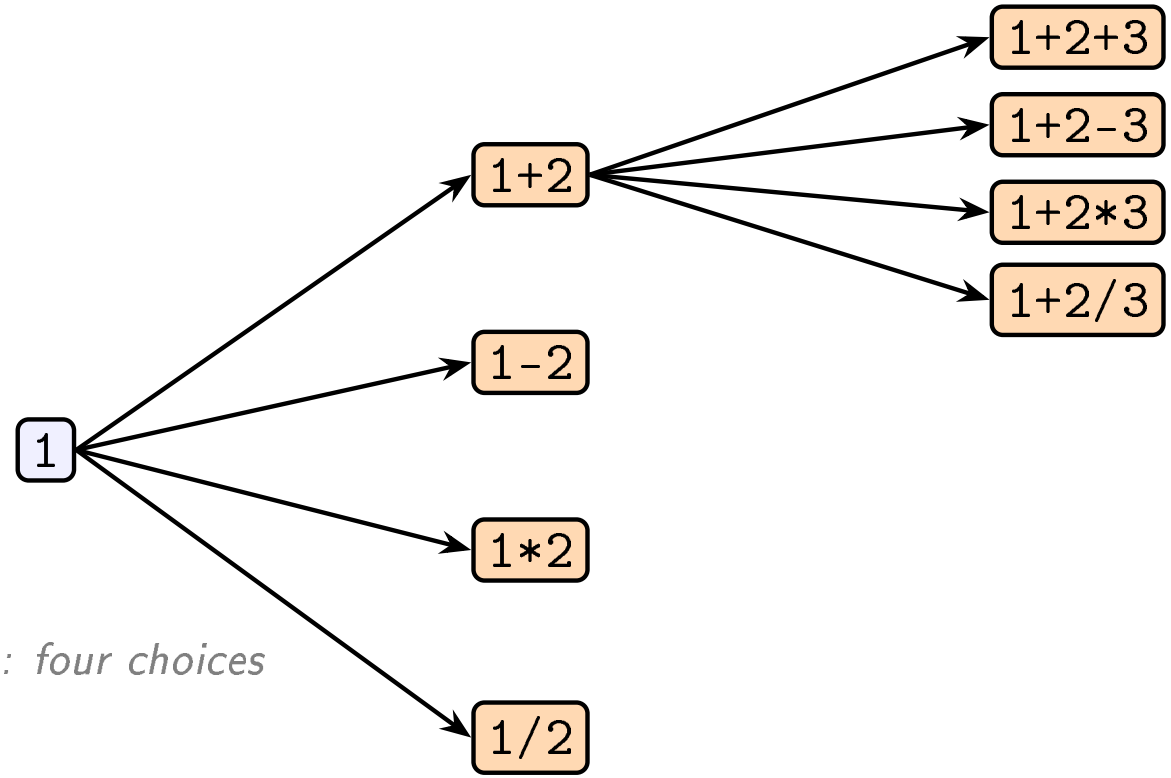
Each step appends **an operator and the next number**: the string `expr` grows, and `k` counts how many numbers it uses.



gap 1: four choices

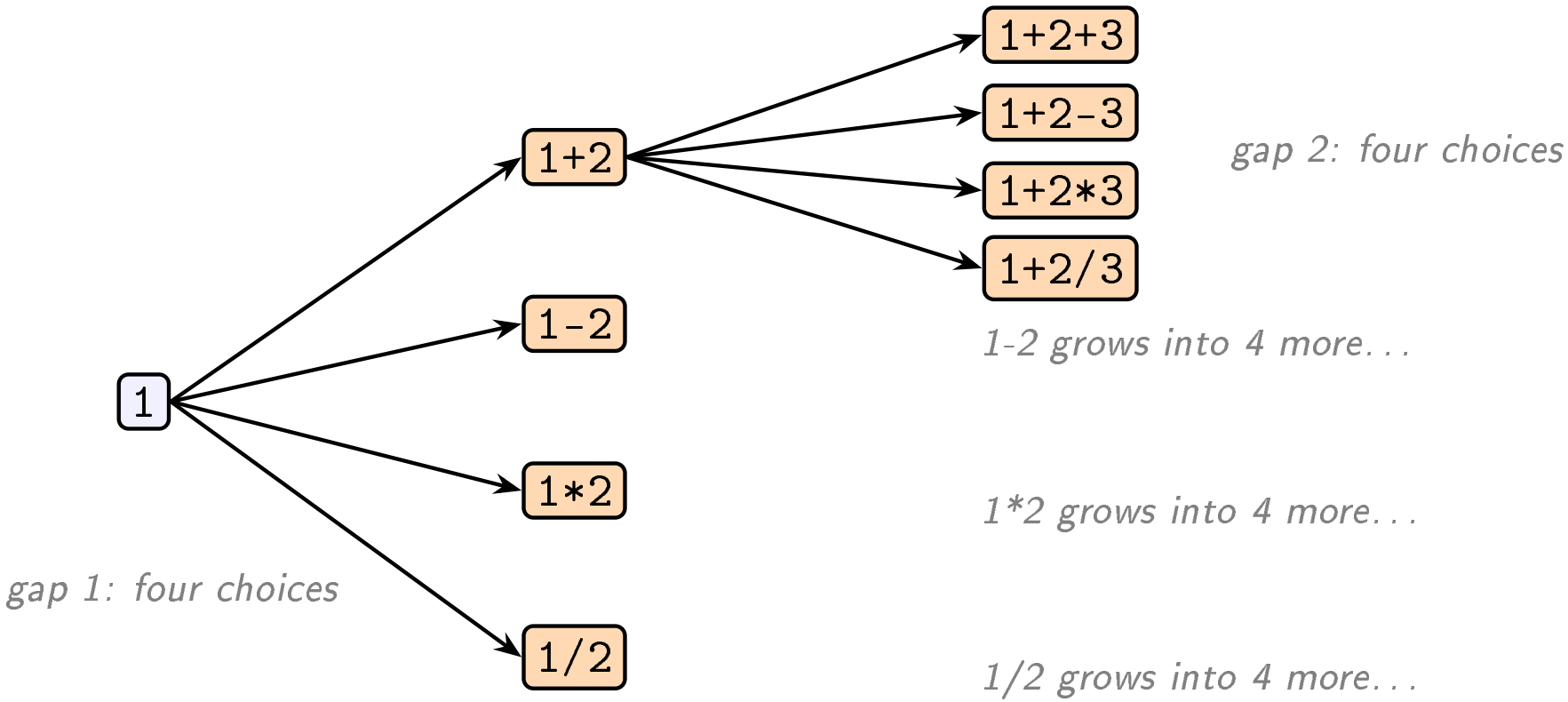
Watch the Expression Grow

Each step appends **an operator and the next number**: the string `expr` grows, and `k` counts how many numbers it uses.



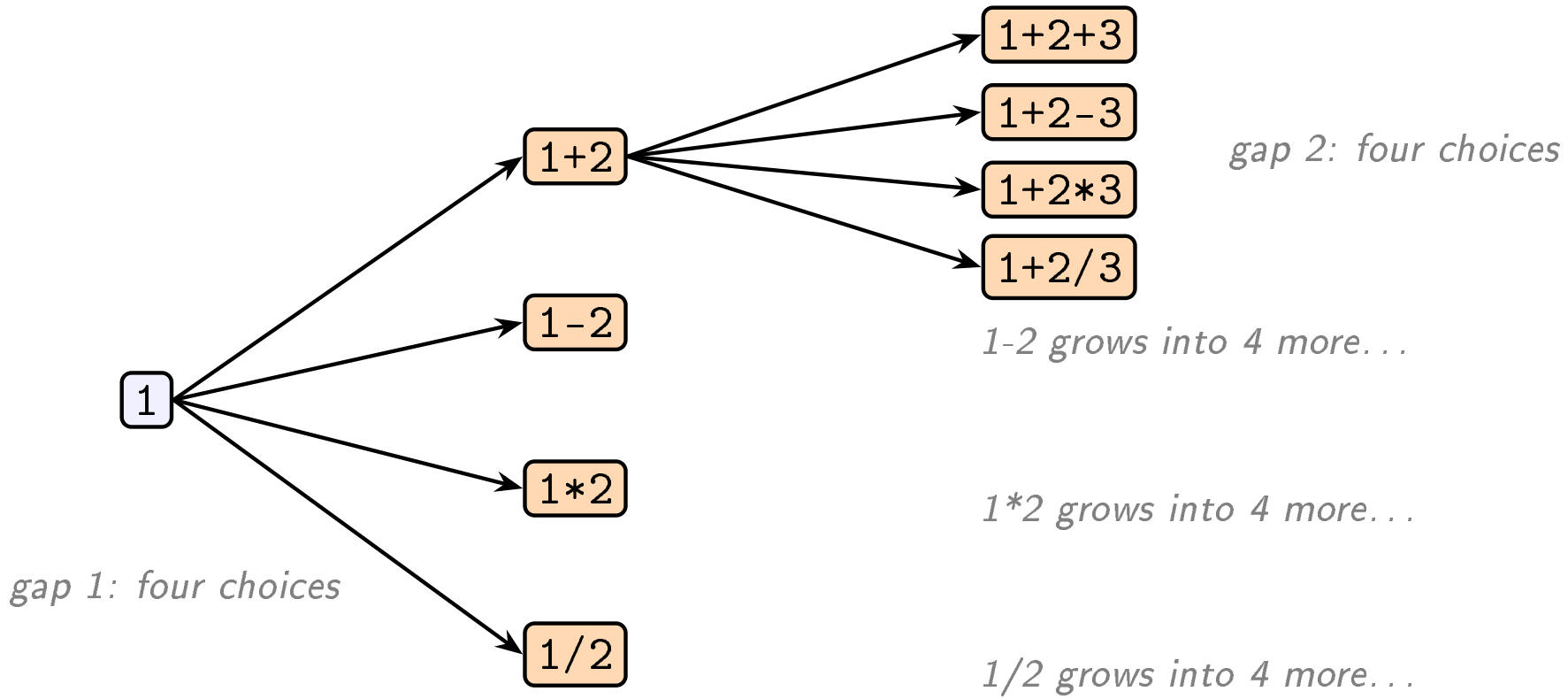
Watch the Expression Grow

Each step appends **an operator and the next number**: the string `expr` grows, and `k` counts how many numbers it uses.



Watch the Expression Grow

Each step appends **an operator and the next number**: the string `expr` grows, and `k` counts how many numbers it uses.



Count. $4 \times 4 = 16$ complete expressions (four shown; each other first-gap choice has four more). **The last gap changes fastest**: we finish all four children of `1+2` before moving on to `1-2`.

Key Ideas of `ops`

1. **State.** The expression string built so far (`expr`) and `k`, how many numbers are already placed.

Key Ideas of `ops`

1. **State.** The expression string built so far (`expr`) and `k`, how many numbers are already placed.
2. **Base case.** When `k` equals the number of values, the expression is complete: print it.

Key Ideas of `ops`

1. **State.** The expression string built so far (`expr`) and `k`, how many numbers are already placed.
2. **Base case.** When `k` equals the number of values, the expression is complete: print it.
3. **Branching.** At each gap, try all four operators: a loop over `"+-*/"`.

Key Ideas of `ops`

1. **State.** The expression string built so far (`expr`) and `k`, how many numbers are already placed.
2. **Base case.** When `k` equals the number of values, the expression is complete: print it.
3. **Branching.** At each gap, try all four operators: a loop over `"+-*/"`.
4. **Choose.** Append the operator and the next number to `expr`.

Key Ideas of `ops`

1. **State.** The expression string built so far (`expr`) and `k`, how many numbers are already placed.
2. **Base case.** When `k` equals the number of values, the expression is complete: print it.
3. **Branching.** At each gap, try all four operators: a loop over `"+-*/"`.
4. **Choose.** Append the operator and the next number to `expr`.
5. **Explore.** Recurse to fill the next gap, with `k + 1`.

Key Ideas of `ops`

1. **State.** The expression string built so far (`expr`) and `k`, how many numbers are already placed.
2. **Base case.** When `k` equals the number of values, the expression is complete: print it.
3. **Branching.** At each gap, try all four operators: a loop over `"+-*/"`.
4. **Choose.** Append the operator and the next number to `expr`.
5. **Explore.** Recurse to fill the next gap, with `k + 1`.
6. **Undo.** `expr` is shared by every call (passed by reference), so cut it back to the length it had before the choose.

Key Ideas of `ops`

1. **State.** The expression string built so far (`expr`) and `k`, how many numbers are already placed.
2. **Base case.** When `k` equals the number of values, the expression is complete: print it.
3. **Branching.** At each gap, try all four operators: a loop over `"+-*/"`.
4. **Choose.** Append the operator and the next number to `expr`.
5. **Explore.** Recurse to fill the next gap, with `k + 1`.
6. **Undo.** `expr` is shared by every call (passed by reference), so cut it back to the length it had before the choose.

Same pattern as permutations: choose, explore, undo, and the undo mirrors the choose. Now **you** write it.

Write It Yourself: ops

Step 1 of 6: Base case

Hint. When is the expression finished? What do we do with it? (k counts the numbers already placed.)

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    // ??? your turn
}
```

Write It Yourself: ops

Step 1 of 6: Base case

Hint. When is the expression finished? What do we do with it? (`k` counts the numbers already placed.)

Why. All numbers are placed when `k` equals `v.size()`. The string is a complete expression: print it and return.

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) {
        std::cout << expr << "\n";
        return;
    }
}
```

Write It Yourself: ops

Step 2 of 6: Try every operator

Hint. Each gap has four choices. Which loop visits them?

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) {
        std::cout << expr << "\n";
        return;
    }
    // ??? your turn
}
```

Write It Yourself: ops

Step 2 of 6: Try every operator

Hint. Each gap has four choices. Which loop visits them?

Why. A range-for over the string "+-*/" gives op the values '+', '-', '*', '/' in turn.

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) {
        std::cout << expr << "\n";
        return;
    }
    for (char op : std::string("+-*/")) {
    }
}
```

Write It Yourself: ops

Step 3 of 6: Remember the length

Hint. We must be able to undo the append later. What do we save first? Why not just call `pop_back()` twice?

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) {
        std::cout << expr << "\n";
        return;
    }
    for (char op : std::string("+-*/")) {
        // ??? your turn
    }
}
```

Write It Yourself: ops

Step 3 of 6: Remember the length

Hint. We must be able to undo the append later. What do we save first? Why not just call `pop_back()` twice?

Why. `std::to_string(v[k])` can be several characters (for example "12"), so a fixed number of `pop_back()` calls is wrong. Save the length instead.

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) {
        std::cout << expr << "\n";
        return;
    }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
    }
}
```

Write It Yourself: ops

Step 4 of 6: Choose

Hint. Extend the string: the operator first, then the next number.

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) {
        std::cout << expr << "\n";
        return;
    }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        // ??? your turn
    }
}
```

Write It Yourself: ops

Step 4 of 6: Choose

Hint. Extend the string: the operator first, then the next number.

Why. `expr += op;` then `expr += std::to_string(v[k]);`. The string now describes the choice we are about to explore.

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) {
        std::cout << expr << "\n";
        return;
    }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op;
        expr += std::to_string(v[k]);
    }
}
```

Write It Yourself: ops

Step 5 of 6: Explore

Hint. Fill the next gap. Which `k`, and which string, do we pass?

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) {
        std::cout << expr << "\n";
        return;
    }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op;
        expr += std::to_string(v[k]);
        // ??? your turn
    }
}
```

Write It Yourself: ops

Step 5 of 6: Explore

Hint. Fill the next gap. Which `k`, and which string, do we pass?

Why. `ops(v, k + 1, expr)`: the same shared string, one number further.

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) {
        std::cout << expr << "\n";
        return;
    }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op;
        expr += std::to_string(v[k]);
        ops(v, k + 1, expr);
    }
}
```

Write It Yourself: ops

Step 6 of 6: Undo

Hint. Restore `expr` to exactly what it was before the choose.

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) {
        std::cout << expr << "\n";
        return;
    }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op;
        expr += std::to_string(v[k]);
        ops(v, k + 1, expr);
        // ??? your turn
    }
}
```

Write It Yourself: ops

Step 6 of 6: Undo

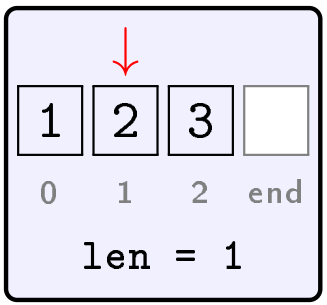
Hint. Restore `expr` to exactly what it was before the choose.

Why. `expr.resize(len)` cuts it back. The undo mirrors the choose, so the next operator starts from the same string.

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) {
        std::cout << expr << "\n";
        return;
    }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op;
        expr += std::to_string(v[k]);
        ops(v, k + 1, expr);
        expr.resize(len);
    }
}
```

Watch the String Change: Call Tree of ops

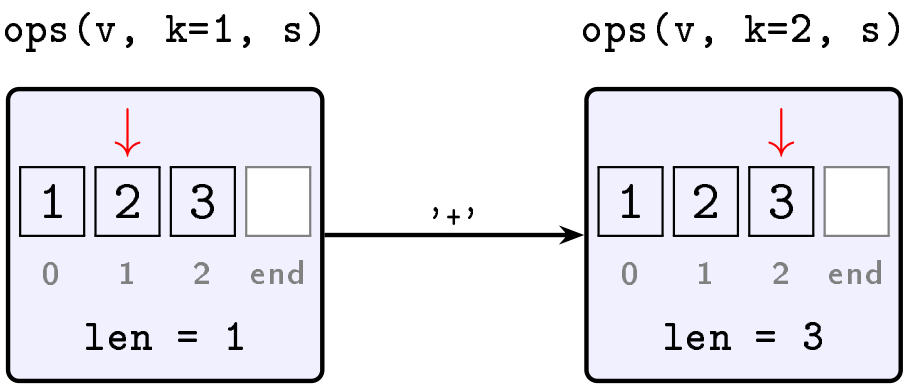
`ops(v, k=1, s)`



`s = "1"`

Shared string `s` (passed by reference, so every call sees the same one). First call: `ops(v, 1, s)`, `s = "1"`. `k` points at `v[1]`. It saves `len = 1`. The empty fourth box is the end, `k == v.size()`, not `v[3]`.

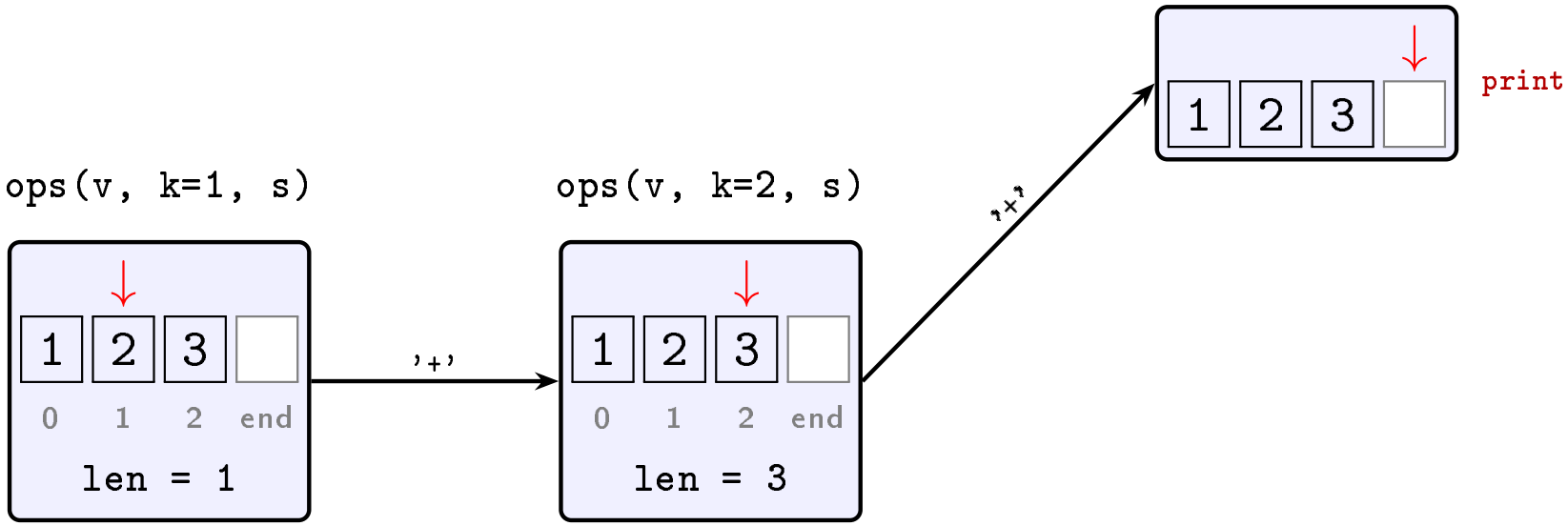
Watch the String Change: Call Tree of ops



s = "1+2"

Shared string `s` (passed by reference, so every call sees the same one). Choose `'+'`: `s += '+'; s += std::to_string(v[1]);` then explore with `k + 1`. The new call saves `len = 3`.

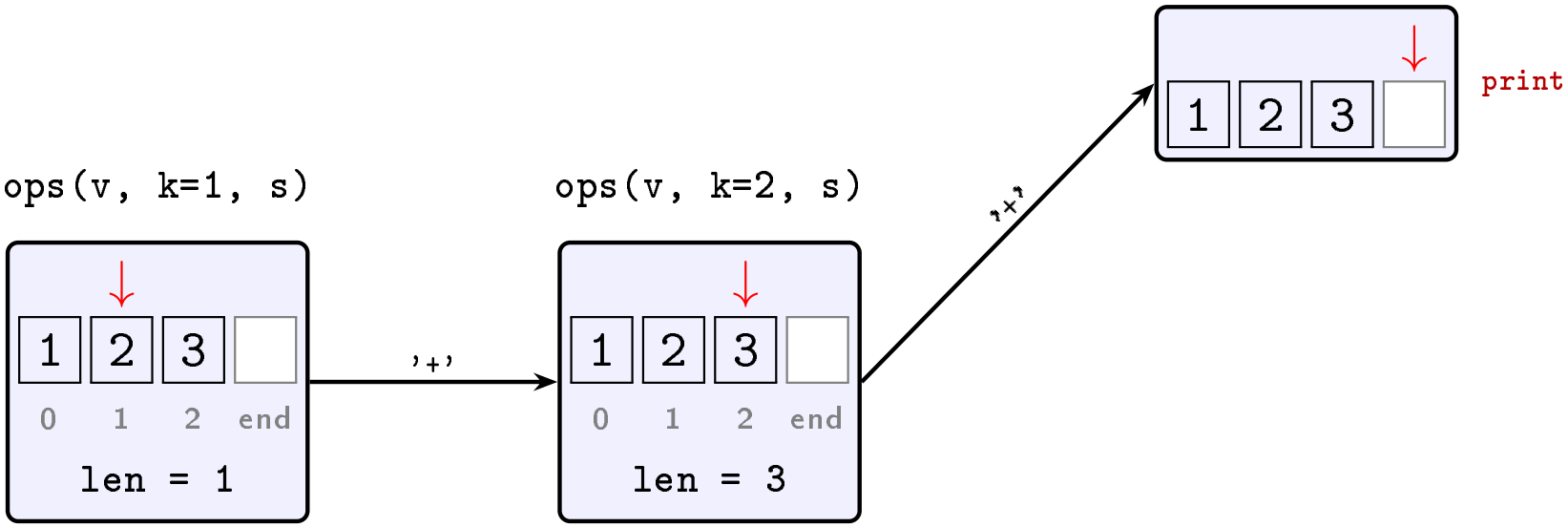
Watch the String Change: Call Tree of ops



s = "1+2+3"

Shared string `s` (passed by reference, so every call sees the same one). Choose `'+'`: `s += '+'; s += std::to_string(v[2]);`. Now `k = 3 = v.size()`: **base case, print**.

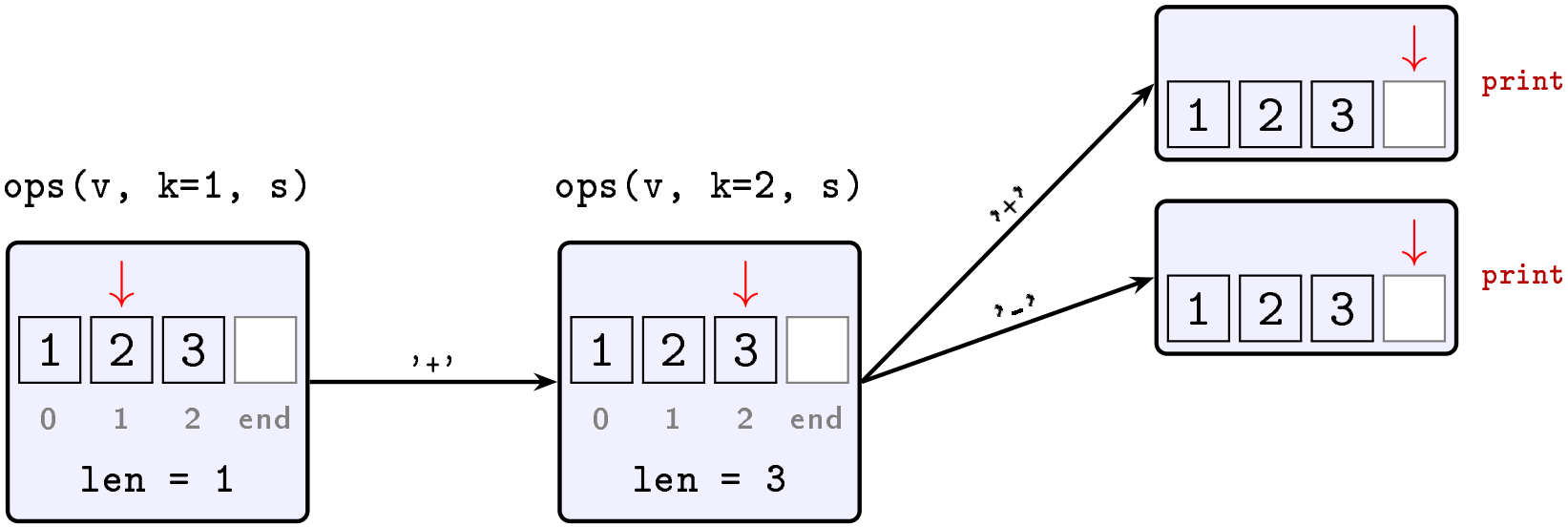
Watch the String Change: Call Tree of ops



s = "1+2" (removed +3)

Shared string `s` (passed by reference, so every call sees the same one). Undo: back in the call with `len = 3`, `s.resize(len)` cuts `s` back to `"1+2"`.

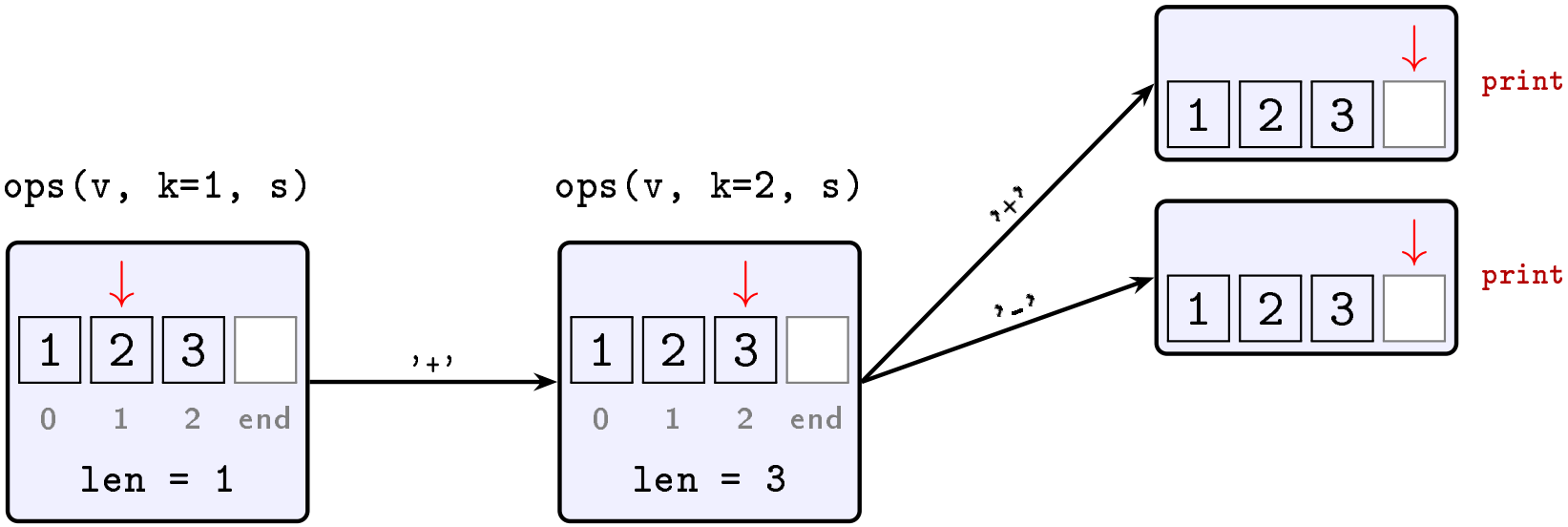
Watch the String Change: Call Tree of ops



s = "1+2-3"

Shared string s (passed by reference, so every call sees the same one). Next operator '-': append -3, base case, print.

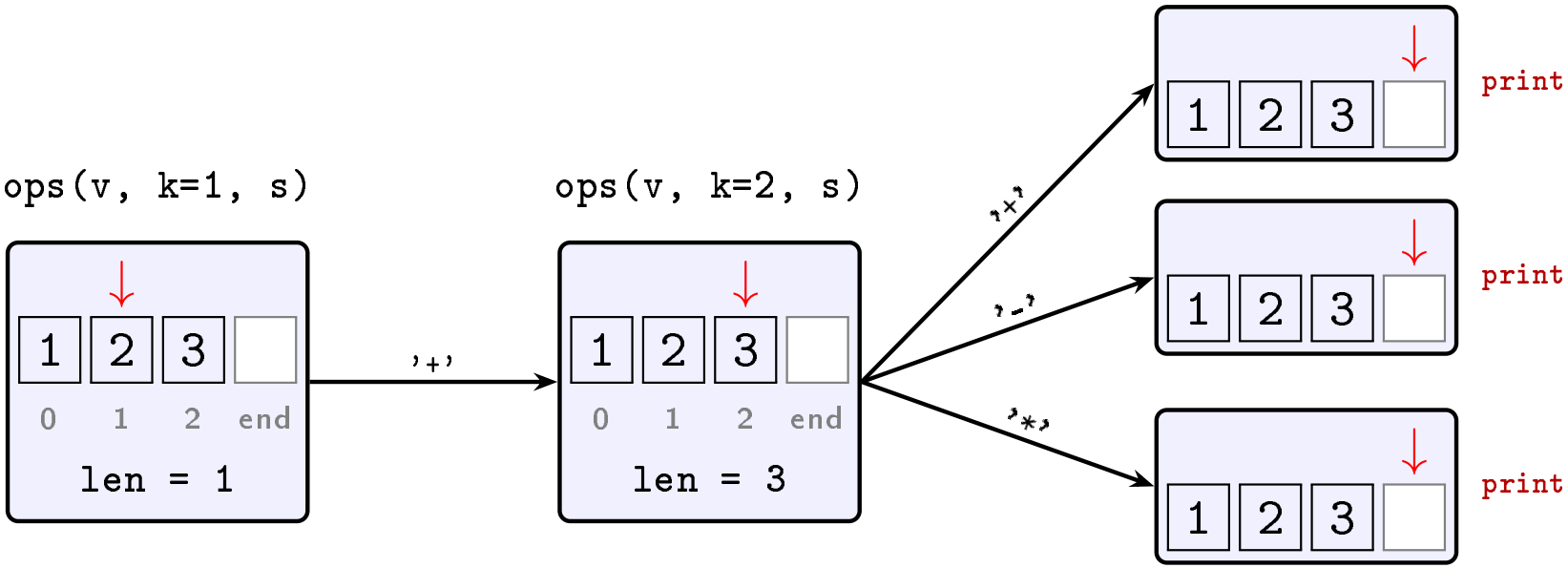
Watch the String Change: Call Tree of ops



s = "1+2" (removed -3)

Shared string `s` (passed by reference, so every call sees the same one). Undo: `s.resize(3)` again.

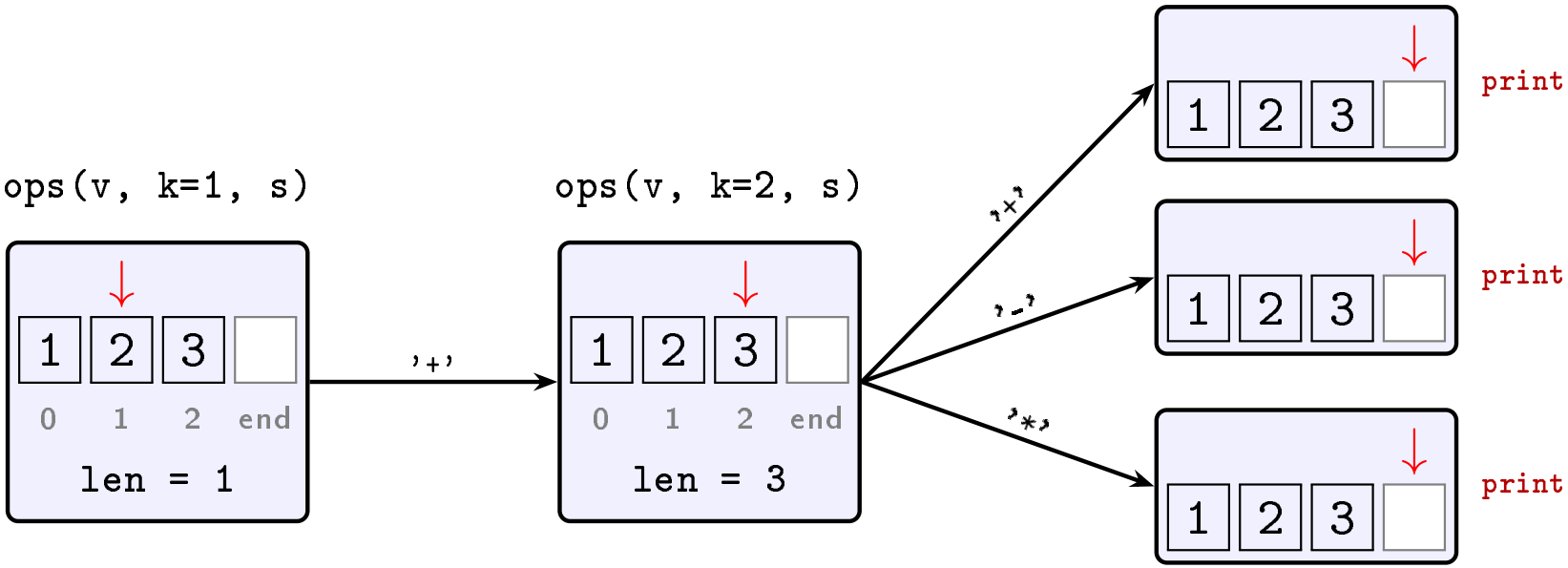
Watch the String Change: Call Tree of ops



s = "1+2*3"

Shared string s (passed by reference, so every call sees the same one). Next operator '*': append *3, print.

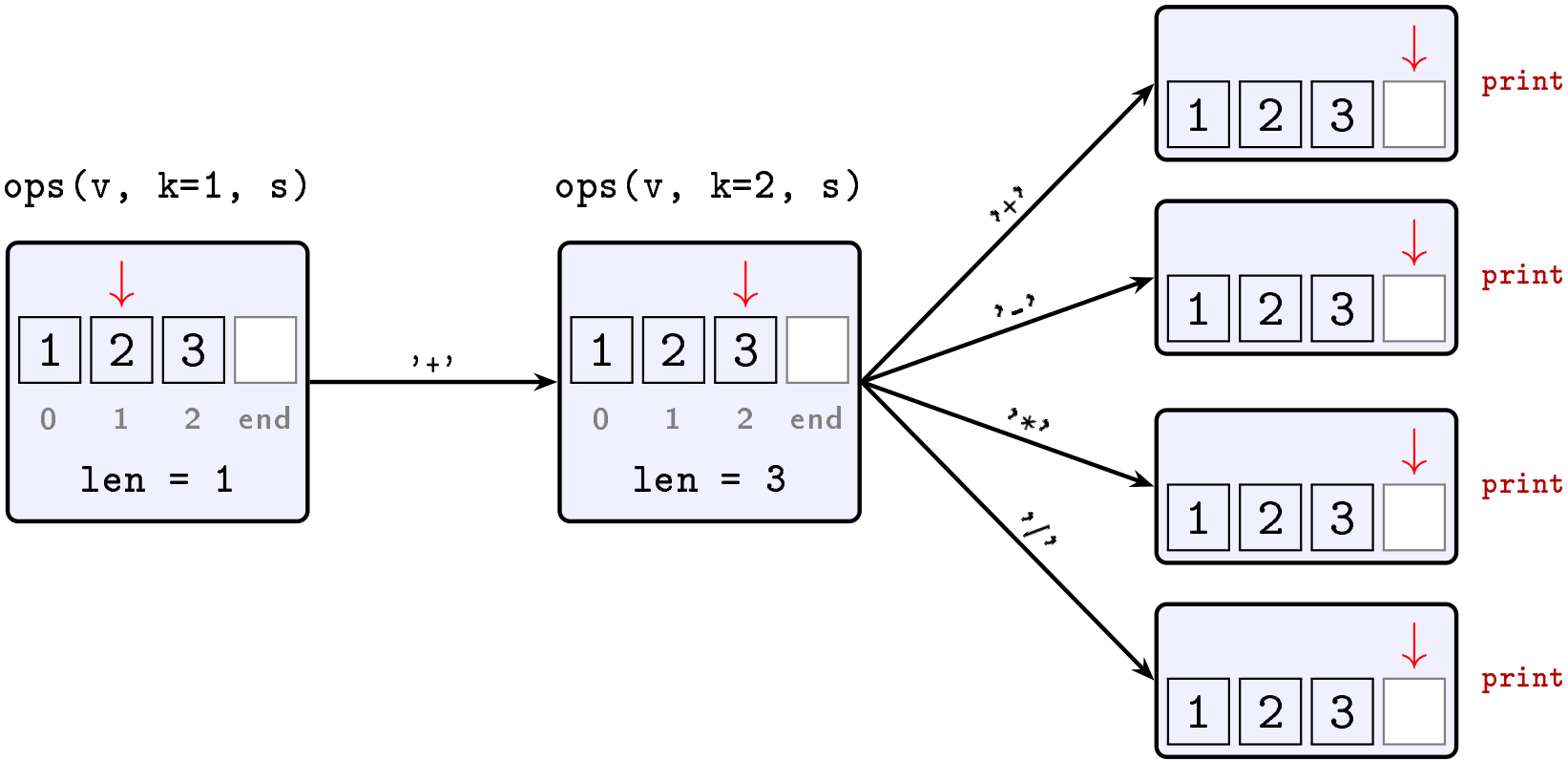
Watch the String Change: Call Tree of ops



s = "1+2" (removed *3)

Shared string `s` (passed by reference, so every call sees the same one). Undo: `s.resize(3)`.

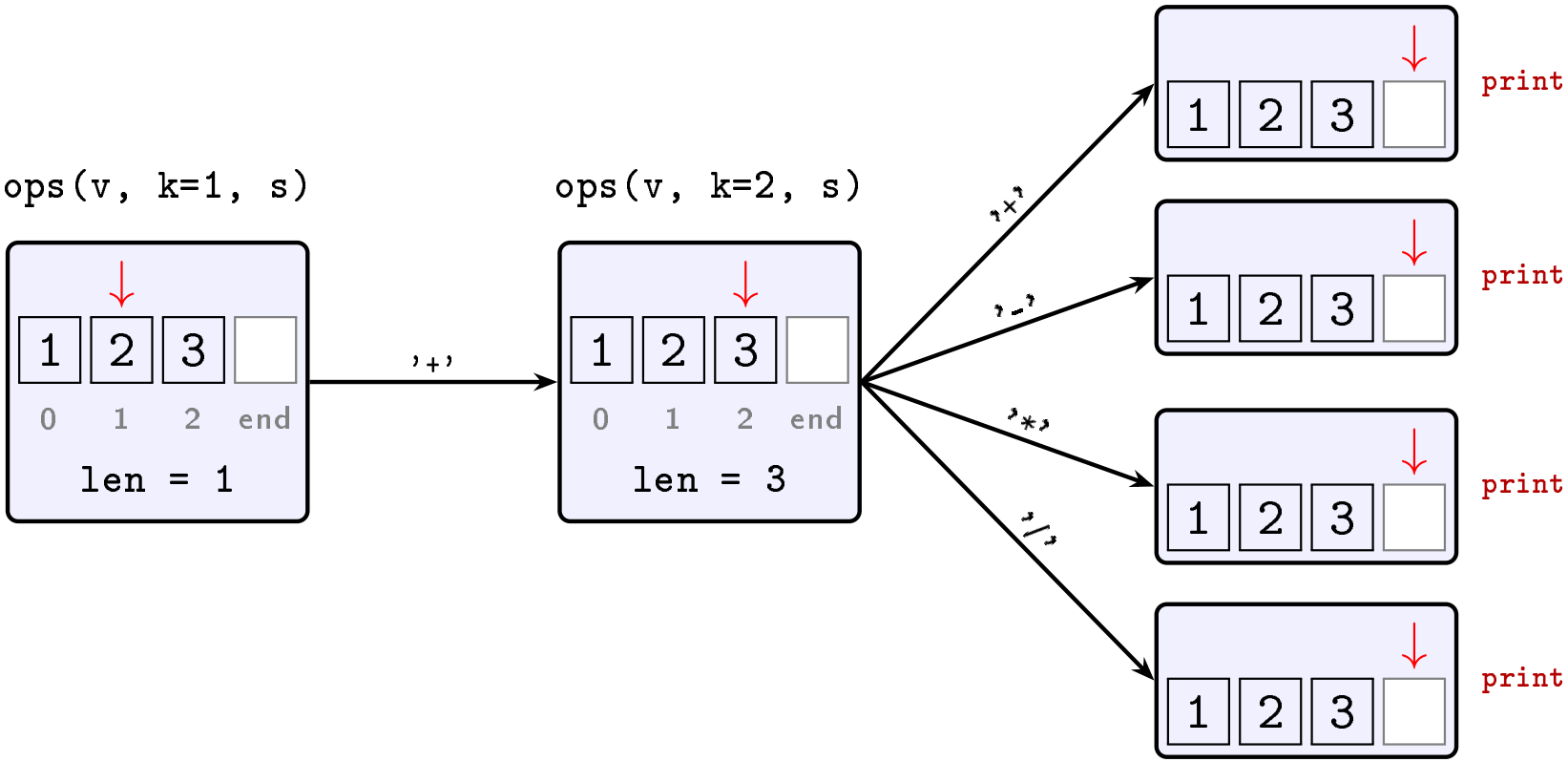
Watch the String Change: Call Tree of ops



s = "1+2/3"

Shared string s (passed by reference, so every call sees the same one). Last operator '/': append /3, print.

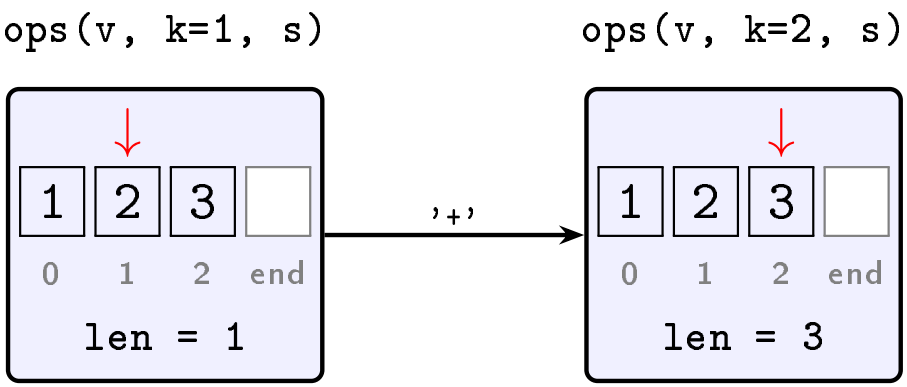
Watch the String Change: Call Tree of ops



s = "1+2" (removed /3)

Shared string `s` (passed by reference, so every call sees the same one). Undo: `s.resize(3)`. The loop in this call is finished, so it returns.

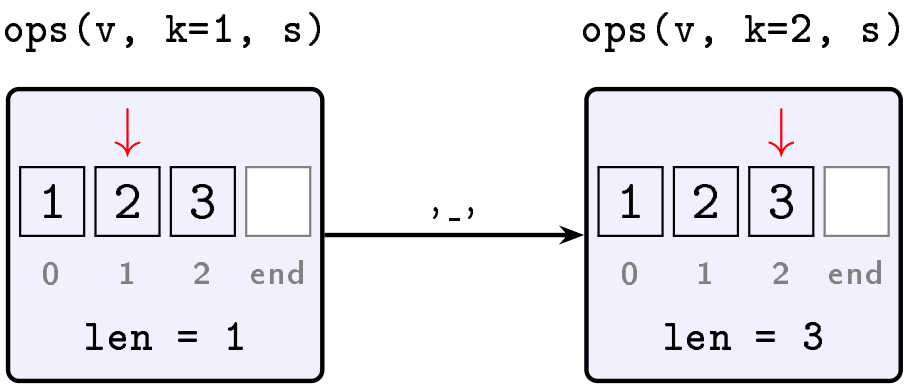
Watch the String Change: Call Tree of `ops`



`s = "1"` (removed +2)

Shared string `s` (passed by reference, so every call sees the same one). Back in the first call: `s.resize(len)` with `len = 1` cuts `s` back to `"1"`.

Watch the String Change: Call Tree of ops



s = "1-2"

Shared string s (passed by reference, so every call sees the same one). Next operator '-': append -2. The same process repeats under "1-2". When everything returns, s is "1" again.

Same Trace as a Table: `ops(1,2,3, 1, "1")`

#	Call	Action	`expr` after
1	`ops(v,1,"1")`	`len = 1`; choose +2	`1+2`

Same Trace as a Table: `ops(1,2,3, 1, "1")`

#	Call	Action	`expr` after
1	`ops(v,1,"1")`	`len = 1`; choose +2	`1+2`
2	`ops(v,2,"1+2")`	`len = 3`; choose +3	`1+2+3`

Same Trace as a Table: `ops(1,2,3, 1, "1")`

#	Call	Action	`expr` after
1	`ops(v,1,"1")`	`len` = 1; choose +2	`1+2`
2	`ops(v,2,"1+2")`	`len` = 3; choose +3	`1+2+3`
3	`ops(v,3,"1+2+3")`	$k = 3$: **print** `1+2+3`, return	`1+2+3`

Same Trace as a Table: `ops(1,2,3, 1, "1")`

#	Call	Action	`expr` after
1	`ops(v,1,"1")`	`len = 1`; choose +2	`1+2`
2	`ops(v,2,"1+2")`	`len = 3`; choose +3	`1+2+3`
3	`ops(v,3,"1+2+3")`	$k = 3$: **print** `1+2+3`, return	`1+2+3`
4	back in call 2	`resize(3)`, then choose `-3`	`1+2-3`

Same Trace as a Table: `ops(1,2,3, 1, "1")`

#	Call	Action	`expr` after
1	`ops(v,1,"1")`	`len = 1`; choose +2	`1+2`
2	`ops(v,2,"1+2")`	`len = 3`; choose +3	`1+2+3`
3	`ops(v,3,"1+2+3")`	$k = 3$: **print** `1+2+3`, return	`1+2+3`
4	back in call 2	`resize(3)`, then choose `-3`	`1+2-3`
5	(call at $k = 3$)	**print** `1+2-3`, return	`1+2-3`

Same Trace as a Table: `ops(1,2,3, 1, "1")`

#	Call	Action	`expr` after
1	`ops(v,1,"1")`	`len = 1`; choose `+2`	`1+2`
2	`ops(v,2,"1+2")`	`len = 3`; choose `+3`	`1+2+3`
3	`ops(v,3,"1+2+3")`	$k = 3$: **print** `1+2+3`, return	`1+2+3`
4	back in call 2	`resize(3)`, then choose `-3`	`1+2-3`
5	(call at $k = 3$)	**print** `1+2-3`, return	`1+2-3`
6	back in call 2	`resize(3)`, choose `*3`, **print**; `/3`, **print**	`1+2/3`

Same Trace as a Table: `ops(1,2,3, 1, "1")`

#	Call	Action	`expr` after
1	`ops(v,1,"1")`	`len = 1`; choose +2	`1+2`
2	`ops(v,2,"1+2")`	`len = 3`; choose +3	`1+2+3`
3	`ops(v,3,"1+2+3")`	$k = 3$: **print** `1+2+3`, return	`1+2+3`
4	back in call 2	`resize(3)`, then choose `-3`	`1+2-3`
5	(call at $k = 3$)	**print** `1+2-3`, return	`1+2-3`
6	back in call 2	`resize(3)`, choose `*3`, **print**; `/3`, **print**	`1+2/3`
7	back in call 2	`resize(3)`: loop over, return	`1+2`

Same Trace as a Table: `ops(1,2,3, 1, "1")`

#	Call	Action	`expr` after
1	`ops(v,1,"1")`	`len = 1`; choose `+2`	`1+2`
2	`ops(v,2,"1+2")`	`len = 3`; choose `+3`	`1+2+3`
3	`ops(v,3,"1+2+3")`	$k = 3$: **print** `1+2+3`, return	`1+2+3`
4	back in call 2	`resize(3)`, then choose `-3`	`1+2-3`
5	(call at $k = 3$)	**print** `1+2-3`, return	`1+2-3`
6	back in call 2	`resize(3)`, choose `*3`, **print**; `/3`, **print**	`1+2/3`
7	back in call 2	`resize(3)`: loop over, return	`1+2`
8	back in call 1	`resize(1)`, then choose `-2`	`1-2`

Same Trace as a Table: `ops(1,2,3, 1, "1")`

#	Call	Action	`expr` after
1	`ops(v,1,"1")`	`len = 1`; choose `+2`	`1+2`
2	`ops(v,2,"1+2")`	`len = 3`; choose `+3`	`1+2+3`
3	`ops(v,3,"1+2+3")`	$k = 3$: **print** `1+2+3`, return	`1+2+3`
4	back in call 2	`resize(3)`, then choose `-3`	`1+2-3`
5	(call at $k = 3$)	**print** `1+2-3`, return	`1+2-3`
6	back in call 2	`resize(3)`, choose `*3`, **print**; `/3`, **print**	`1+2/3`
7	back in call 2	`resize(3)`: loop over, return	`1+2`
8	back in call 1	`resize(1)`, then choose `-2`	`1-2`

After the last return, `expr` is `"1"` again, exactly as it was passed in. **That is what the undo guarantees.**

Recall: printAll

```
void printAll(std::string& s, int k) {
    if (k == static_cast<int>(s.size())) {  // every seat filled
        std::cout << s << "\n";
        return;
    }
    for (int i = k; i < static_cast<int>(s.size()); ++i) {
        std::swap(s[k], s[i]);              // choose
        printAll(s, k + 1);                 // explore
        std::swap(s[k], s[i]);              // undo
    }
}
```

Question. For "ABC" it prints 6 lines. For "ABB", how many lines, and how many *different* strings?

Recall: printAll

```
void printAll(std::string& s, int k) {
    if (k == static_cast<int>(s.size())) {  // every seat filled
        std::cout << s << "\n";
        return;
    }
    for (int i = k; i < static_cast<int>(s.size()); ++i) {
        std::swap(s[k], s[i]);              // choose
        printAll(s, k + 1);                 // explore
        std::swap(s[k], s[i]);              // undo
    }
}
```

Question. For "ABC" it prints 6 lines. For "ABB", how many lines, and how many *different* strings?

Answer. 6 lines but only 3 different strings (ABB, BAB, BBA), each printed twice. The code treats the two B's as different people.

Call Tree: `printAll(s, 0)` for `s = "ABC"`

`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.

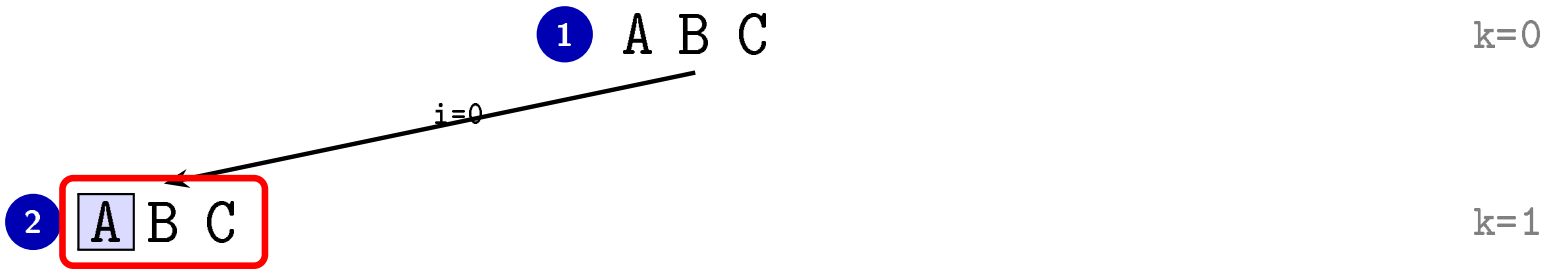


k=0

First call, call 1: $k = 0$, all three seats are free.

Call Tree: `printAll(s, 0)` for `s = "ABC"`

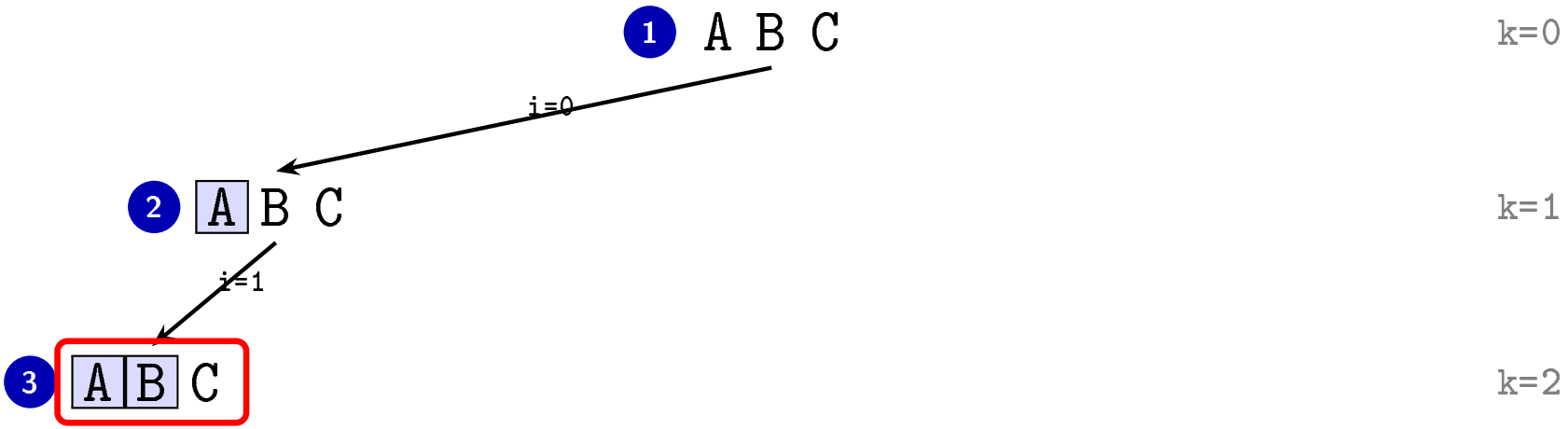
`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.



Call 1 recurses: call 2 fills seat 0 (k=1).

Call Tree: `printAll(s, 0)` for `s = "ABC"`

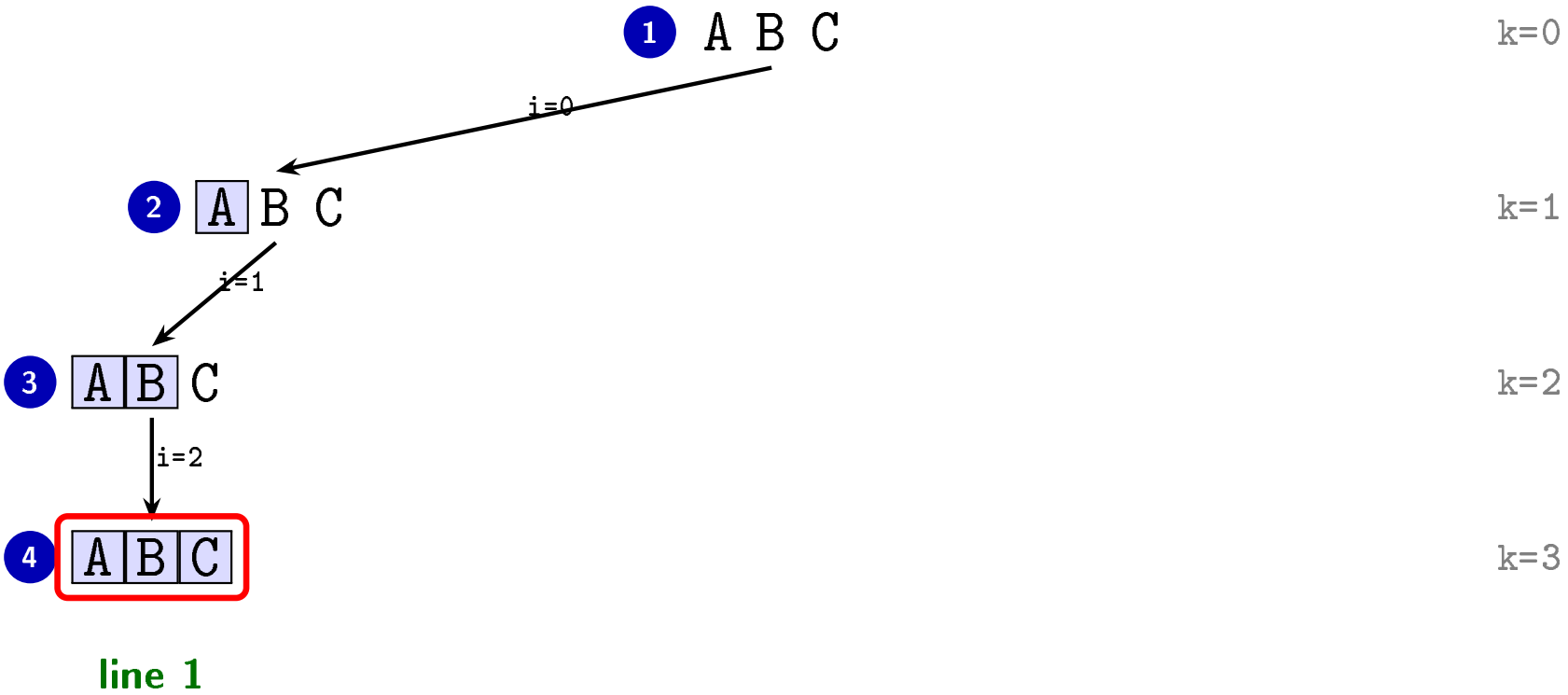
`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.



Call 2 recurses: call 3 fills seat 1 (k=2).

Call Tree: `printAll(s, 0)` for `s = "ABC"`

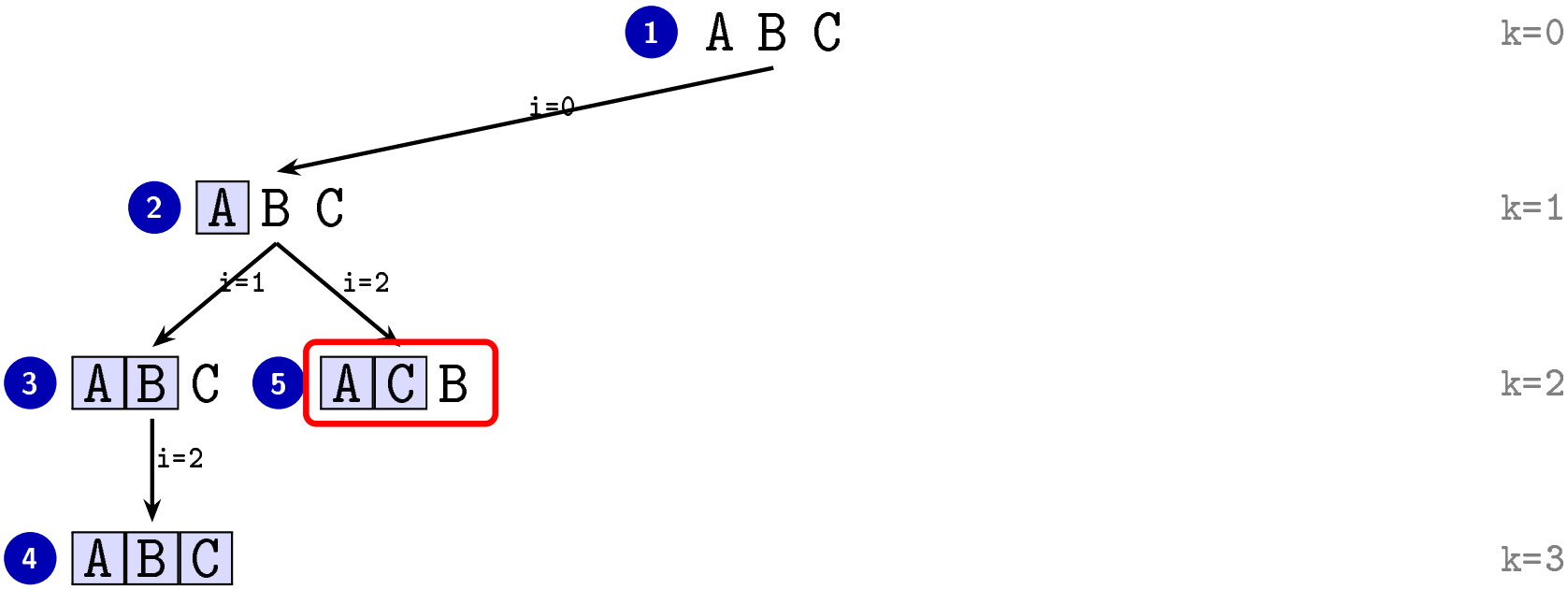
`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.



Call 3 recurses: call 4 fills seat 2 (k=3). All seats occupied: **base case**, print.

Call Tree: `printAll(s, 0)` for `s = "ABC"`

`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.

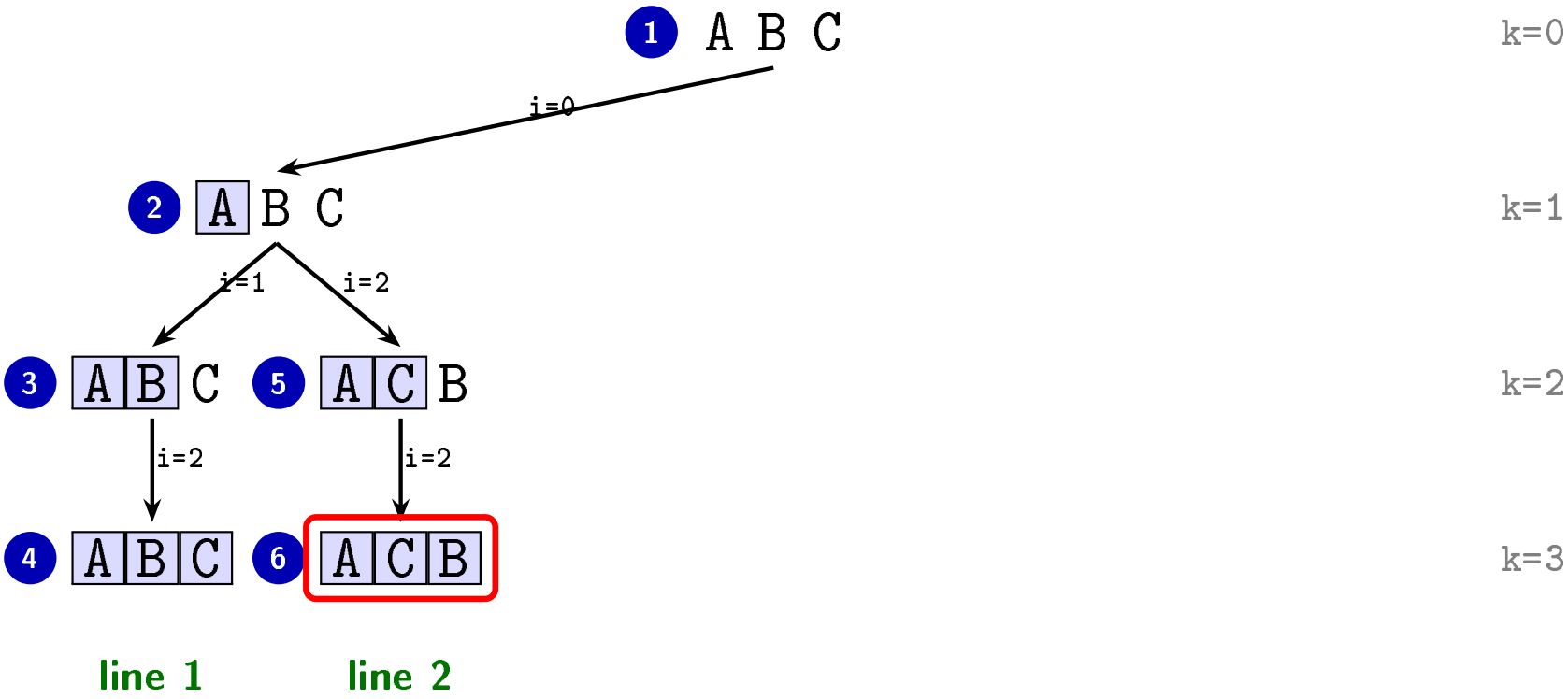


line 1

Backtrack to call 2; its loop moves to the next `i`, which makes call 5.

Call Tree: `printAll(s, 0)` for `s = "ABC"`

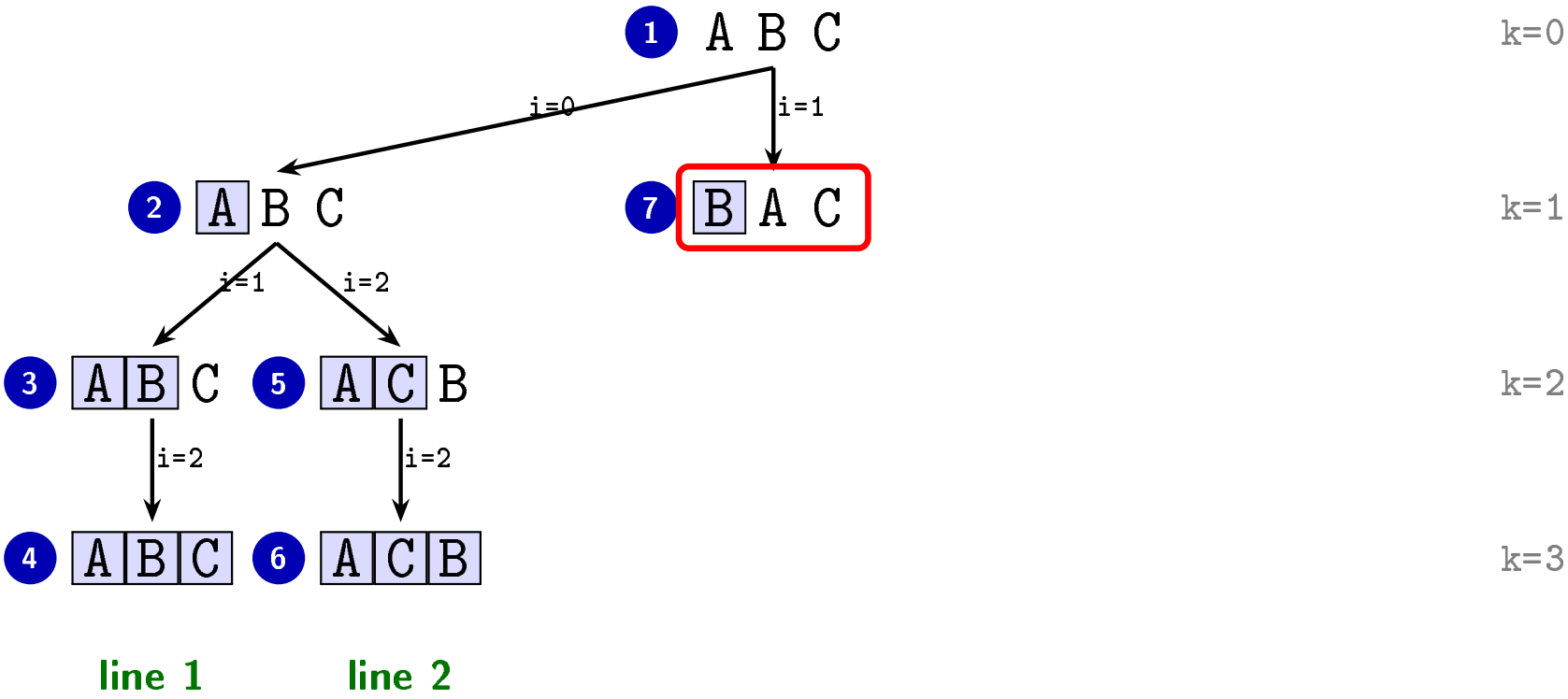
`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.



Call 5 recurses: call 6 fills seat 2 (k=3). All seats occupied: **base case**, print.

Call Tree: `printAll(s, 0)` for `s = "ABC"`

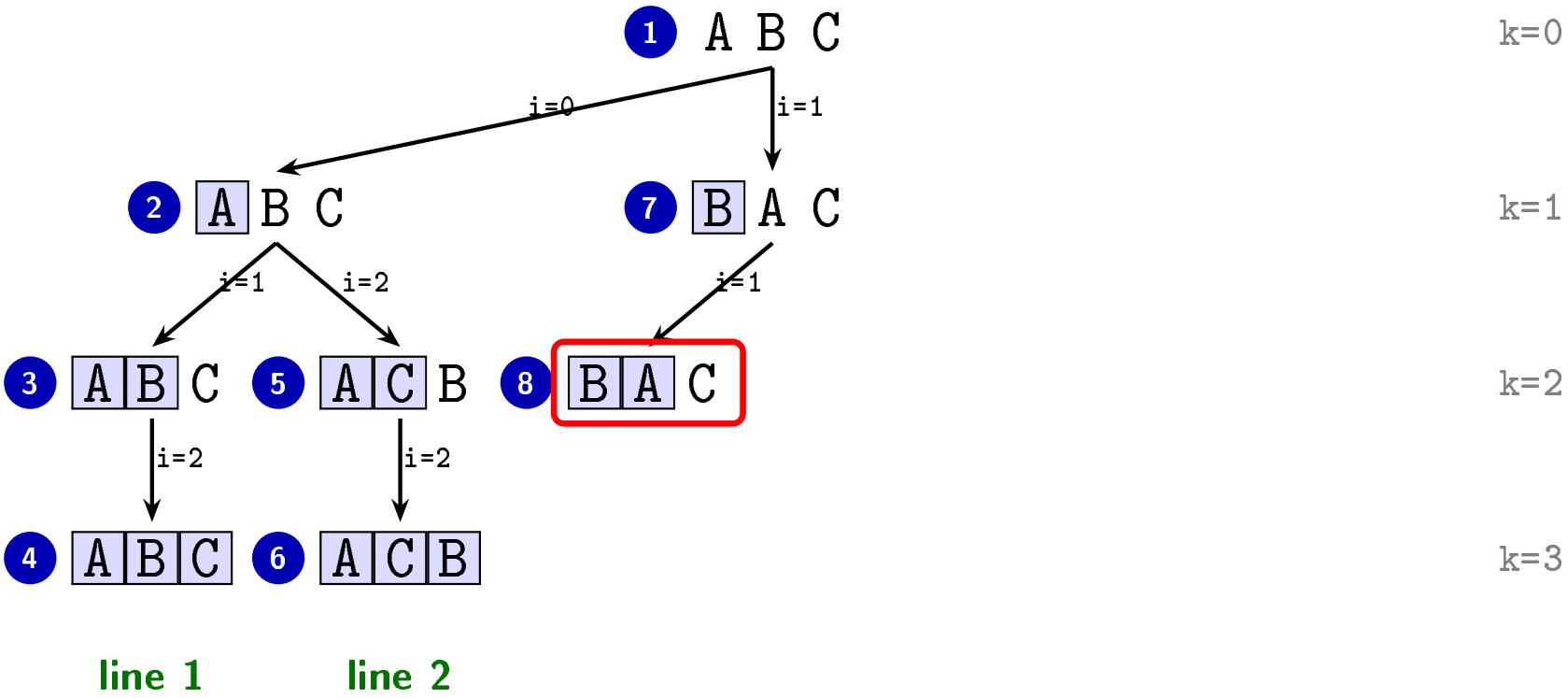
`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.



Backtrack to call 1; its loop moves to the next `i`, which makes call 7.

Call Tree: `printAll(s, 0)` for `s = "ABC"`

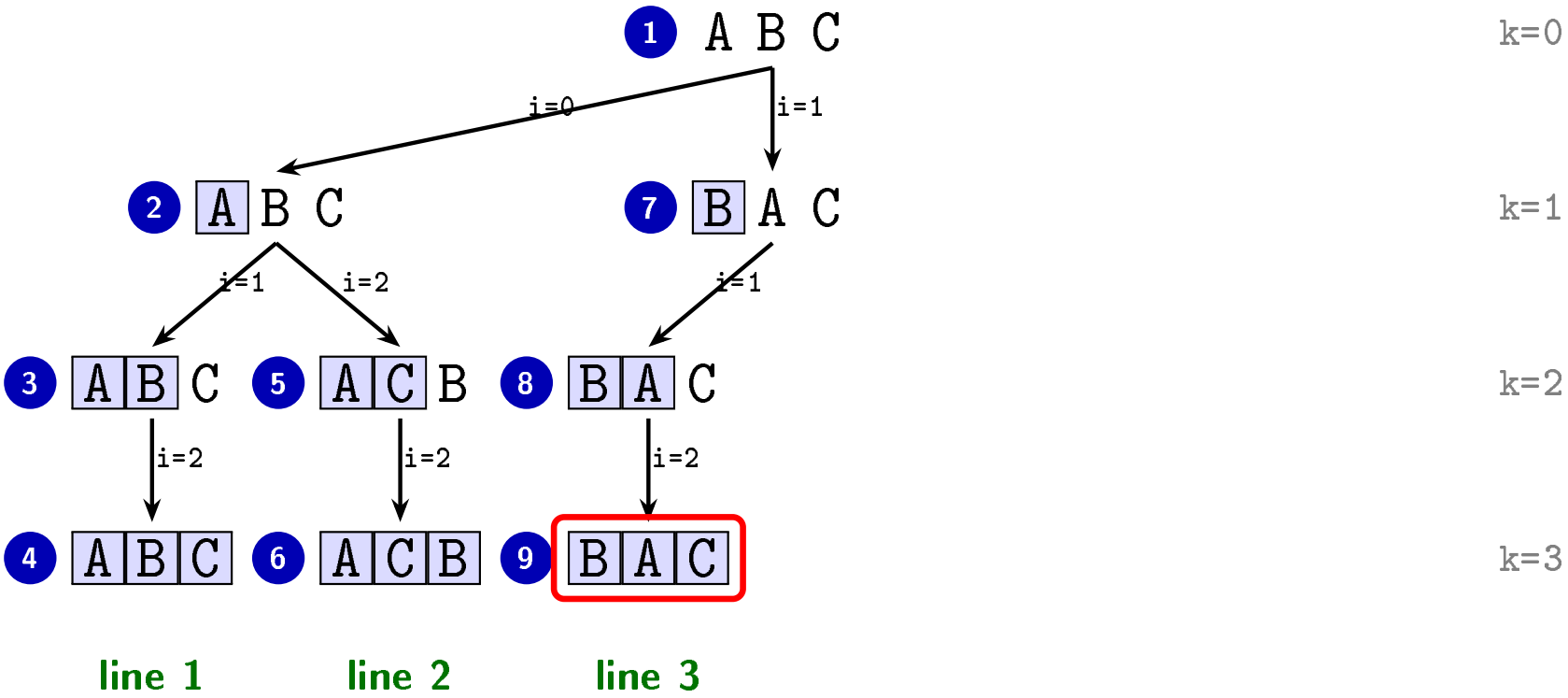
`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.



Call 7 recurses: call 8 fills seat 1 (k=2).

Call Tree: `printAll(s, 0)` for `s = "ABC"`

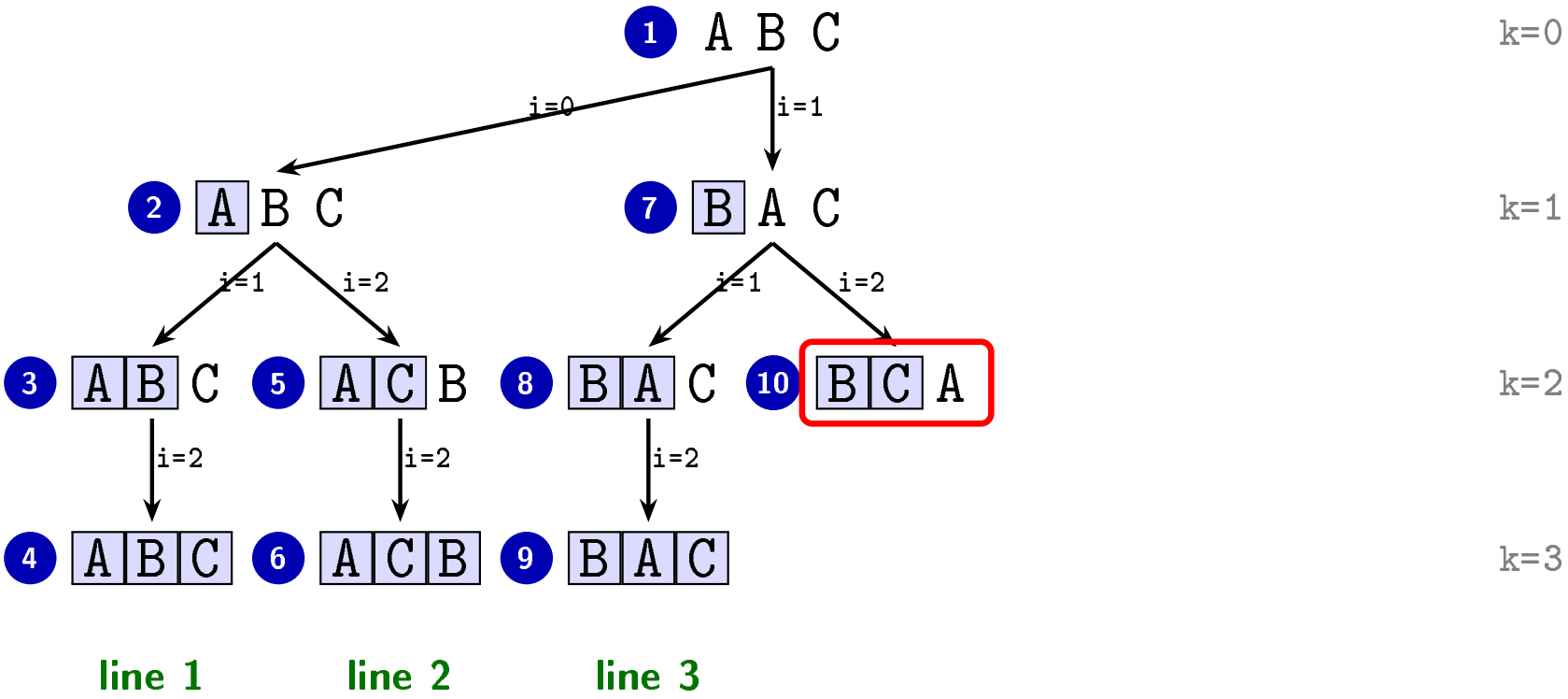
`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.



Call 8 recurses: call 9 fills seat 2 (k=3). All seats occupied: **base case**, print.

Call Tree: `printAll(s, 0)` for `s = "ABC"`

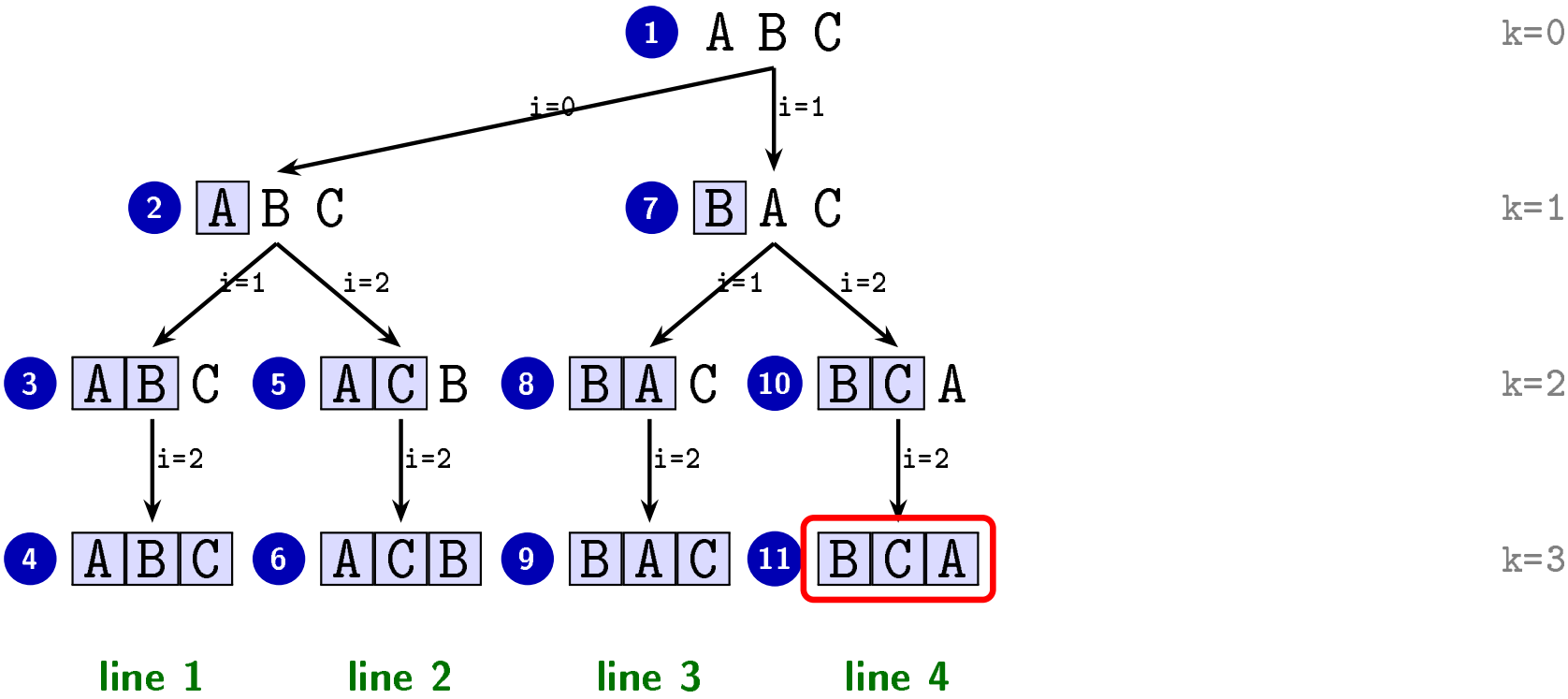
`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.



Backtrack to call 7; its loop moves to the next `i`, which makes call 10.

Call Tree: `printAll(s, 0)` for `s = "ABC"`

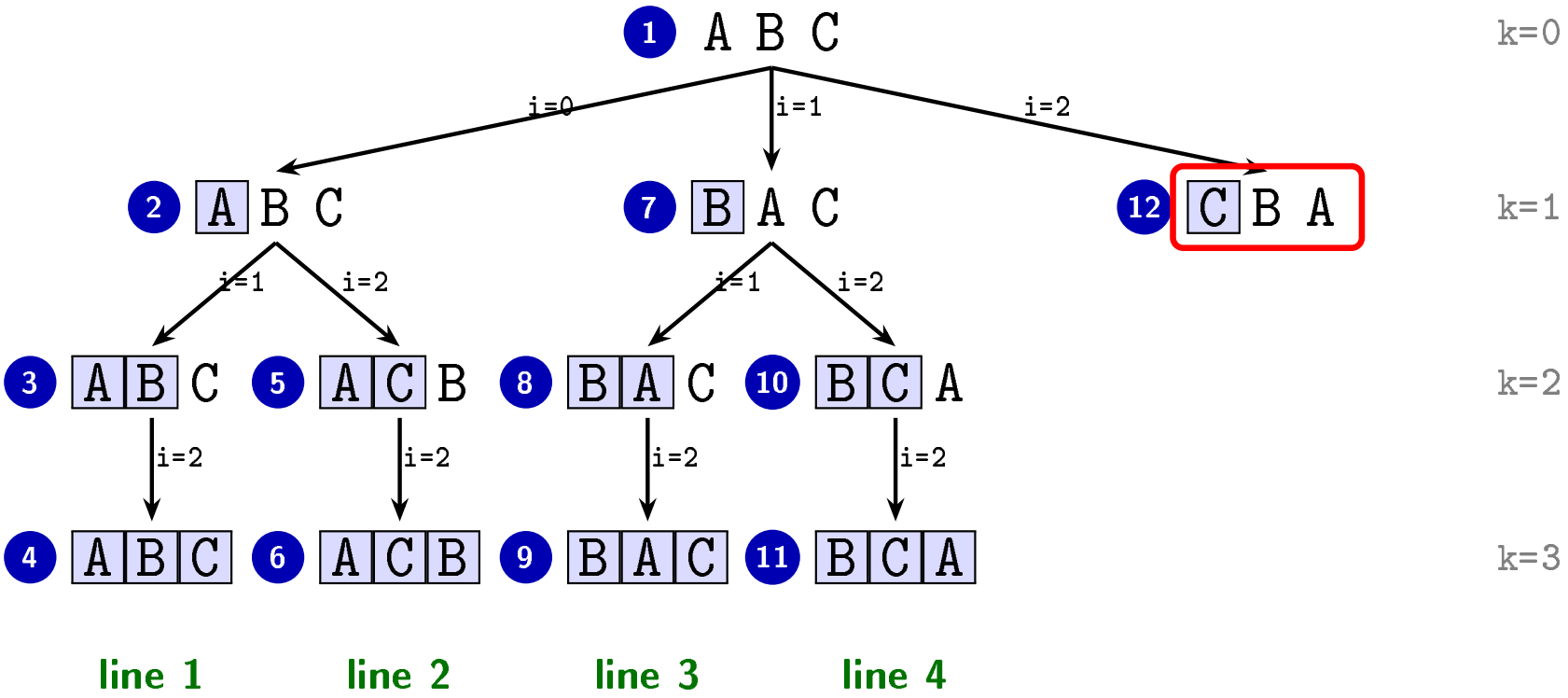
`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.



Call 10 recurses: call 11 fills seat 2 (k=3). All seats occupied: **base case**, print.

Call Tree: `printAll(s, 0)` for `s = "ABC"`

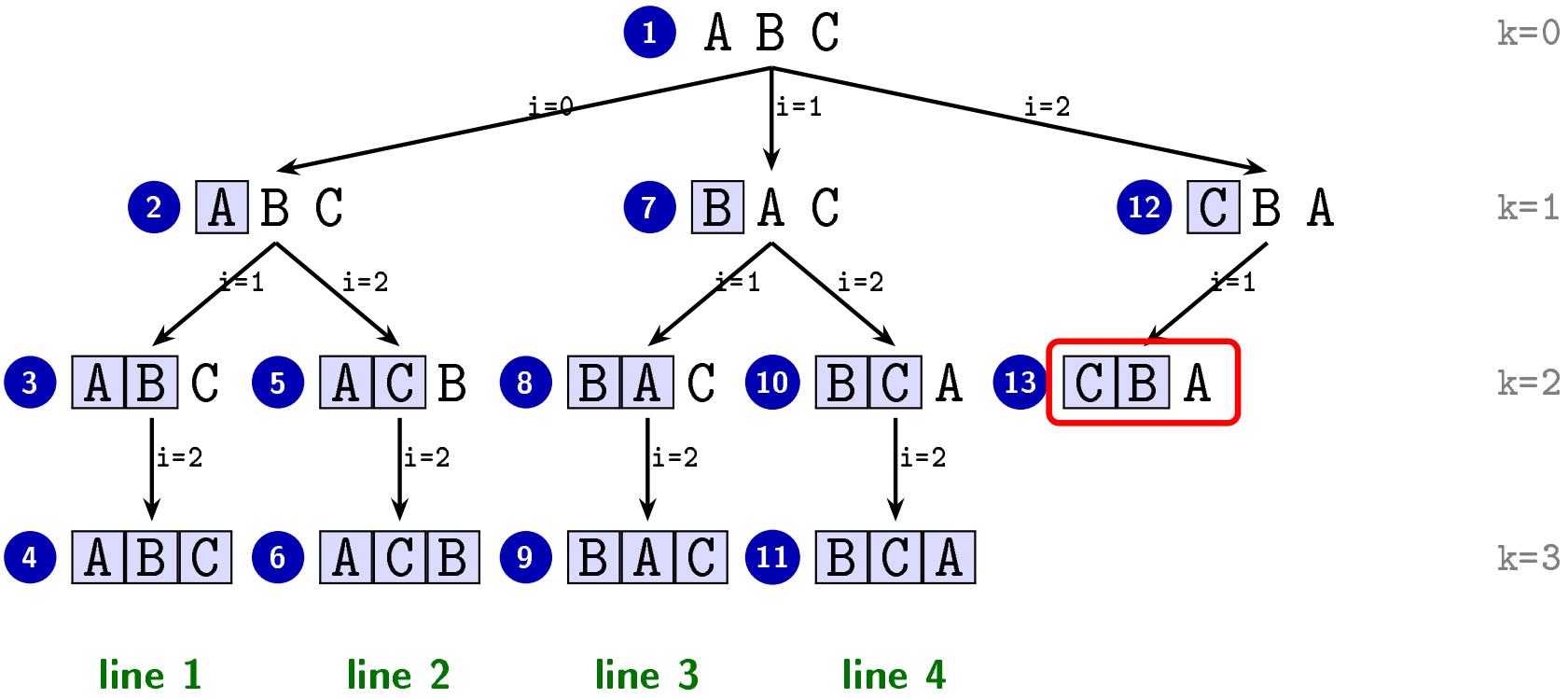
`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.



Backtrack to call 1; its loop moves to the next `i`, which makes call 12.

Call Tree: `printAll(s, 0)` for `s = "ABC"`

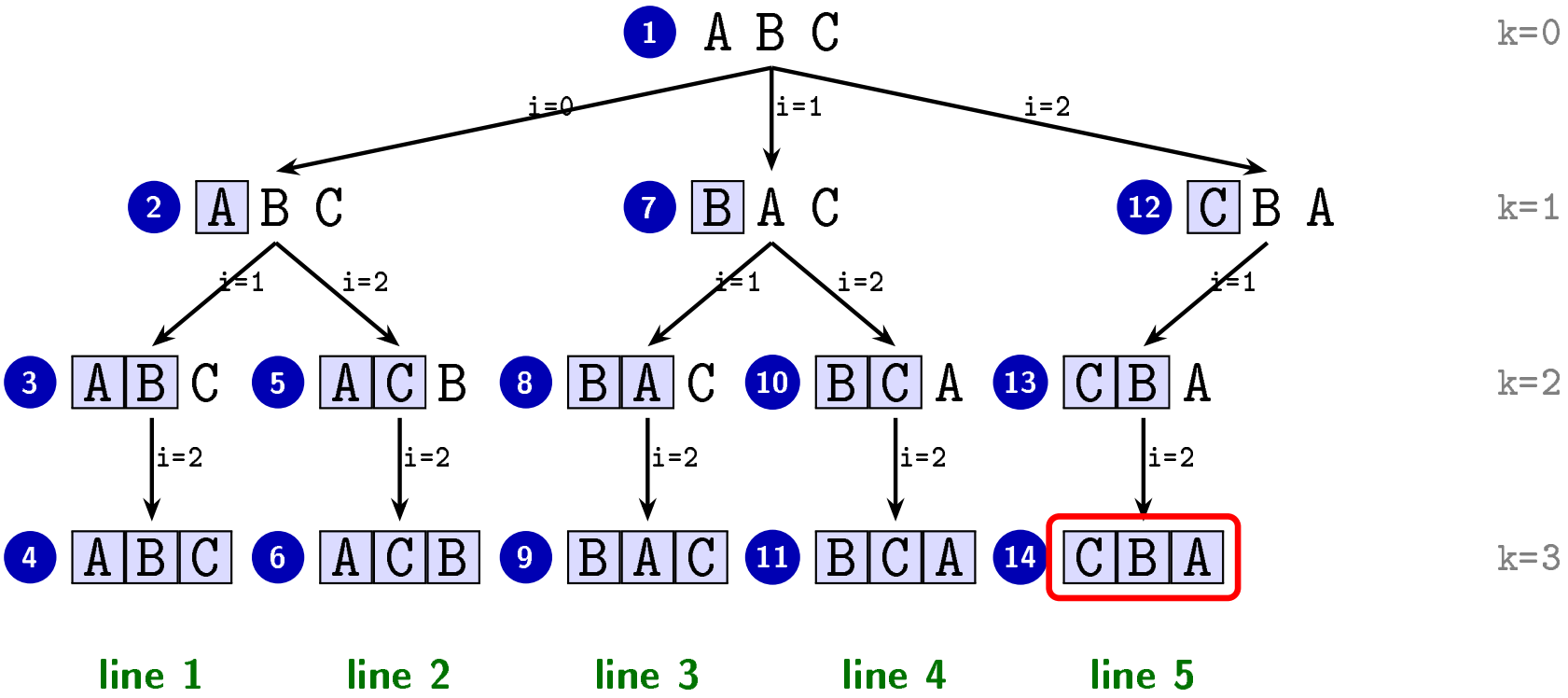
`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.



Call 12 recurses: call 13 fills seat 1 (k=2).

Call Tree: `printAll(s, 0)` for `s = "ABC"`

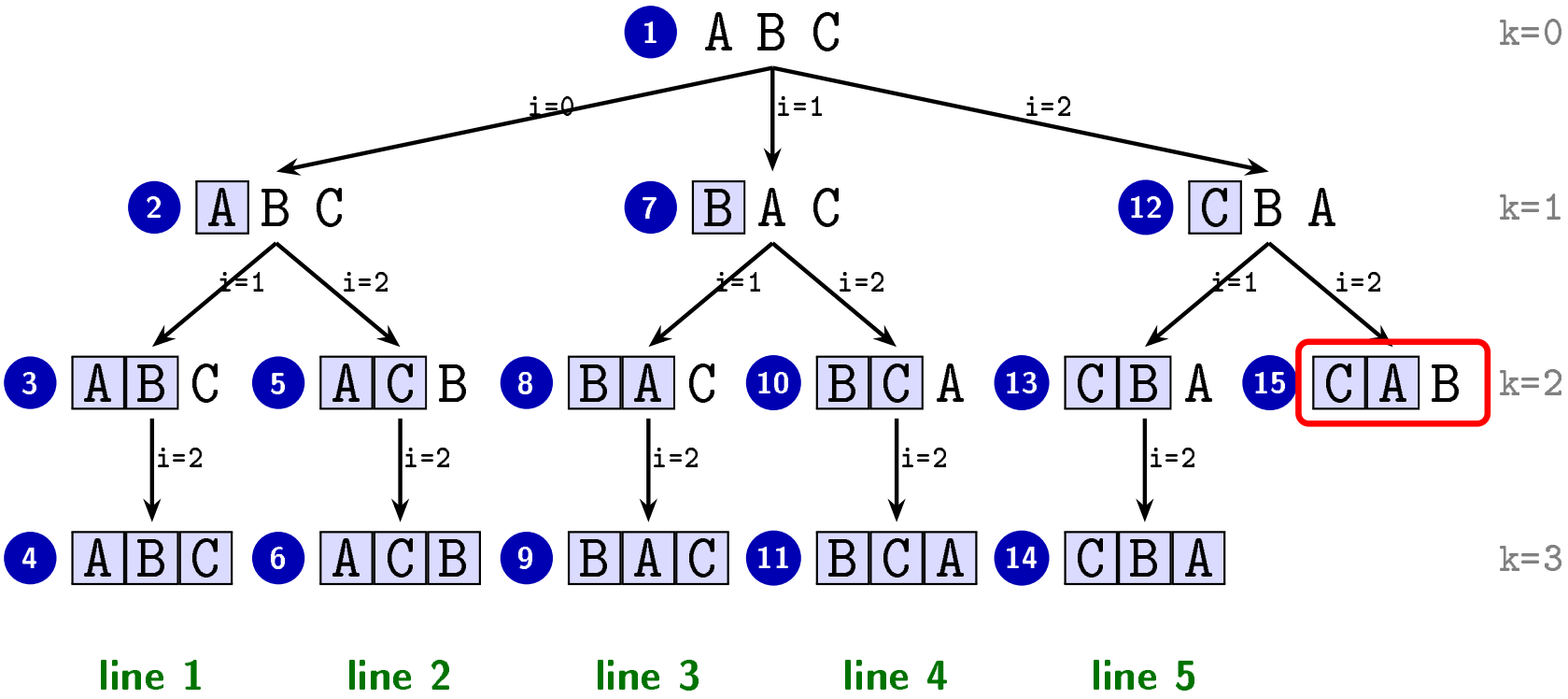
`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.



Call 13 recurses: call 14 fills seat 2 (k=3). All seats occupied: **base case**, print.

Call Tree: `printAll(s, 0)` for `s = "ABC"`

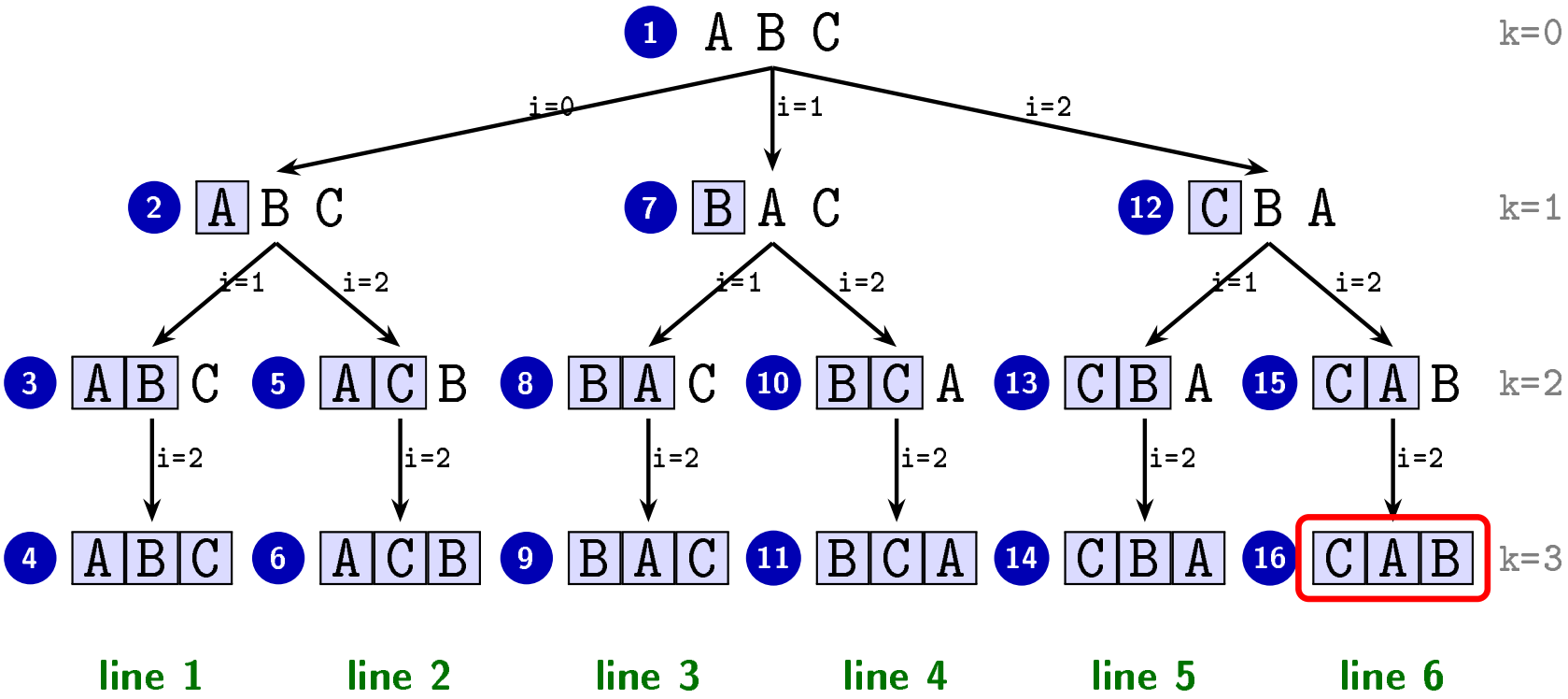
`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.



Backtrack to call 12; its loop moves to the next `i`, which makes call 15.

Call Tree: `printAll(s, 0)` for `s = "ABC"`

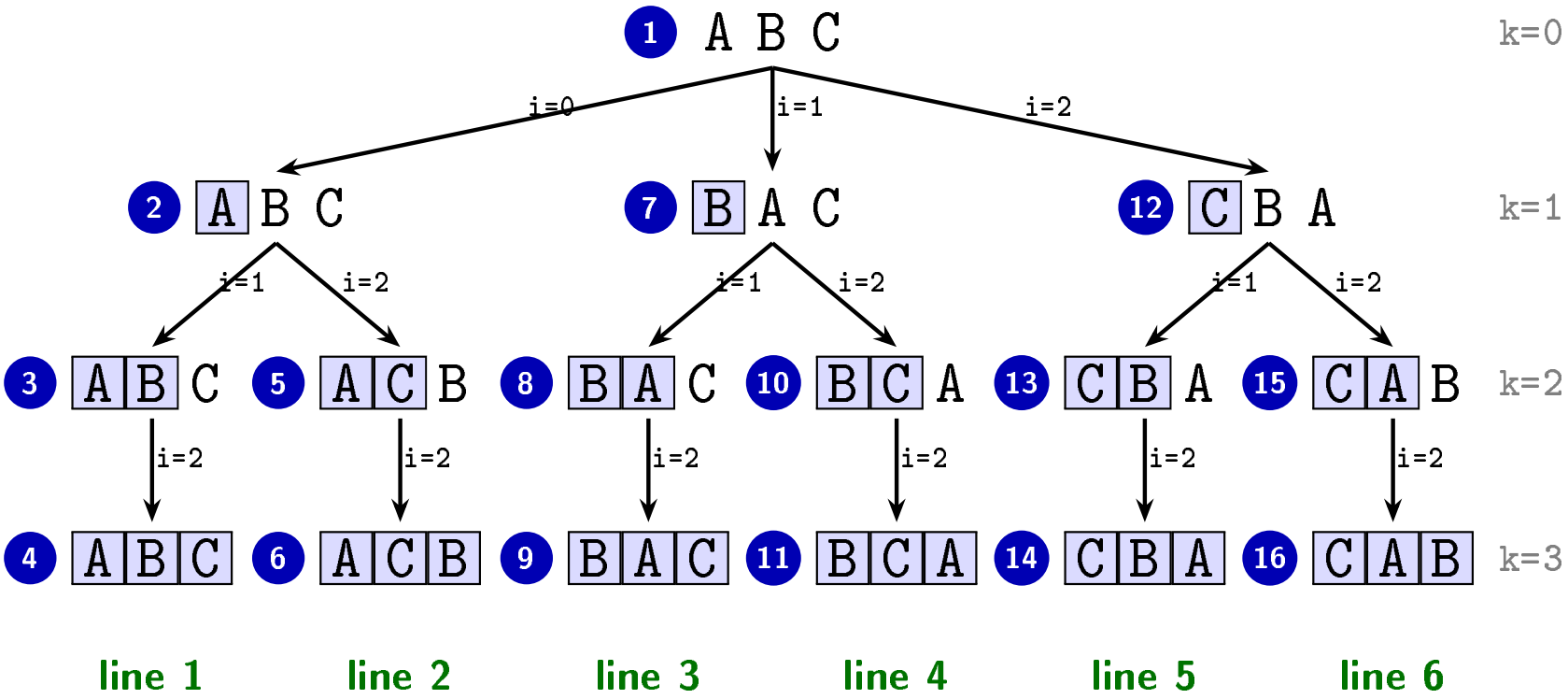
`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.



Call 15 recurses: call 16 fills seat 2 (k=3). All seats occupied: **base case**, print.

Call Tree: `printAll(s, 0)` for `s = "ABC"`

`boxed` = occupied seat (index $< k$). Each node is one call; the **number** is the **order of the call**.



One branch is finished completely before the next one starts (depth first). **6 lines, 16 calls** $(1 + 3 + 6 + 6)$.

Key Ideas of `printUnique`

1. **Why repeats happen.** At one seat, the loop tries the *same letter* twice (two B's), and each try repeats a whole branch of the call tree.

Key Ideas of `printUnique`

1. **Why repeats happen.** At one seat, the loop tries the *same letter* twice (two B's), and each try repeats a whole branch of the call tree.
2. **Memory per seat.** Remember which letters this seat has already tried: a `std::set<char> tried`. A set keeps each element at most once.

Key Ideas of `printUnique`

1. **Why repeats happen.** At one seat, the loop tries the *same letter* twice (two B's), and each try repeats a whole branch of the call tree.
2. **Memory per seat.** Remember which letters this seat has already tried: a `std::set<char> tried`. A set keeps each element at most once.
3. **One set per call.** Declare it inside the function, so every seat starts with an empty set.

Key Ideas of `printUnique`

1. **Why repeats happen.** At one seat, the loop tries the *same letter* twice (two B's), and each try repeats a whole branch of the call tree.
2. **Memory per seat.** Remember which letters this seat has already tried: a `std::set<char> tried`. A set keeps each element at most once.
3. **One set per call.** Declare it inside the function, so every seat starts with an empty set.
4. **Skip.** If the letter was already tried at this seat, `continue`: skip the whole branch. This is **pruning**: a branch that can only repeat strings we have printed.

Key Ideas of `printUnique`

1. **Why repeats happen.** At one seat, the loop tries the *same letter* twice (two B's), and each try repeats a whole branch of the call tree.
2. **Memory per seat.** Remember which letters this seat has already tried: a `std::set<char> tried`. A set keeps each element at most once.
3. **One set per call.** Declare it inside the function, so every seat starts with an empty set.
4. **Skip.** If the letter was already tried at this seat, `continue`: skip the whole branch. This is **pruning**: a branch that can only repeat strings we have printed.
5. **Everything else is unchanged:** choose, explore, undo.

Key Ideas of `printUnique`

1. **Why repeats happen.** At one seat, the loop tries the *same letter* twice (two B's), and each try repeats a whole branch of the call tree.
2. **Memory per seat.** Remember which letters this seat has already tried: a `std::set<char> tried`. A set keeps each element at most once.
3. **One set per call.** Declare it inside the function, so every seat starts with an empty set.
4. **Skip.** If the letter was already tried at this seat, `continue`: skip the whole branch. This is **pruning**: a branch that can only repeat strings we have printed.
5. **Everything else is unchanged:** choose, explore, undo.

Tool. `tried.insert(x)` returns a pair; `.second` is `true` if `x` was new and `false` if it was already there. Now **you** add two lines to `printAll`.

Write It Yourself: `printUnique`

Step 1 of 2: Remember what this seat has tried

Hint. Should the memory be shared by all calls, or belong to one call (one seat)? Which type holds letters with no repeats?

```
void printUnique(std::string& s, int k) {
    if (k == static_cast<int>(s.size())) {
        std::cout << s << "\n";
        return;
    }
    // ??? your turn
    for (int i = k; i < static_cast<int>(s.size()); ++i) {
        std::swap(s[k], s[i]);  // choose
        printUnique(s, k + 1);  // explore
        std::swap(s[k], s[i]);  // undo
    }
}
```

Write It Yourself: `printUnique`

Step 1 of 2: Remember what this seat has tried

Hint. Should the memory be shared by all calls, or belong to one call (one seat)? Which type holds letters with no repeats?

Why. A `std::set<char>` declared **inside** the function, before the loop, so every call starts empty. A global set would remember letters from other seats and wrongly skip good branches.

```
void printUnique(std::string& s, int k) {
    if (k == static_cast<int>(s.size())) {
        std::cout << s << "\n";
        return;
    }
    std::set<char> tried;       // letters already tried at this seat
    for (int i = k; i < static_cast<int>(s.size()); ++i) {
        std::swap(s[k], s[i]);  // choose
        printUnique(s, k + 1);  // explore
        std::swap(s[k], s[i]);  // undo
    }
}
```

Write It Yourself: `printUnique`

Step 2 of 2: Skip a repeated letter

Hint. `insert` returns a pair whose `.second` is `true` when the letter was new. When do we skip? Inside the loop, before the choose.

```
void printUnique(std::string& s, int k) {
    if (k == static_cast<int>(s.size())) {
        std::cout << s << "\n";
        return;
    }
    std::set<char> tried;       // letters already tried at this seat
    for (int i = k; i < static_cast<int>(s.size()); ++i) {
        // ??? your turn
        std::swap(s[k], s[i]);  // choose
        printUnique(s, k + 1);  // explore
        std::swap(s[k], s[i]);  // undo
    }
}
```

Write It Yourself: `printUnique`

Step 2 of 2: Skip a repeated letter

Hint. `insert` returns a pair whose `.second` is `true` when the letter was new. When do we skip? Inside the loop, before the choose.

Why. `if (!tried.insert(s[i]).second) { continue; }`: a letter already tried at this seat would repeat a whole branch. The first time a letter appears, `insert` adds it and the loop goes on.

```
void printUnique(std::string& s, int k) {
    if (k == static_cast<int>(s.size())) {
        std::cout << s << "\n";
        return;
    }
    std::set<char> tried;       // letters already tried at this seat
    for (int i = k; i < static_cast<int>(s.size()); ++i) {
        if (!tried.insert(s[i]).second) { continue; } // same letter: skip
        std::swap(s[k], s[i]);  // choose
        printUnique(s, k + 1);  // explore
        std::swap(s[k], s[i]);  // undo
    }
}
```

Another Way to Write `printUnique`

Both versions do the same thing: **skip a letter this seat has already tried**.

Version 1: skip early

```
std::set<char> tried;
for (int i = k; i < len; ++i) {
  if (!tried.insert(s[i]).second) {
    continue;
  }
  std::swap(s[k], s[i]);
  printUnique(s, k + 1);
  std::swap(s[k], s[i]);
}
```

Version 2: try only new letters

```
std::set<char> tried;
for (int i = k; i < len; ++i) {
  if (tried.count(s[i]) == 0) {
    tried.insert(s[i]);
    std::swap(s[k], s[i]);
    printUnique(s, k + 1);
    std::swap(s[k], s[i]);
  }
}
```

Another Way to Write `printUnique`

Both versions do the same thing: **skip a letter this seat has already tried**.

```
if (!tried.insert(s[i]).second) { continue; }
```

What does `!tried.insert(s[i]).second` mean?

1. `tried.insert(s[i])` tries to add the letter to the set. It returns a pair: `(position, bool)`.
2. `.second` is that `bool`: `true` if the letter was **new** (added), `false` if it was **already there** (nothing added).
3. `!` flips it, so the `if` is true exactly when the letter is a **repeat**: then `continue` skips the branch.

Letter at this seat	`.second`	`!.second`
first `B`	`true` (added)	`false`: go on
second `B`	`false` (already in)	`true`: `continue`

Another Way to Write `printUnique`

Both versions do the same thing: **skip a letter this seat has already tried**.

Version 1: skip early

```
std::set<char> tried;
for (int i = k; i < len; ++i) {
  if (!tried.insert(s[i]).second) {
    continue;
  }
  std::swap(s[k], s[i]);
  printUnique(s, k + 1);
  std::swap(s[k], s[i]);
}
```

Version 2: try only new letters

```
std::set<char> tried;
for (int i = k; i < len; ++i) {
  if (tried.count(s[i]) == 0) {
    tried.insert(s[i]);
    std::swap(s[k], s[i]);
    printUnique(s, k + 1);
    std::swap(s[k], s[i]);
  }
}
```

Version 2 reads in two steps: `count(s[i]) == 0` asks "have we tried this letter at this seat?" (0 means no), and `insert` records it. Version 1 asks and records in one line: `insert` returns `.second == false` when the letter was already there.

Another Way to Write `printUnique`

Both versions do the same thing: **skip a letter this seat has already tried**.

Version 1: skip early

```
std::set<char> tried;
for (int i = k; i < len; ++i) {
  if (!tried.insert(s[i]).second) {
    continue;
  }
  std::swap(s[k], s[i]);
  printUnique(s, k + 1);
  std::swap(s[k], s[i]);
}
```

Version 2: try only new letters

```
std::set<char> tried;
for (int i = k; i < len; ++i) {
  if (tried.count(s[i]) == 0) {
    tried.insert(s[i]);
    std::swap(s[k], s[i]);
    printUnique(s, k + 1);
    std::swap(s[k], s[i]);
  }
}
```

Version 2 reads in two steps: `count(s[i]) == 0` asks "have we tried this letter at this seat?" (0 means no), and `insert` records it. Version 1 asks and records in one line: `insert` returns `.second == false` when the letter was already there.

Which should you write? Either one. Version 2 is easier to read aloud; Version 1 is shorter and keeps the loop body flat. In both, the choose, explore, and undo lines are unchanged.

Call Tree: `printUnique(s, 0)` for `s = "ABB"`

Boxed = occupied seat. **Number** = order of the call. A dashed red node is a **skipped** branch: never called.

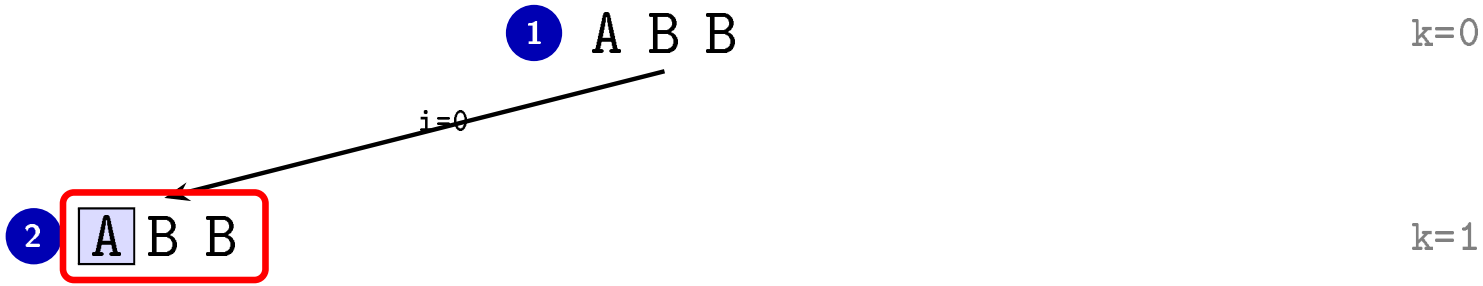


`k=0`

First call, call 1: $k = 0$. `tried` is empty.

Call Tree: `printUnique(s, 0)` for `s = "ABB"`

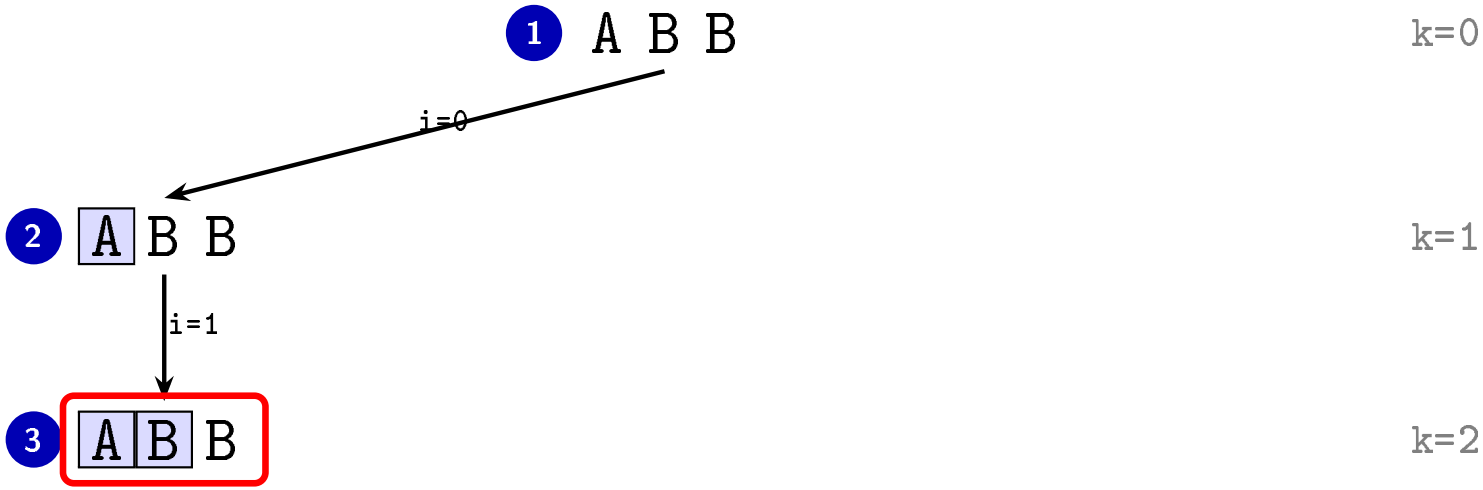
Boxed = occupied seat. **Number** = order of the call. A dashed red node is a **skipped** branch: never called.



Call 1 recurses: call 2 fills seat 0 (`k=1`).

Call Tree: `printUnique(s, 0)` for `s = "ABB"`

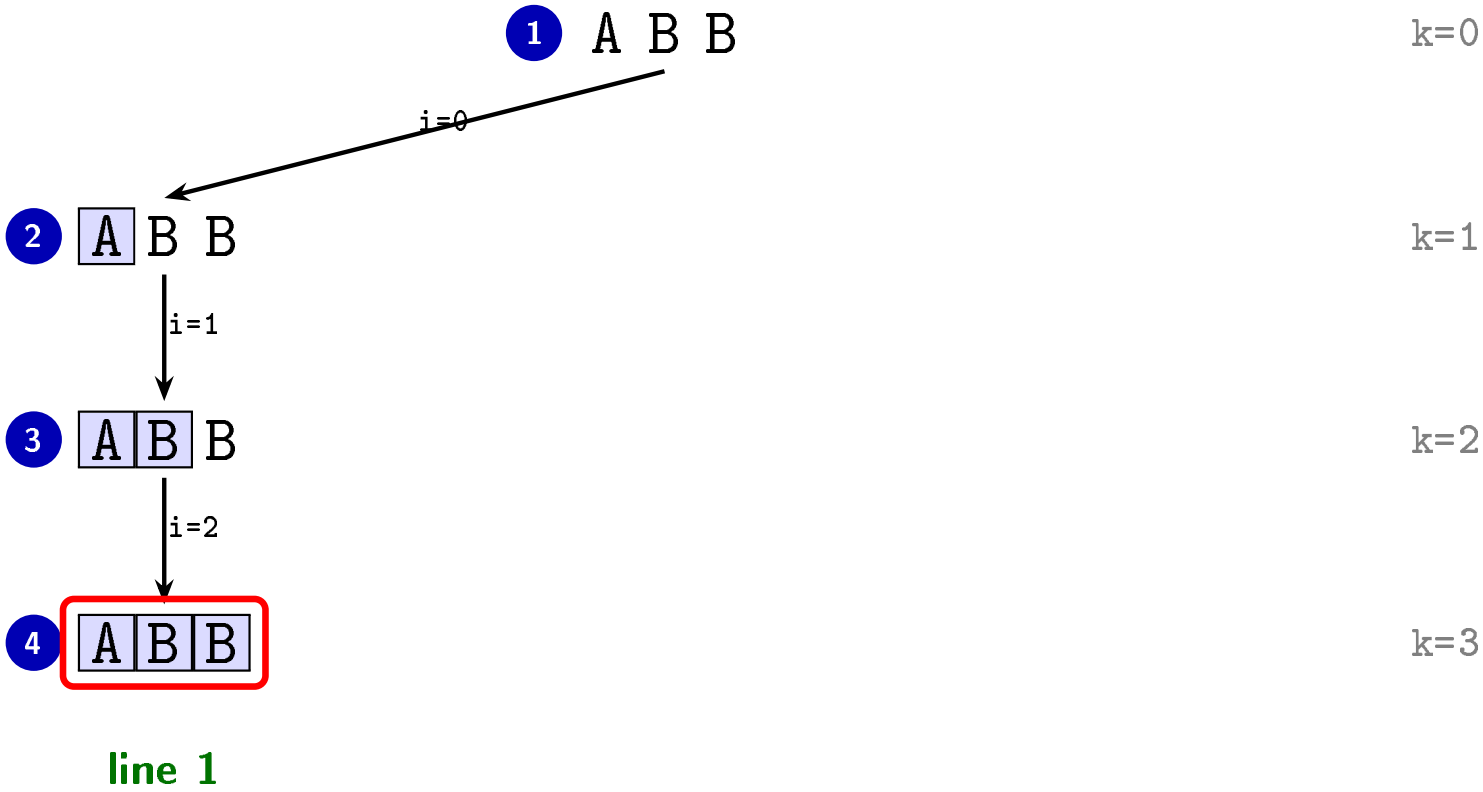
Boxed = occupied seat. **Number** = order of the call. A dashed red node is a **skipped** branch: never called.



Call 2 recurses: call 3 fills seat 1 (k=2).

Call Tree: `printUnique(s, 0)` for `s = "ABB"`

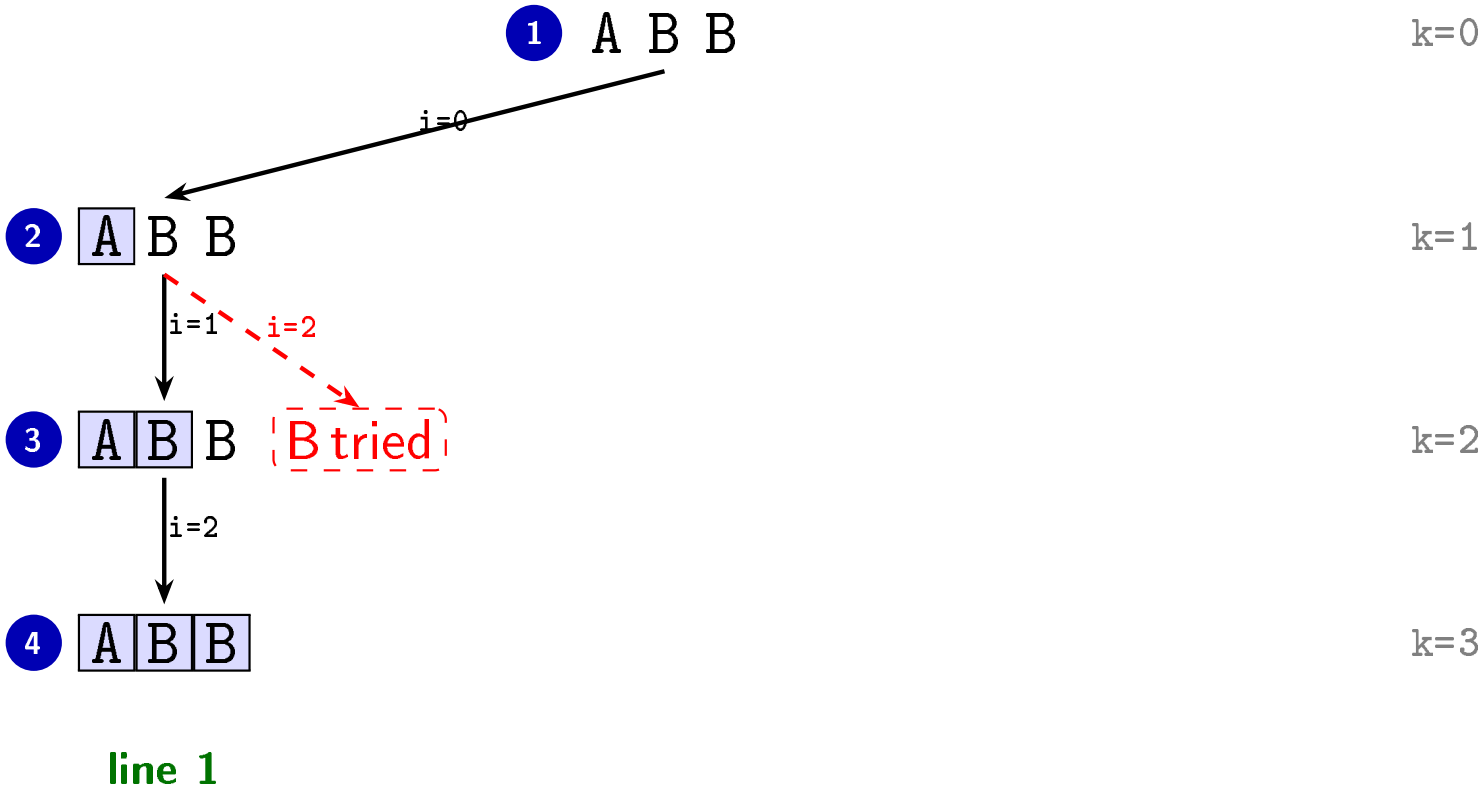
Boxed = occupied seat. **Number** = order of the call. A dashed red node is a **skipped** branch: never called.



Call 3 recurses: call 4 fills seat 2 (k=3). Base case, print.

Call Tree: `printUnique(s, 0)` for `s = "ABB"`

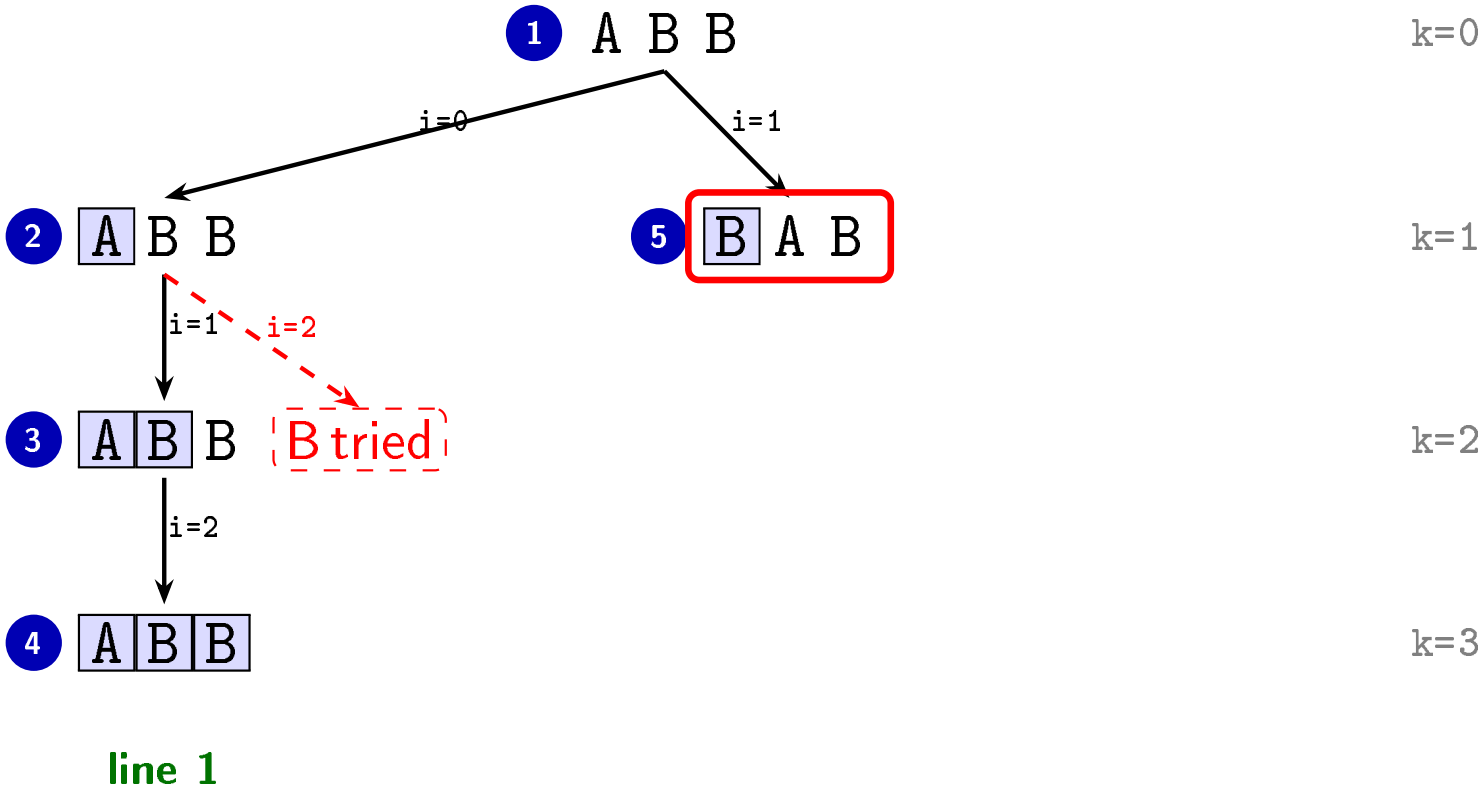
Boxed = occupied seat. **Number** = order of the call. A dashed red node is a **skipped** branch: never called.



Back in call 2: `i=2` is `B`, already in `tried`. **Skip**: no call is made.

Call Tree: `printUnique(s, 0)` for `s = "ABB"`

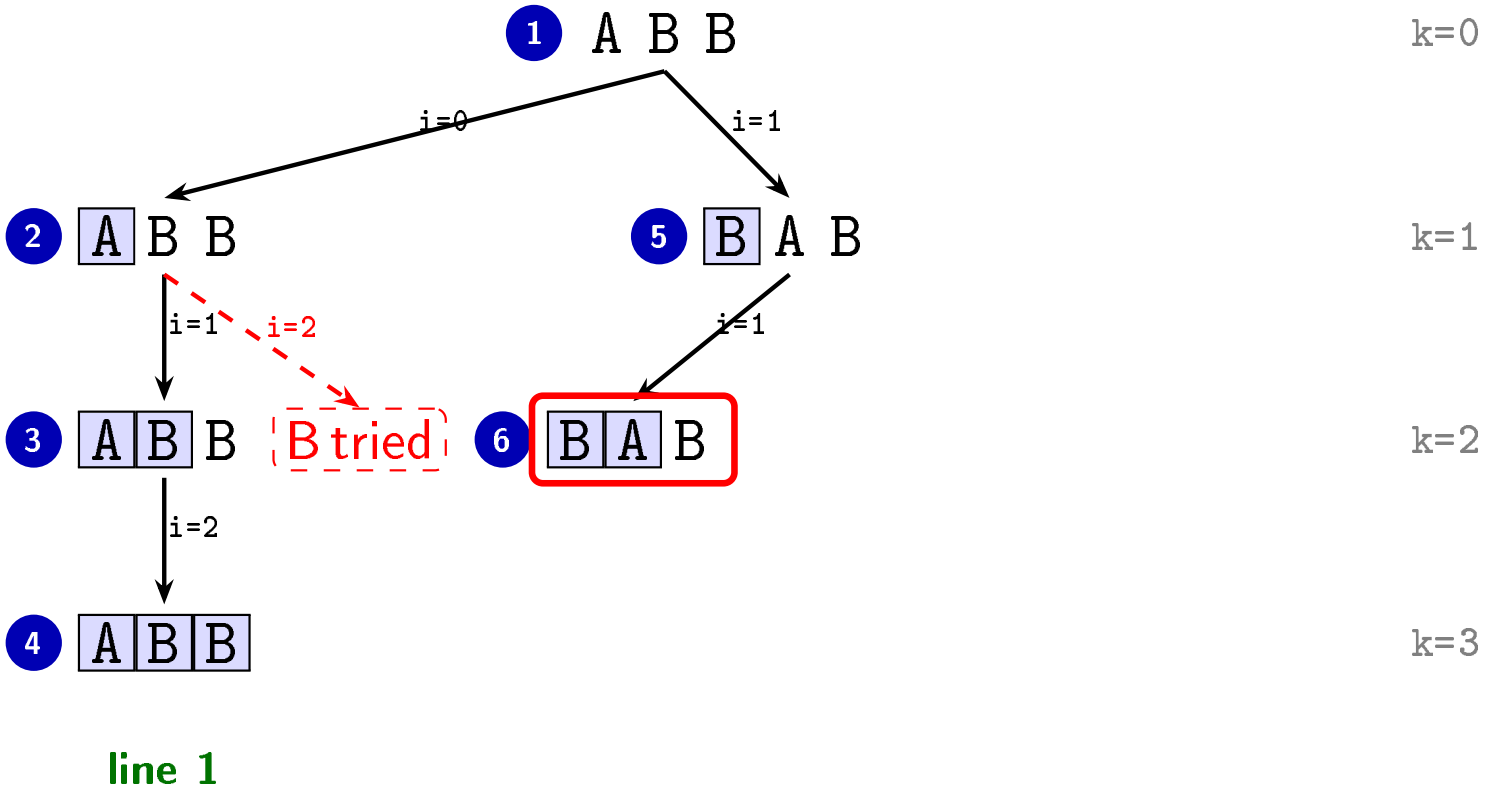
Boxed = occupied seat. **Number** = order of the call. A dashed red node is a **skipped** branch: never called.



Backtrack to call 1; its loop moves to the next `i`, which makes call 5.

Call Tree: `printUnique(s, 0)` for `s = "ABB"`

Boxed = occupied seat. **Number** = order of the call. A dashed red node is a **skipped** branch: never called.

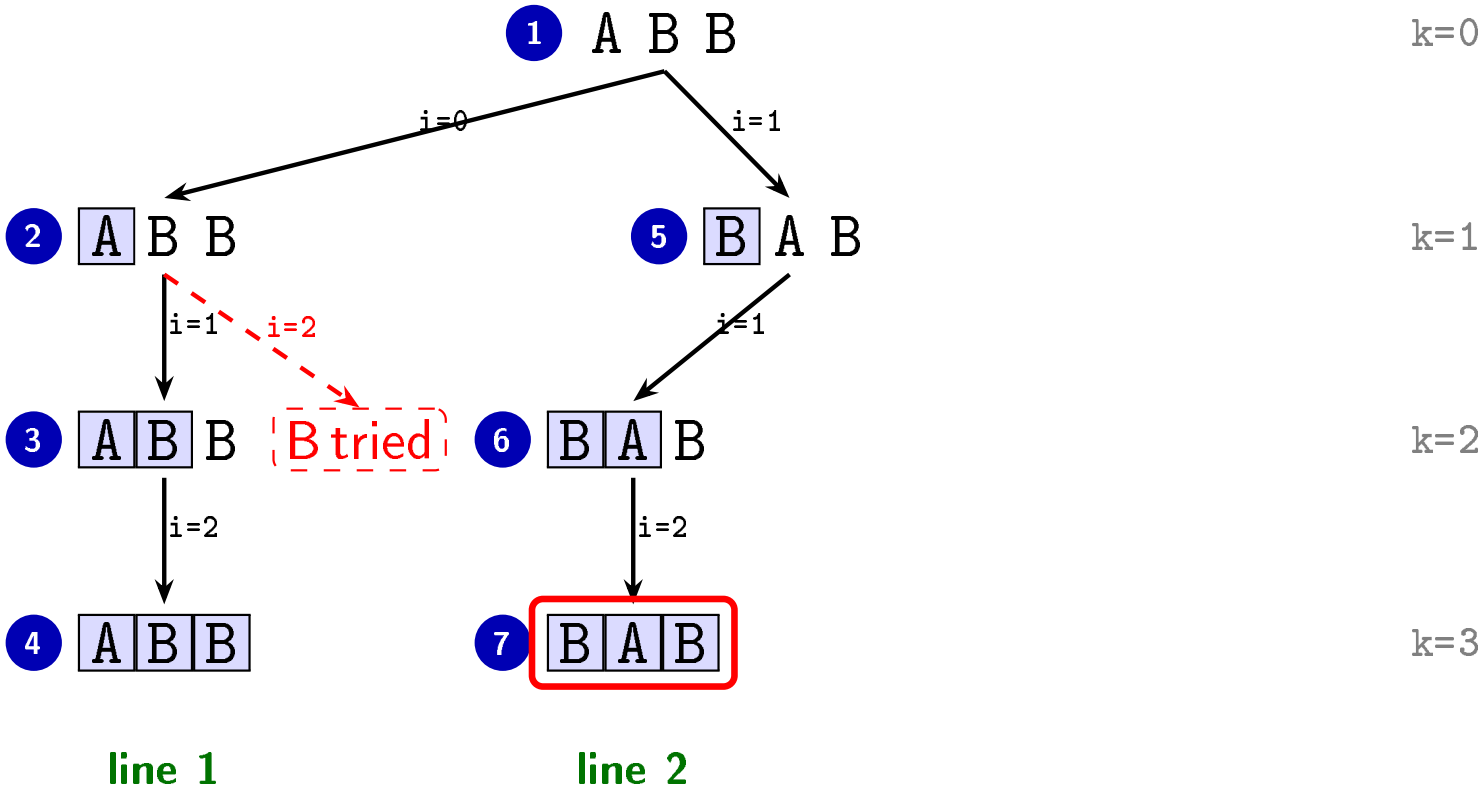


line 1

Call 5 recurses: call 6 fills seat 1 (k=2).

Call Tree: `printUnique(s, 0)` for `s = "ABB"`

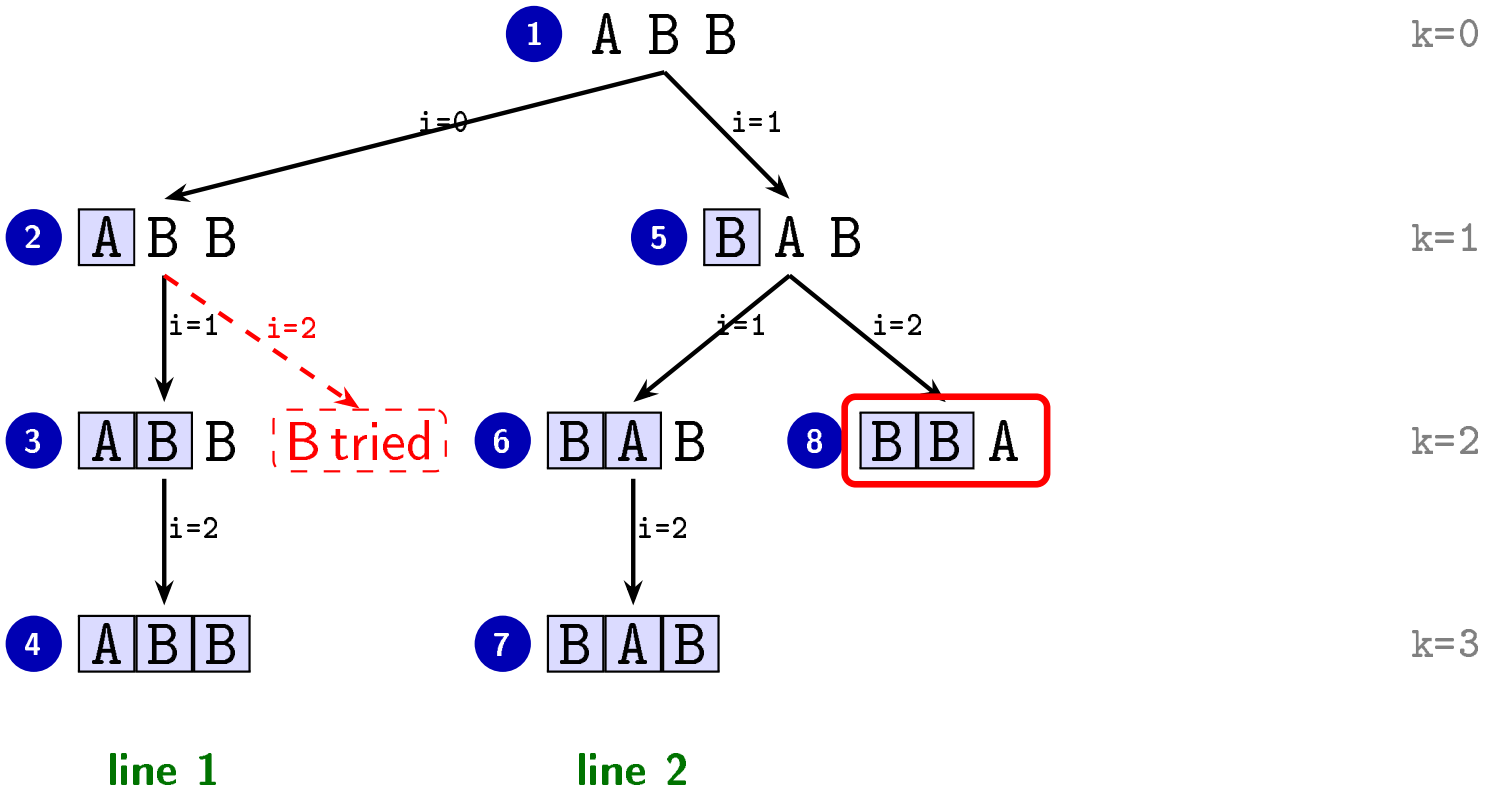
Boxed = occupied seat. **Number** = order of the call. A dashed red node is a **skipped** branch: never called.



Call 6 recurses: call 7 fills seat 2 (k=3). Base case, print.

Call Tree: `printUnique(s, 0)` for `s = "ABB"`

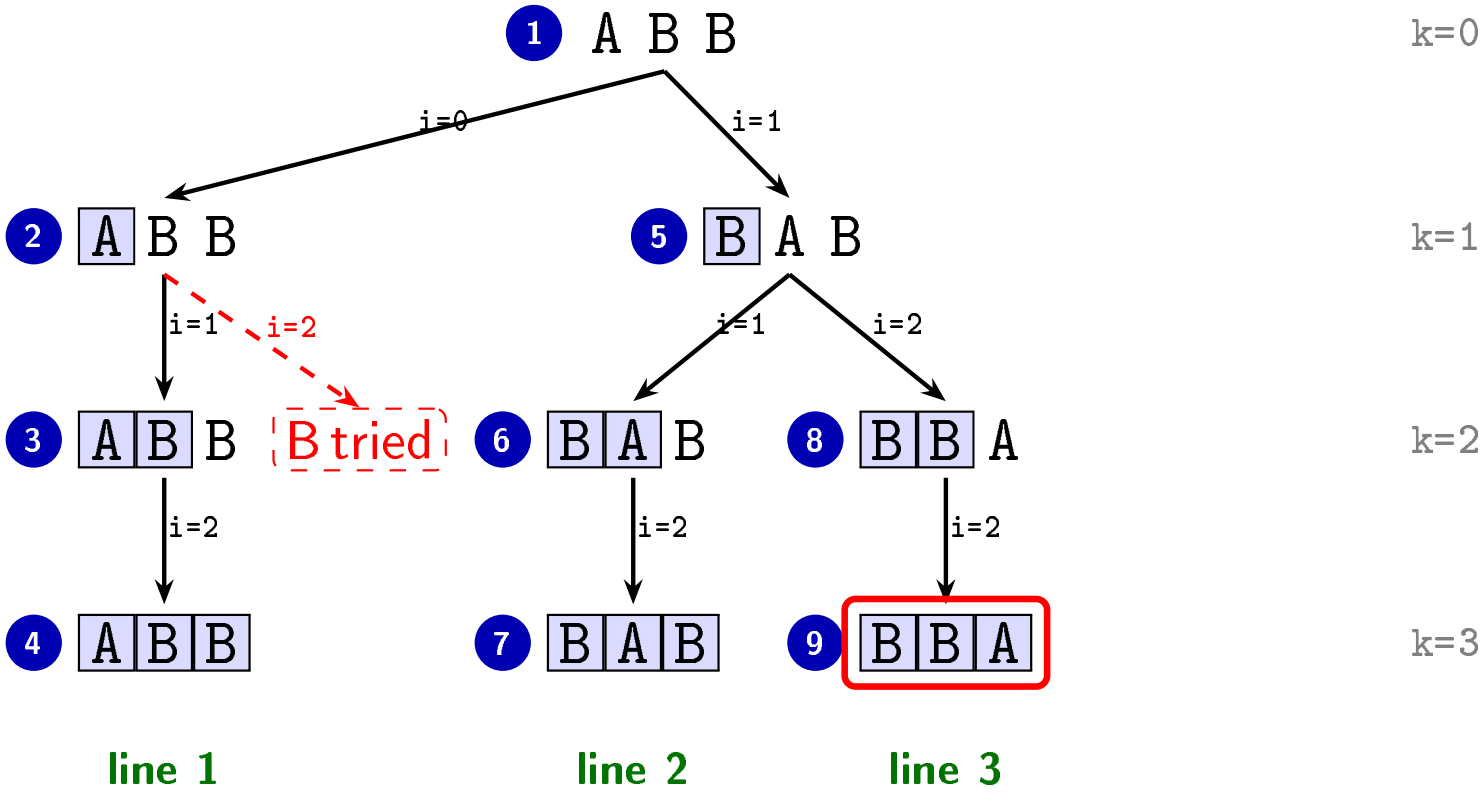
Boxed = occupied seat. **Number** = order of the call. A dashed red node is a **skipped** branch: never called.



Backtrack to call 5; its loop moves to the next `i`, which makes call 8.

Call Tree: `printUnique(s, 0)` for `s = "ABB"`

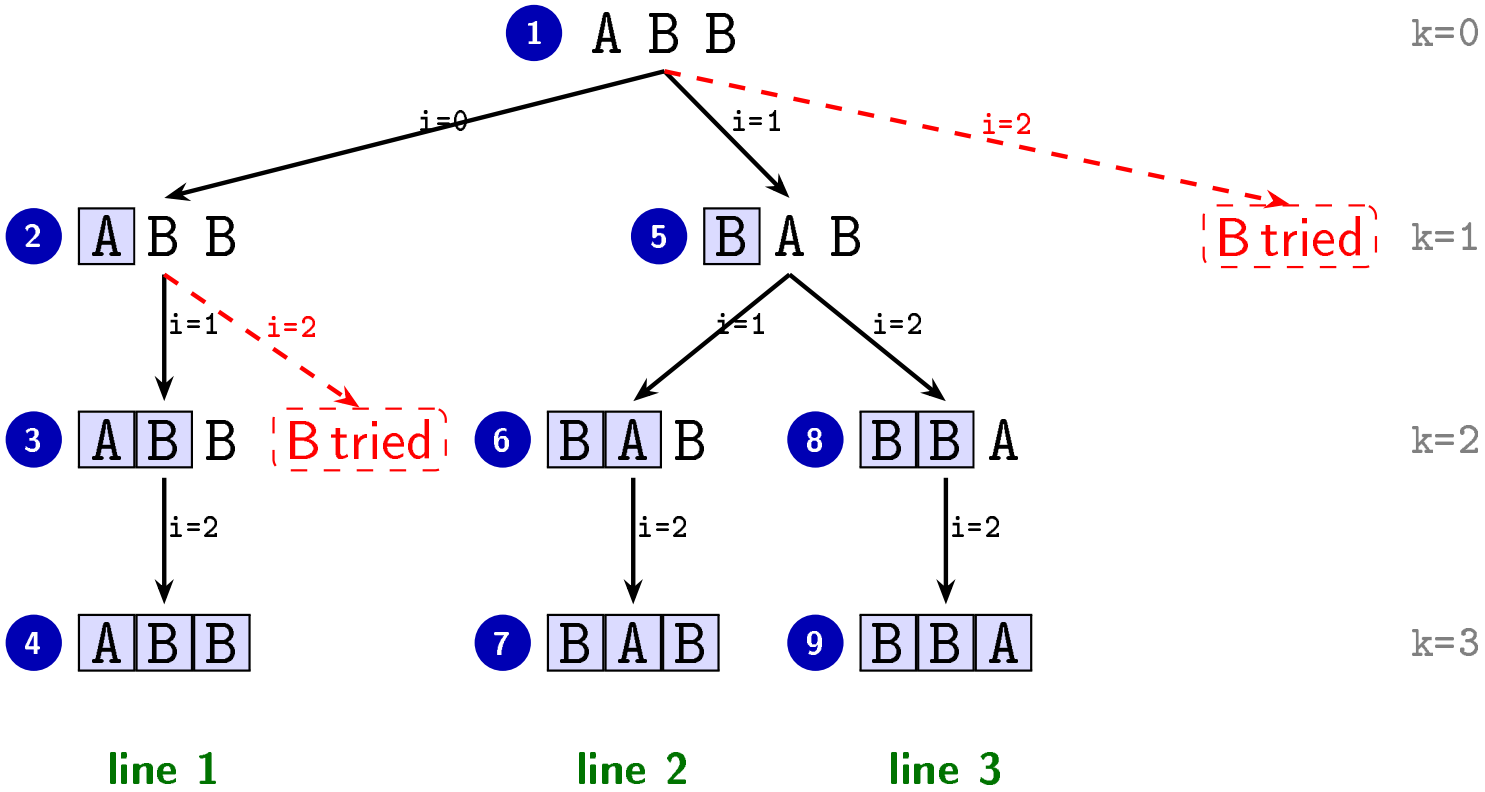
Boxed = occupied seat. **Number** = order of the call. A dashed red node is a **skipped** branch: never called.



Call 8 recurses: call 9 fills seat 2 (k=3). Base case, print.

Call Tree: `printUnique(s, 0)` for `s = "ABB"`

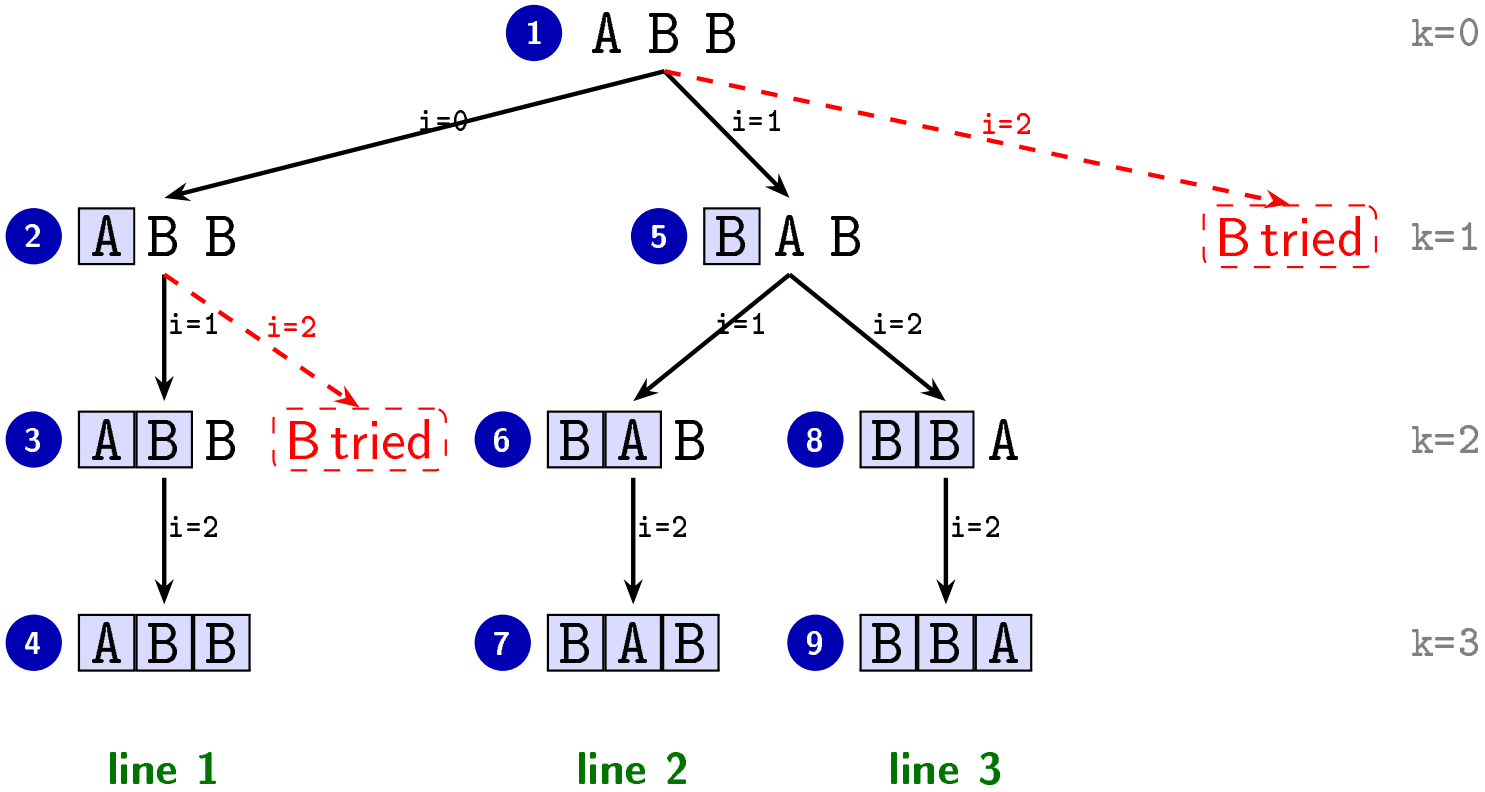
Boxed = occupied seat. **Number** = order of the call. A dashed red node is a **skipped** branch: never called.



Back in call 1: `i=2` is `B`, already in `tried`. **Skip**: no call is made.

Call Tree: `printUnique(s, 0)` for `s = "ABB"`

Boxed = occupied seat. **Number** = order of the call. A dashed red node is a **skipped** branch: never called.



3 lines, 9 calls $(1 + 2 + 3 + 3)$, depth first. `printAll("ABB")` would make 16 calls and print 6 lines.

Check: How Much Work Did We Save?

	`printAll`		`printUnique`	
Input	lines	calls	lines	calls
`"ABC"`	6	16	6	16

Check: How Much Work Did We Save?

	`printAll`		`printUnique`	
Input	lines	calls	lines	calls
`"ABC"`	6	16	6	16
`"ABB"`	6	16	**3**	**9**

Check: How Much Work Did We Save?

	`printAll`		`printUnique`	
Input	lines	calls	lines	calls
`"ABC"`	6	16	6	16
`"ABB"`	6	16	**3**	**9**

Reading the table. With all different letters nothing is skipped, so `printUnique` makes the same recursive calls. With repeats, it prints each string once and makes 9 calls instead of 16.

Check: How Much Work Did We Save?

	`printAll`		`printUnique`	
Input	lines	calls	lines	calls
`"ABC"`	6	16	6	16
`"ABB"`	6	16	**3**	**9**

Reading the table. With all different letters nothing is skipped, so `printUnique` makes the same recursive calls. With repeats, it prints each string once and makes 9 calls instead of 16.

Moral. Pruning is a promise: we skip a branch only when we can *prove* it adds nothing new. Here the proof is: "the same letter at the same seat gives the same set of strings".

Same Idea in Make 24: Seen States

Two move orders, one state. With four 1's on the table, merging the first pair and then the second pair reaches `{2, 2}`, and so does merging the second pair first. Everything below `{2, 2}` is searched twice.

Same Idea in Make 24: Seen States

Two move orders, one state. With four 1's on the table, merging the first pair and then the second pair reaches `{2, 2}`, and so does merging the second pair first. Everything below `{2, 2}` is searched twice.

1. The **state** is the numbers on the table (a *multiset*: duplicates count). `{2,1,1}` and `{1,2,1}` are the **same state**, so sort before comparing.

Same Idea in Make 24: Seen States

Two move orders, one state. With four 1's on the table, merging the first pair and then the second pair reaches `{2, 2}`, and so does merging the second pair first. Everything below `{2, 2}` is searched twice.

1. The **state** is the numbers on the table (a *multiset*: duplicates count). `{2,1,1}` and `{1,2,1}` are the **same state**, so sort before comparing.
2. Keep a `std::set` of the states already searched (a **seen set**), exactly like `tried`, but for whole hands.

Same Idea in Make 24: Seen States

Two move orders, one state. With four 1's on the table, merging the first pair and then the second pair reaches `{2, 2}`, and so does merging the second pair first. Everything below `{2, 2}` is searched twice.

1. The **state** is the numbers on the table (a *multiset*: duplicates count). `{2,1,1}` and `{1,2,1}` are the **same state**, so sort before comparing.
2. Keep a `std::set` of the states already searched (a **seen set**), exactly like `tried`, but for whole hands.
3. If the state is in the set, `return false`: a successful earlier visit would already have ended the search.

Same Idea in Make 24: Seen States

Two move orders, one state. With four 1's on the table, merging the first pair and then the second pair reaches `{2, 2}`, and so does merging the second pair first. Everything below `{2, 2}` is searched twice.

1. The **state** is the numbers on the table (a *multiset*: duplicates count). `{2,1,1}` and `{1,2,1}` are the **same state**, so sort before comparing.
2. Keep a `std::set` of the states already searched (a **seen set**), exactly like `tried`, but for whole hands.
3. If the state is in the set, `return false`: a successful earlier visit would already have ended the search.

`{1,1,1,1}`, **no solution**	**calls**
Without `seen` (unordered pairs, 6 results)	4 141
With `seen`	139

Same Idea in Make 24: Seen States

Two move orders, one state. With four 1's on the table, merging the first pair and then the second pair reaches `{2, 2}`, and so does merging the second pair first. Everything below `{2, 2}` is searched twice.

1. The **state** is the numbers on the table (a *multiset*: duplicates count). `{2,1,1}` and `{1,2,1}` are the **same state**, so sort before comparing.
2. Keep a `std::set` of the states already searched (a **seen set**), exactly like `tried`, but for whole hands.
3. If the state is in the set, `return false`: a successful earlier visit would already have ended the search.

`{1,1,1,1}`, **no solution**	**calls**
Without `seen` (unordered pairs, 6 results)	4 141
With `seen`	139

Use it for Tasks A and B only, not Task C (Task C must return every expression). Keys built from rounded doubles are approximate. The optional section of the Project 3 page gives the code.

Project 3: Two Kinds of State

	`search` / **Task A**	`ops` **(today)**
Question	Can we reach 24?	Show me the expressions.
Carries	a **number**	a **string**
Returns	`bool`	`void` (prints)

Task B needs both: to know whether a hand reaches 24 *and* to show how, each value must carry **its number and the text that produced it**.

Project 3: Two Kinds of State

	`search` / **Task A**	`ops` **(today)**
Question	Can we reach 24?	Show me the expressions.
Carries	a **number**	a **string**
Returns	`bool`	`void` (prints)

Task B needs both: to know whether a hand reaches 24 *and* to show how, each value must carry **its number and the text that produced it**.

Key ideas of Task B

1. Pair every number with its text: `struct Expression_Value { std::string expression; double value; };`
2. At the start, a number's text is itself: `{"3", 3}`.
3. Combining two values builds **new text** `"(" + left + op + right + ")"` and a **new value**.
4. Base case: one value left. If it is 24 return its **text**; otherwise return `""`.
5. The first non-empty string found stops the search and travels back up.

From Task A to Task B: What Changes?

	Task A	Task B
Hand	`vector<double>`	`vector<Expression_Value>`

From Task A to Task B: What Changes?

	Task A	Task B
Hand	`vector<double>`	`vector<Expression_Value>`
Returns	`bool`	`std::string`

From Task A to Task B: What Changes?

	Task A	**Task B**
Hand	`vector<double>`	`vector<Expression_Value>`
Returns	`bool`	`std::string`
Base case	`|v - 24| <= DELTA`	the **text** if `|v - 24| <= DELTA`, else `""`

From Task A to Task B: What Changes?

	Task A	**Task B**
Hand	`vector<double>`	`vector<Expression_Value>`
Returns	`bool`	`std::string`
Base case	`|v - 24| <= DELTA`	the **text** if `|v - 24| <= DELTA`, else `""`
Combine	`arithmetic` gives numbers	`arithmetic` gives numbers **and** their text

From Task A to Task B: What Changes?

	Task A	**Task B**
Hand	`vector<double>`	`vector<Expression_Value>`
Returns	`bool`	`std::string`
Base case	`|v - 24| <= DELTA`	the **text** if `|v - 24| <= DELTA`, else `""`
Combine	`arithmetic` gives numbers	`arithmetic` gives numbers **and** their text
Success	`if (make24(next))` `return true;`	`r = make24(next);` `if (!r.empty()) return r;`

From Task A to Task B: What Changes?

	Task A	**Task B**
Hand	`vector<double>`	`vector<Expression_Value>`
Returns	`bool`	`std::string`
Base case	`|v - 24| <= DELTA`	the **text** if `|v - 24| <= DELTA`, else `""`
Combine	`arithmetic` gives numbers	`arithmetic` gives numbers **and** their text
Success	`if (make24(next)) return true;`	`r = make24(next); if (!r.empty()) return r;`
Failure	`return false;`	`return "";`

From Task A to Task B: What Changes?

	Task A	**Task B**
Hand	`vector<double>`	`vector<Expression_Value>`
Returns	`bool`	`std::string`
Base case	`|v - 24| <= DELTA`	the **text** if `|v - 24| <= DELTA`, else `""`
Combine	`arithmetic` gives numbers	`arithmetic` gives numbers **and** their text
Success	`if (make24(next)) return true;`	`r = make24(next); if (!r.empty()) return r;`
Failure	`return false;`	`return "";`

Same search, same loops, same swap-and-pop removal. Only the type of the answer changes: yes/no becomes "an expression, or nothing".

How One Expression Is Built: {1, 2, 3, 4}

Each box is one value: its **text** above, its number below. One path through the search (pairs and operators are tried in the required order):

Hand 4: "1"/1 "2"/2 "3"/3 "4"/4

How One Expression Is Built: {1, 2, 3, 4}

Each box is one value: its **text** above, its number below. One path through the search (pairs and operators are tried in the required order):

Hand 4:	"1"/1	"2"/2	"3"/3	"4"/4
Hand 3:	"3"/3	"4"/4	"(1+2)"/3	1 and 2, +

How One Expression Is Built: {1, 2, 3, 4}

Each box is one value: its **text** above, its number below. One path through the search (pairs and operators are tried in the required order):

Hand 4:	"1"/1	"2"/2	"3"/3	"4"/4	
Hand 3:	"3"/3	"4"/4	"(1+2)"/3		1 and 2, +
Hand 2:	"4"/4	"(3+(1+2))"/6			3 and the new text, +

How One Expression Is Built: {1, 2, 3, 4}

Each box is one value: its **text** above, its number below. One path through the search (pairs and operators are tried in the required order):

Hand 4:	"1"/1	"2"/2	"3"/3	"4"/4	
Hand 3:	"3"/3	"4"/4	"(1+2)"/3		1 and 2, +
Hand 2:	"4"/4	"(3+(1+2))"/6			3 and the new text, +
Hand 1:	"(4*(3+(1+2)))"/24				4 and the new text, *

How One Expression Is Built: {1, 2, 3, 4}

Each box is one value: its **text** above, its number below. One path through the search (pairs and operators are tried in the required order):

Hand 4:	"1"/1	"2"/2	"3"/3	"4"/4
Hand 3:	"3"/3	"4"/4	"(1+2)"/3	1 and 2, +
Hand 2:	"4"/4	"(3+(1+2))"/6		3 and the new text, +
Hand 1:	"(4*(3+(1+2)))"/24			4 and the new text, *

Base case. One value left and $|24 - 24| \leq$ `DELTA`: return its text `"(4*(3+(1+2)))"`. Each caller sees a non-empty string and returns it unchanged, all the way back to the first call.

How One Expression Is Built: {1, 2, 3, 4}

Each box is one value: its **text** above, its number below. One path through the search (pairs and operators are tried in the required order):

Hand 4:	"1"/1	"2"/2	"3"/3	"4"/4
Hand 3:	"3"/3	"4"/4	"(1+2)"/3	1 and 2, +
Hand 2:	"4"/4	"(3+(1+2))"/6		3 and the new text, +
Hand 1:	"(4*(3+(1+2)))"/24			4 and the new text, *

Base case. One value left and $|24 - 24| \leq$ `DELTA`: return its text `"(4*(3+(1+2)))"`. Each caller sees a non-empty string and returns it unchanged, all the way back to the first call.

The hand shrinks by one each step (4, 3, 2, 1), and the text **grows**: each new text wraps two older texts in parentheses.

Write It Yourself: Task B `make24` (skeleton)

Step 1 of 4: Base case

Hint. In Task A this returned a `bool`. The answer is now text. What do we return when the one remaining value is 24? When it is not?

```
std::string make24(std::vector<Expression_Value> ev_vec) {
    int size = static_cast<int>(ev_vec.size());
    // ??? your turn
    // skeleton: pair loops and cloneVec (swap-and-pop) omitted
    //   tmpVec = arithmetic(ev_vec[i], ev_vec[j]);
    //   for each candidate in tmpVec:
    //       cloneVec.push_back(tmpVec[k]);
    //       cloneVec.pop_back();
}
```

Write It Yourself: Task B `make24` (skeleton)

Step 1 of 4: Base case

Hint. In Task A this returned a `bool`. The answer is now text. What do we return when the one remaining value is 24? When it is not?

Why. If `|value - GOAL| <= DELTA`, return the **text carried by that value**; otherwise return the empty string, meaning "no solution along this path".

```
std::string make24(std::vector<Expression_Value> ev_vec) {
    int size = static_cast<int>(ev_vec.size());
    if (size == 1) {
        if (std::fabs(ev_vec[0].value - GOAL) <= DELTA) {
            return ev_vec[0].expression;
        }
        return "";
    }
    // skeleton: pair loops and cloneVec (swap-and-pop) omitted
    //   tmpVec = arithmetic(ev_vec[i], ev_vec[j]);
    //   for each candidate in tmpVec:
    //       cloneVec.push_back(tmpVec[k]);
    //       cloneVec.pop_back();
}
```

Write It Yourself: Task B `make24` (skeleton)

Step 2 of 4: Keep the answer of the recursive call

Hint. The recursive call now returns a string. Where do we keep it?

```
std::string make24(std::vector<Expression_Value> ev_vec) {
    int size = static_cast<int>(ev_vec.size());
    if (size == 1) {
        if (std::fabs(ev_vec[0].value - GOAL) <= DELTA) {
            return ev_vec[0].expression;
        }
        return "";
    }
    // skeleton: pair loops and cloneVec (swap-and-pop) omitted
    //   tmpVec = arithmetic(ev_vec[i], ev_vec[j]);
    //   for each candidate in tmpVec:
    //       cloneVec.push_back(tmpVec[k]);
           // ??? your turn
    //       cloneVec.pop_back();
}
```

Write It Yourself: Task B `make24` (skeleton)

Step 2 of 4: Keep the answer of the recursive call

Hint. The recursive call now returns a string. Where do we keep it?

Why. `std::string result = make24(cloneVec);` It is either a complete expression for 24 or the empty string.

```
std::string make24(std::vector<Expression_Value> ev_vec) {
    int size = static_cast<int>(ev_vec.size());
    if (size == 1) {
        if (std::fabs(ev_vec[0].value - GOAL) <= DELTA) {
            return ev_vec[0].expression;
        }
        return "";
    }
    // skeleton: pair loops and cloneVec (swap-and-pop) omitted
    //   tmpVec = arithmetic(ev_vec[i], ev_vec[j]);
    //   for each candidate in tmpVec:
    //       cloneVec.push_back(tmpVec[k]);
           std::string result = make24(cloneVec);
    //       cloneVec.pop_back();
}
```

Write It Yourself: Task B `make24` (skeleton)

Step 3 of 4: Stop at the first solution

Hint. Task A wrote `if (make24(...)) return true;`. What is the Task B version?

```
std::string make24(std::vector<Expression_Value> ev_vec) {
    int size = static_cast<int>(ev_vec.size());
    if (size == 1) {
        if (std::fabs(ev_vec[0].value - GOAL) <= DELTA) {
            return ev_vec[0].expression;
        }
        return "";
    }
    // skeleton: pair loops and cloneVec (swap-and-pop) omitted
    //   tmpVec = arithmetic(ev_vec[i], ev_vec[j]);
    //   for each candidate in tmpVec:
    //       cloneVec.push_back(tmpVec[k]);
           std::string result = make24(cloneVec);
           // ??? your turn
    //       cloneVec.pop_back();
}
```

Write It Yourself: Task B `make24` (skeleton)

Step 3 of 4: Stop at the first solution

Hint. Task A wrote `if (make24(...)) return true;`. What is the Task B version?

Why. Any **non-empty** string is already a solution: `if (!result.empty()) { return result; }`. Every caller above does the same, so the first solution travels all the way up.

```
std::string make24(std::vector<Expression_Value> ev_vec) {
    int size = static_cast<int>(ev_vec.size());
    if (size == 1) {
        if (std::fabs(ev_vec[0].value - GOAL) <= DELTA) {
            return ev_vec[0].expression;
        }
        return "";
    }
    // skeleton: pair loops and cloneVec (swap-and-pop) omitted
    //   tmpVec = arithmetic(ev_vec[i], ev_vec[j]);
    //   for each candidate in tmpVec:
    //       cloneVec.push_back(tmpVec[k]);
           std::string result = make24(cloneVec);
           if (!result.empty()) { return result; }
    //       cloneVec.pop_back();
}
```

Write It Yourself: Task B `make24` (skeleton)

Step 4 of 4: Nothing worked

Hint. Every pair and every candidate failed. What does this call report?

```
std::string make24(std::vector<Expression_Value> ev_vec) {
    int size = static_cast<int>(ev_vec.size());
    if (size == 1) {
        if (std::fabs(ev_vec[0].value - GOAL) <= DELTA) {
            return ev_vec[0].expression;
        }
        return "";
    }
    // skeleton: pair loops and cloneVec (swap-and-pop) omitted
    //   tmpVec = arithmetic(ev_vec[i], ev_vec[j]);
    //   for each candidate in tmpVec:
    //       cloneVec.push_back(tmpVec[k]);
           std::string result = make24(cloneVec);
           if (!result.empty()) { return result; }
    //       cloneVec.pop_back();
    // ??? your turn
}
```

Write It Yourself: Task B `make24` (skeleton)

Step 4 of 4: Nothing worked

Hint. Every pair and every candidate failed. What does this call report?

Why. The empty string: this hand cannot make 24. (Task A returned `false` here.)

```
std::string make24(std::vector<Expression_Value> ev_vec) {
    int size = static_cast<int>(ev_vec.size());
    if (size == 1) {
        if (std::fabs(ev_vec[0].value - GOAL) <= DELTA) {
            return ev_vec[0].expression;
        }
        return "";
    }
    // skeleton: pair loops and cloneVec (swap-and-pop) omitted
    //   tmpVec = arithmetic(ev_vec[i], ev_vec[j]);
    //   for each candidate in tmpVec:
    //       cloneVec.push_back(tmpVec[k]);
            std::string result = make24(cloneVec);
            if (!result.empty()) { return result; }
    //       cloneVec.pop_back();
    return "";
}
```

Question: What Does `arithmetic` Build?

In Task B, `arithmetic(ev1, ev2)` is called with `ev1 = {"3", 3}` and `ev2 = {"4", 4}`. What is its **multiplication** entry?

(A) `{"3*4", 12}`

(B) `{"(3*4)", 12}`

(C) `{"(3*4)", 7}`

(D) `{"34", 12}`

Question: What Does `arithmetic` Build?

In Task B, `arithmetic(ev1, ev2)` is called with `ev1 = {"3", 3}` and `ev2 = {"4", 4}`. What is its **multiplication** entry?

(A) `{"3*4", 12}`

(B) `{"(3*4)", 12}`

(C) `{"(3*4)", 7}`

(D) `{"34", 12}`

(B). The text keeps the parentheses so that precedence stays correct when this result is combined again: compare `5/(2*3)` with the wrong `5/2*3`. (A) loses those parentheses. (C) has the wrong value: the value is computed, the text is only the story. (D) glues the digits together.

Three Notations

The operator can go **between**, **before**, or **after** its operands.

Infix (operator between)	**Prefix** (before)	**Postfix** (after)
`3 + 4`	`+ 3 4`	`3 4 +`
`3 + 4 * 2`		
`(3 + 4) * 2`		

Question: write `(3 + 4) * 2` in postfix first, then compare.

Three Notations

The operator can go **between**, **before**, or **after** its operands.

Infix (operator between)	**Prefix** (before)	**Postfix** (after)
`3 + 4`	`+ 3 4`	`3 4 +`
`3 + 4 * 2`	`+ 3 * 4 2`	`3 4 2 * +`
`(3 + 4) * 2`	`* + 3 4 2`	`3 4 + 2 *`

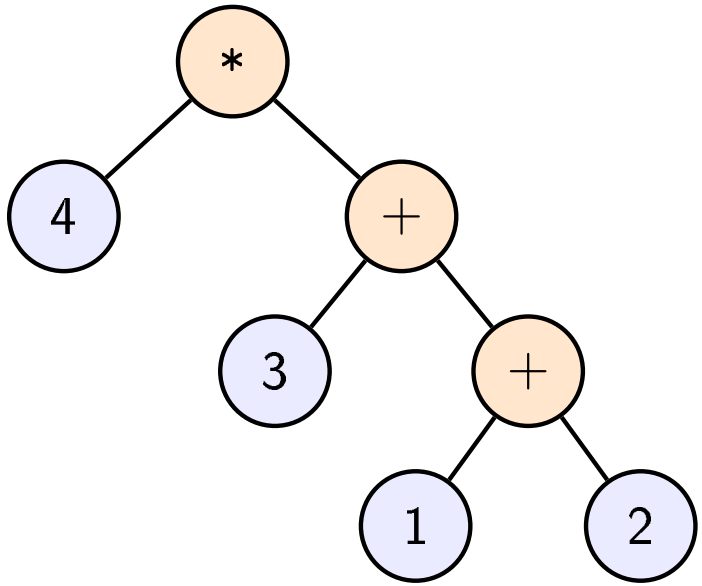
Question: write `(3 + 4) * 2` in postfix first, then compare. Notice that **postfix has no parentheses and no precedence rules**: the order of the tokens already says what to do first. That is why a stack can evaluate it with no precedence decisions to make.

Task B's Answer Is a Tree

The string `(4*(3+(1+2)))` that your solver returns is a **tree** written on one line: each parenthesized combination is an operator node, and each number is a leaf.

Build it (first operation first):
$1 + 2$, then $3 +$ that, then $4 \times$ that.
The last operation, $\times$, is the **root**.

Read it with the three rules (left subtree before right):



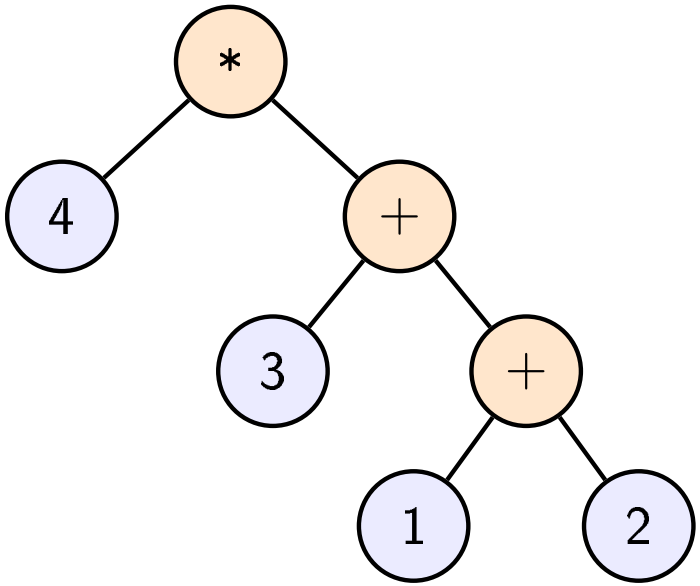
Task B's Answer Is a Tree

The string `(4*(3+(1+2)))` that your solver returns is a **tree** written on one line: each parenthesized combination is an operator node, and each number is a leaf.

Build it (first operation first):
$1 + 2$, then $3 +$ that, then $4 \times$ that.
The last operation, $\times$, is the **root**.

Read it with the three rules (left subtree before right):

Pre: `* 4 + 3 + 1 2`



Task B's Answer Is a Tree

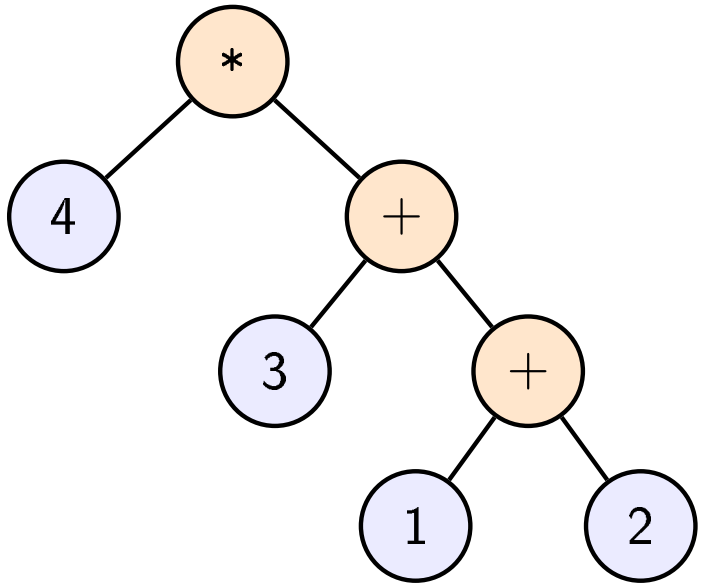
The string `(4*(3+(1+2)))` that your solver returns is a **tree** written on one line: each parenthesized combination is an operator node, and each number is a leaf.

Build it (first operation first):
$1 + 2$, then $3 +$ that, then $4 \times$ that.
The last operation, $\times$, is the **root**.

Read it with the three rules (left subtree before right):

Pre: `* 4 + 3 + 1 2`

In (parens): `(4 * (3 + (1 + 2)))`



Task B's Answer Is a Tree

The string `(4*(3+(1+2)))` that your solver returns is a **tree** written on one line: each parenthesized combination is an operator node, and each number is a leaf.

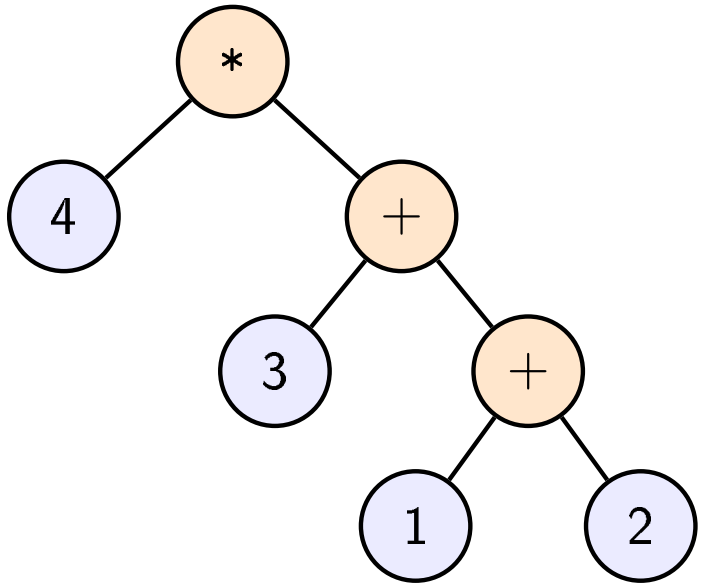
Build it (first operation first):
$1 + 2$, then $3 +$ that, then $4 \times$ that.
The last operation, $\times$, is the **root**.

Read it with the three rules (left subtree before right):

Pre: `* 4 + 3 + 1 2`

In (parens): `(4 * (3 + (1 + 2)))`

Post: `4 3 1 2 + + *`



Task B's Answer Is a Tree

The string `(4*(3+(1+2)))` that your solver returns is a **tree** written on one line: each parenthesized combination is an operator node, and each number is a leaf.

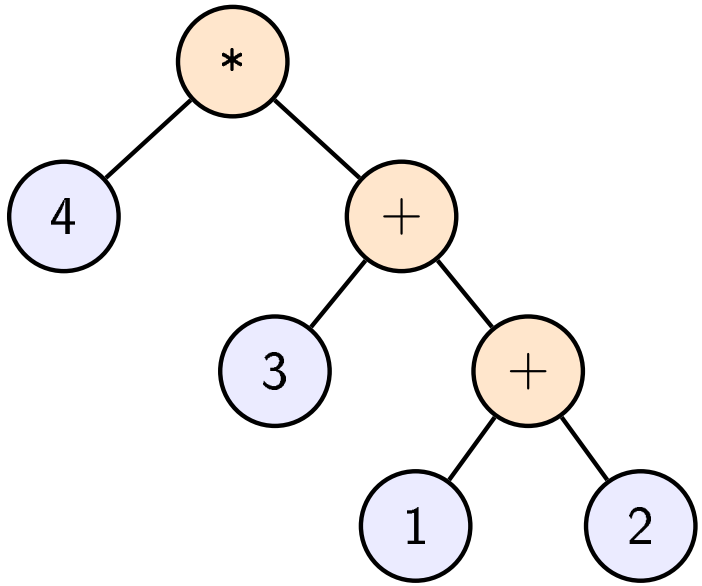
Build it (first operation first):
$1 + 2$, then $3 +$ that, then $4 \times$ that.
The last operation, $\times$, is the **root**.

Read it with the three rules (left subtree before right):

Pre: `* 4 + 3 + 1 2`

In (parens): `(4 * (3 + (1 + 2)))`

Post: `4 3 1 2 + + *`



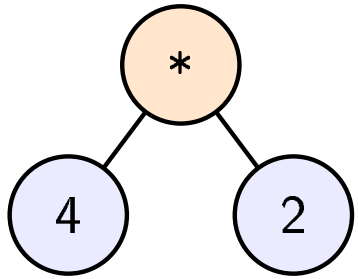
Post-order puts every operator after its two operands: `4 3 1 2 + + *`. Next class we will see why a computer likes this form.

From Expression to Tree: First Operation First

Idea. Operations run in this order: (1) parentheses, (2) $*$ and / before $+$ and $-$, (3) ties left to right. Build the expression tree **in the order operations are performed**. Each new operator becomes the **root of a larger subtree**, which acts as a single operand for the next one. The **last operator** becomes the **root of the whole tree**.

Build `3 + 4 * 2`

1. First: `4 * 2`. Make $*$ with children 4, 2.

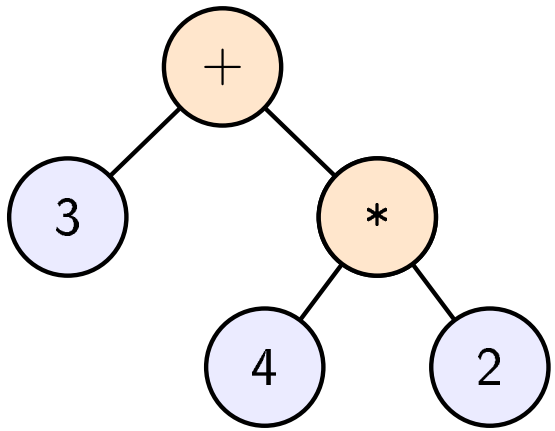


From Expression to Tree: First Operation First

Idea. Operations run in this order: (1) parentheses, (2) $*$ and / before $+$ and $-$, (3) ties left to right. Build the expression tree **in the order operations are performed**. Each new operator becomes the **root of a larger subtree**, which acts as a single operand for the next one. The **last operator** becomes the **root of the whole tree**.

Build `3 + 4 * 2`

1. First: `4 * 2`. Make $*$ with children 4, 2.
2. Next: 3 + (that subtree). Make $+$ with children 3 and the $*$ subtree.

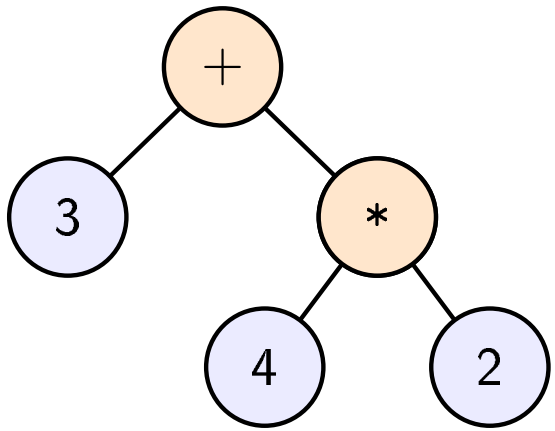


From Expression to Tree: First Operation First

Idea. Operations run in this order: (1) parentheses, (2) $*$ and / before $+$ and $-$, (3) ties left to right. Build the expression tree **in the order operations are performed**. Each new operator becomes the **root of a larger subtree**, which acts as a single operand for the next one. The **last operator** becomes the **root of the whole tree**.

Build `3 + 4 * 2`

1. First: `4 * 2`. Make $*$ with children 4, 2.
2. Next: 3 + (that subtree). Make $+$ with children 3 and the $*$ subtree.
3. Nothing is left: the last one, $+$, is the **root**.

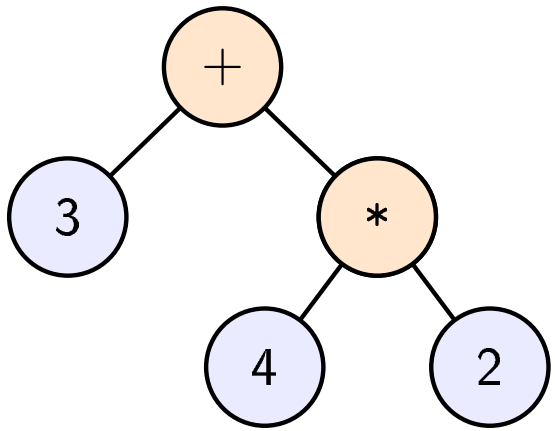


From Expression to Tree: First Operation First

Idea. Operations run in this order: (1) parentheses, (2) $*$ and / before $+$ and $-$, (3) ties left to right. Build the expression tree **in the order operations are performed**. Each new operator becomes the **root of a larger subtree**, which acts as a single operand for the next one. The **last operator** becomes the **root of the whole tree**.

Build `3 + 4 * 2`

1. First: `4 * 2`. Make $*$ with children 4, 2.
2. Next: 3 + (that subtree). Make $+$ with children 3 and the $*$ subtree.
3. Nothing is left: the last one, $+$, is the **root**.

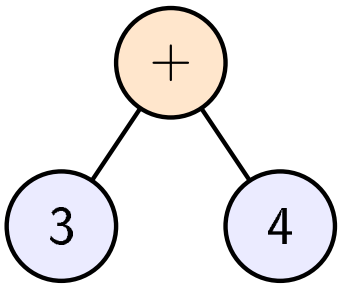


Parentheses and ties

`(3 + 4) * 2`: parentheses first, so $+$ is built first and $*$ is the root. `8 - 3 - 2`: ties go left to right, so the second $-$ is the root.

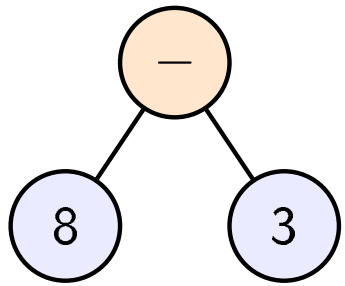
Parentheses and Ties: Two Trees

```
(3 + 4) * 2
```



First: $3 + 4$ (parentheses).

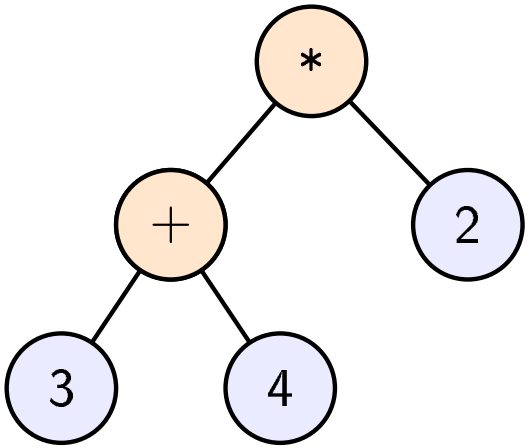
```
8 - 3 - 2
```



First: $8 - 3$ (ties go left to right).

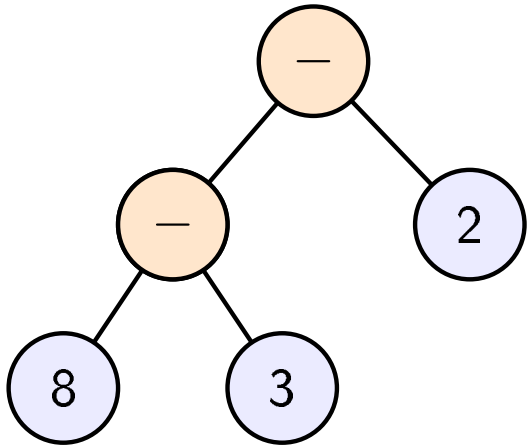
Parentheses and Ties: Two Trees

`(3 + 4) * 2`



Last built: $*$, the **root**.

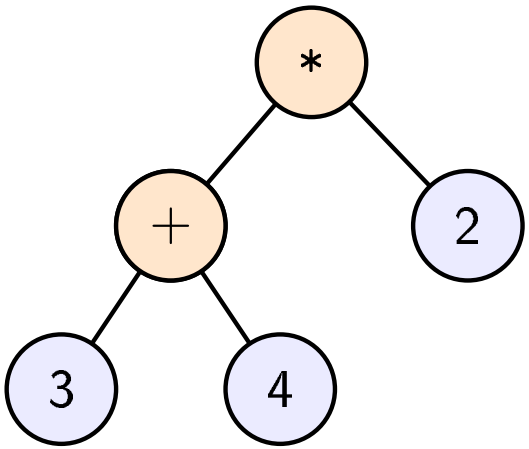
`8 - 3 - 2`



Last built: the second $-$, the **root**.

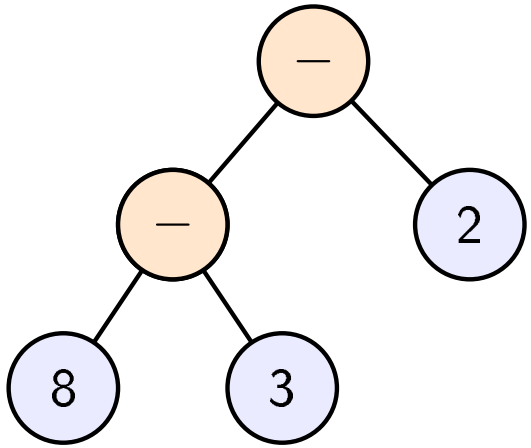
Parentheses and Ties: Two Trees

`(3 + 4) * 2`



Last built: $*$, the **root**.

`8 - 3 - 2`



Last built: the second $-$, the **root**.

In-order with parentheses:

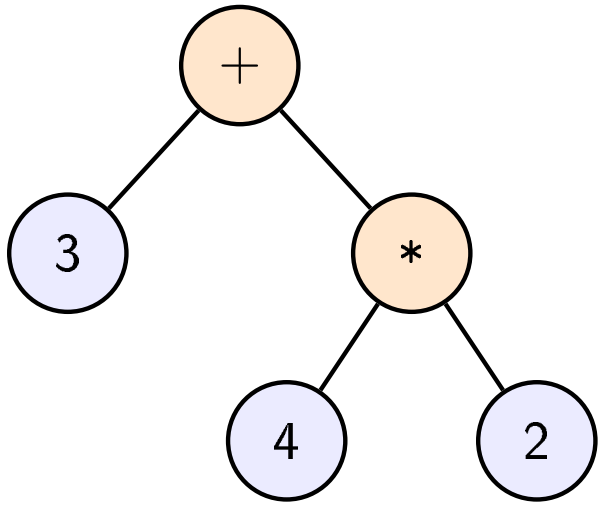
`((3 + 4) * 2)` and `((8 - 3) - 2)`.

The nesting of the parentheses mirrors the shape of the tree.

Reading the Tree: Pre-, In-, and Post-Order

Rule. The words **pre**, **in**, and **post** say where the **root** goes. The **left subtree always comes before the right subtree.**

- **Pre-order: root**, left subtree, right subtree



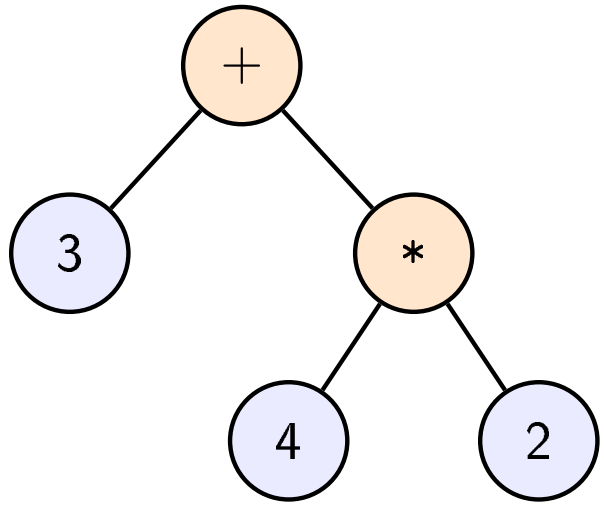
Every subtree is read by the **same rule**: that is recursion.

Pre-order $\rightarrow$ prefix: `+ 3 * 4 2` *(root +, left 3, right subtree)*

Reading the Tree: Pre-, In-, and Post-Order

Rule. The words **pre**, **in**, and **post** say where the **root** goes. The **left subtree always comes before the right subtree.**

- **Pre-order: root**, left subtree, right subtree
- **In-order:** left subtree, **root**, right subtree



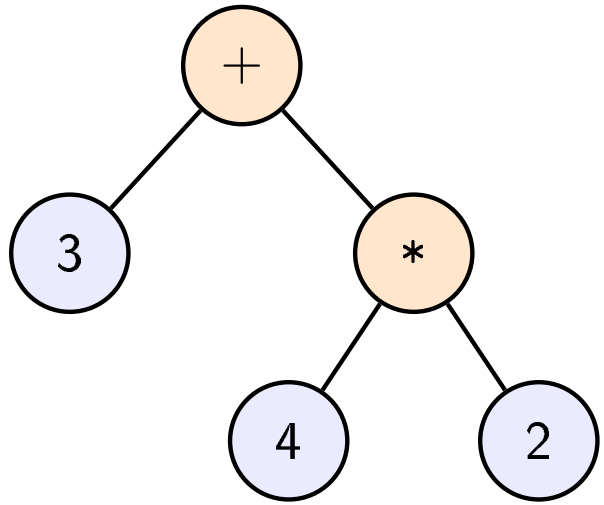
Every subtree is read by the **same rule**: that is recursion.

Pre-order → prefix:	`+ 3 * 4 2`	*(root +, left 3, right subtree)*
In-order → infix:	`(3 + (4 * 2))`	*(parenthesized)*

Reading the Tree: Pre-, In-, and Post-Order

Rule. The words **pre**, **in**, and **post** say where the **root** goes. The **left subtree always comes before the right subtree.**

- **Pre-order: root**, left subtree, right subtree
- **In-order:** left subtree, **root**, right subtree
- **Post-order:** left subtree, right subtree, **root**



Every subtree is read by the **same rule**: that is recursion.

Pre-order → prefix:	`+ 3 * 4 2`	*(root +, left 3, right subtree)*
In-order → infix:	`(3 + (4 * 2))`	*(parenthesized)*
Post-order → postfix:	`3 4 2 * +`	*(left 3, right subtree, root +)*

Preorder, Step 1: Node structure

Goal: print an expression tree in **pre-order** (root, left subtree, right subtree). Pick an answer, then check the reasons.

Each node holds a token (`+` or `3`). Which `Node` stores it and reaches its operands?

(A) `struct Node { std::string token; };`

(B) `struct Node { std::string token; Node* next; };`

(C) `struct Node { std::string token; Node* left; Node* right; };`

(D) `struct Node { std::string token; Node* left; Node* middle; Node* right; };`

Preorder, Step 1: Node structure

Goal: print an expression tree in **pre-order** (root, left subtree, right subtree). Pick an answer, then check the reasons.

Each node holds a token (`+` or `3`). Which `Node` stores it and reaches its operands?

(A) `struct Node { std::string token; };`

No links: not a tree.

(B) `struct Node { std::string token; Node* next; };`

One link makes a list; a binary operator needs two links.

(C) `struct Node { std::string token; Node* left; Node* right; };`

Correct. Two operands, so two links, `left` and `right` ($3 - 4$ is not $4 - 3$). Each link is a pointer to a subtree; `nullptr` means "no child".

(D) `struct Node { std::string token; Node* left; Node* middle; Node* right; };`

`middle` would never be used.

Preorder, Step 1: Node structure

Goal: print an expression tree in **pre-order** (root, left subtree, right subtree). Pick an answer, then check the reasons.

Each node holds a token (`+` or `3`). Which `Node` stores it and reaches its operands?

(A) `struct Node { std::string token; };`

No links: not a tree.

(B) `struct Node { std::string token; Node* next; };`

One link makes a list; a binary operator needs two links.

(C) `struct Node { std::string token; Node* left; Node* right; };`

Correct. Two operands, so two links, `left` and `right` ($3 - 4$ is not $4 - 3$). Each link is a pointer to a subtree; `nullptr` means "no child".

(D) `struct Node { std::string token; Node* left; Node* middle; Node* right; };`

`middle` would never be used.

Code so far, new line: `struct Node { std::string token; Node* left; Node* right; };`

Preorder, Step 2: Function header

Which header visits one subtree and prints its tokens?

(A)
```
void preorder(Node root)
```

(B)
```
void preorder(const Node* root)
```

(C)
```
void preorder(const Node* root, const Node* left,
    const Node* right)
```

(D)
```
Node* preorder(Node* root)
```

Preorder, Step 2: Function header

Which header visits one subtree and prints its tokens?

(A) `void preorder(Node root)`

Copies the whole node on every call, and has no way to say "this subtree is empty" (there is no null `Node`).

(B) `void preorder(const Node* root)`

Correct. It returns nothing because its job is to print. A pointer to the subtree's root can be `nullptr` for an empty subtree, and `const` promises that we only read the tree. `root` means the root of whatever subtree this call is given.

(C) `void preorder(const Node* root, const Node* left, const Node* right)`

The children are already reachable as `root->left` and `root->right`; extra parameters add nothing.

(D) `Node* preorder(Node* root)`

Returns a pointer nobody uses, and the non-`const` pointer suggests we may modify the tree.

Preorder, Step 2: Function header

Which header visits one subtree and prints its tokens?

(A) `void preorder(Node root)`

Copies the whole node on every call, and has no way to say "this subtree is empty" (there is no null `Node`).

(B) `void preorder(const Node* root)`

Correct. It returns nothing because its job is to print. A pointer to the subtree's root can be `nullptr` for an empty subtree, and `const` promises that we only read the tree. `root` means the root of whatever subtree this call is given.

(C) `void preorder(const Node* root, const Node* left, const Node* right)`

The children are already reachable as `root->left` and `root->right`; extra parameters add nothing.

(D) `Node* preorder(Node* root)`

Returns a pointer nobody uses, and the non-`const` pointer suggests we may modify the tree.

Code so far, new line: `void preorder(const Node* root) { ... }`

Preorder, Step 3: Base case

Which line stops the recursion at an empty subtree?

(A) `if (root->left == nullptr) { return; }`

(B) `if (root == nullptr) { return; }`

(C) `if (root != nullptr) { return; }`

(D) `if (root == nullptr) { std::cout << root->token; }`

Preorder, Step 3: Base case

Which line stops the recursion at an empty subtree?

(A) `if (root->left == nullptr) { return; }`

A leaf has no left child, so a leaf would return before printing its own token. It also dereferences `root`, which may itself be `nullptr`.

(B) `if (root == nullptr) { return; }`

Correct. An empty subtree has nothing to print. Every chain of calls ends here, because each call moves to a smaller subtree until it reaches `nullptr`.

(C) `if (root != nullptr) { return; }`

Returns on every real node, so nothing is printed; a null root falls through and is dereferenced.

(D) `if (root == nullptr) { std::cout << root->token; }`

Dereferences `nullptr` exactly when `root` is null: crash.

Preorder, Step 3: Base case

Which line stops the recursion at an empty subtree?

(A) `if (root->left == nullptr) { return; }`

A leaf has no left child, so a leaf would return before printing its own token. It also dereferences `root`, which may itself be `nullptr`.

(B) `if (root == nullptr) { return; }`

Correct. An empty subtree has nothing to print. Every chain of calls ends here, because each call moves to a smaller subtree until it reaches `nullptr`.

(C) `if (root != nullptr) { return; }`

Returns on every real node, so nothing is printed; a null root falls through and is dereferenced.

(D) `if (root == nullptr) { std::cout << root->token; }`

Dereferences `nullptr` exactly when `root` is null: crash.

Code so far, new line: `if (root == nullptr) { return; }`

Preorder, Step 4: First line (non-empty subtree)

Pre-order means the root goes first. What is the first line?

(A) `preorder(root->left);`

(B) `std::cout << root->token << ' ';`

(C) `preorder(root->right);`

(D) `return;`

Preorder, Step 4: First line (non-empty subtree)

Pre-order means the root goes first. What is the first line?

(A) `preorder(root->left);`

That visits the left subtree before the root. Root-after-left is in-order or post-order.

(B) `std::cout << root->token << ' ';`

*Correct. **Pre** means the root is visited **before** its subtrees, so print it now.*

(C) `preorder(root->right);`

The right subtree must never come before the left subtree.

(D) `return;`

The base case already handled the empty subtree. Here the root exists, so returning would skip the whole subtree.

Preorder, Step 4: First line (non-empty subtree)

Pre-order means the root goes first. What is the first line?

(A) `preorder(root->left);`

That visits the left subtree before the root. Root-after-left is in-order or post-order.

(B) `std::cout << root->token << ' ';`

*Correct. **Pre** means the root is visited **before** its subtrees, so print it now.*

(C) `preorder(root->right);`

The right subtree must never come before the left subtree.

(D) `return;`

The base case already handled the empty subtree. Here the root exists, so returning would skip the whole subtree.

Code so far, new line: `std::cout << root->token << ' ';`

Preorder, Step 5: Traverse the left subtree

Which line traverses the entire left subtree?

(A) `preorder(root);`

(B) `preorder(root->left);`

(C) `std::cout << root->left->token << ' ';`

(D) `preorder(root->right);`

Preorder, Step 5: Traverse the left subtree

Which line traverses the entire left subtree?

(A) `preorder(root);`

Same argument, so the call never gets smaller: infinite recursion.

(B) `preorder(root->left);`

Correct. The left subtree is a smaller tree, and we already know how to traverse a tree: call the same function. Each call shrinks the problem until it reaches `nullptr`.

(C) `std::cout << root->left->token << ' ';`

Prints only the left child, not the whole subtree, and crashes if `left` *is* `nullptr`.

(D) `preorder(root->right);`

That is the right subtree. Left comes first.

Preorder, Step 5: Traverse the left subtree

Which line traverses the entire left subtree?

(A) `preorder(root);`

Same argument, so the call never gets smaller: infinite recursion.

(B) `preorder(root->left);`

Correct. The left subtree is a smaller tree, and we already know how to traverse a tree: call the same function. Each call shrinks the problem until it reaches `nullptr`.

(C) `std::cout << root->left->token << ' ';`

Prints only the left child, not the whole subtree, and crashes if `left` is `nullptr`.

(D) `preorder(root->right);`

That is the right subtree. Left comes first.

Code so far, new line: `preorder(root->left);`

Preorder, Step 6: Traverse the right subtree

After the left subtree is completely finished, which line comes next?

(A) `preorder(root->right);`

(B) `preorder(root->left);`

(C) `preorder(root->next);`

(D) `return root->right;`

Preorder, Step 6: Traverse the right subtree

After the left subtree is completely finished, which line comes next?

(A) `preorder(root->right);`

Correct. The same function on the other child. The left subtree always finishes before the right subtree starts; that fixed order makes the output unique.

(B) `preorder(root->left);`

Visits the left subtree twice and never reaches the right one.

(C) `preorder(root->next);`

Our `Node` has no `next`: we chose `left` and `right` in Step 1.

(D) `return root->right;`

`preorder` returns `void`, so this does not compile, and it never visits the right subtree.

Preorder, Step 6: Traverse the right subtree

After the left subtree is completely finished, which line comes next?

(A) `preorder(root->right);`

Correct. The same function on the other child. The left subtree always finishes before the right subtree starts; that fixed order makes the output unique.

(B) `preorder(root->left);`

Visits the left subtree twice and never reaches the right one.

(C) `preorder(root->next);`

Our `Node` has no `next`: we chose `left` and `right` in Step 1.

(D) `return root->right;`

`preorder` returns `void`, so this does not compile, and it never visits the right subtree.

Code so far, new line: `preorder(root->right);`

Preorder: Putting the Six Steps Together

```
struct Node { std::string token; Node* left; Node* right; };

void preorder(const Node* root) {
    if (root == nullptr) { return; }      // base case: empty subtree
    std::cout << root->token << ' ';      // root first
    preorder(root->left);                 // whole left subtree
    preorder(root->right);                // then whole right subtree
}
```

Why two recursive calls? The Node has two links, so each non-empty call has two subtrees to visit. The **base case** ends each branch; the **recursive calls** shrink the problem; the **order** of the three lines is what makes it pre-order.

Your Turn: `inorder` and `postorder`

Copy `preorder` and change **one thing**. Same `Node`, same base case, same two recursive calls; the left subtree is still before the right.

`inorder` *Hint:* **in**-order puts the root **between** the two subtrees. Where does the print line go?

`postorder` *Hint:* **post**-order visits **both** subtrees first and the root **last**. Where does the print line go?

Your Turn: `inorder` and `postorder`

Copy `preorder` and change **one thing**. Same `Node`, same base case, same two recursive calls; the left subtree is still before the right.

`inorder` *Hint:* **in**-order puts the root **between** the two subtrees. Where does the print line go?

```
void inorder(const Node* root) {
    if (root == nullptr) { return; }
    inorder(root->left);
    std::cout << root->token << ' ';
    inorder(root->right);
}
```

`postorder` *Hint:* **post**-order visits **both** subtrees first and the root **last**. Where does the print line go?

Your Turn: `inorder` and `postorder`

Copy `preorder` and change **one thing**. Same `Node`, same base case, same two recursive calls; the left subtree is still before the right.

`inorder` *Hint:* **in**-order puts the root **between** the two subtrees. Where does the print line go?

```
void inorder(const Node* root) {
    if (root == nullptr) { return; }
    inorder(root->left);
    std::cout << root->token << ' ';
    inorder(root->right);
}
```

`postorder` *Hint:* **post**-order visits **both** subtrees first and the root **last**. Where does the print line go?

```
void postorder(const Node* root) {
    if (root == nullptr) { return; }
    postorder(root->left);
    postorder(root->right);
    std::cout << root->token << ' ';
}
```

Your Turn: `inorder` and `postorder`

Copy `preorder` and change **one thing**. Same `Node`, same base case, same two recursive calls; the left subtree is still before the right.

`inorder` *Hint:* **in**-order puts the root **between** the two subtrees. Where does the print line go?

```
void inorder(const Node* root) {
    if (root == nullptr) { return; }
    inorder(root->left);
    std::cout << root->token << ' ';
    inorder(root->right);
}
```

`postorder` *Hint:* **post**-order visits **both** subtrees first and the root **last**. Where does the print line go?

```
void postorder(const Node* root) {
    if (root == nullptr) { return; }
    postorder(root->left);
    postorder(root->right);
    std::cout << root->token << ' ';
}
```

Summary: the three functions are identical except for the position of the one line that prints the root (in red): **first** (pre), **middle** (in), **last** (post). In-order prints no parentheses, so it matches the original expression only when none were needed.

In-order Printing with Parentheses

In-order traversal only says *when* to visit the node: left, node, right. Two versions: plain `inorder` gives `5 + 1 + 2 * 4 - 3`; `printInOrderWParen` gives `((5 + ((1 + 2) * 4)) - 3)`.

```
void printInOrderWParen(const Node* root) {
    if (root == nullptr) { return; }
    bool isOperator = (root->left != nullptr); // leaves are numbers
    if (isOperator) { std::cout << "("; }
    printInOrderWParen(root->left);
    std::cout << (isOperator ? " " + root->token + " " : root->token);
    printInOrderWParen(root->right);
    if (isOperator) { std::cout << ")"; }
}
```

Rule. A number node prints just its number. An operator node prints (left-expression operator right-expression).

In-order Printing with Parentheses

In-order traversal only says *when* to visit the node: left, node, right. Two versions: plain `inorder` gives `5 + 1 + 2 * 4 - 3`; `printInOrderWParen` gives `((5 + ((1 + 2) * 4)) - 3)`.

```
void printInOrderWParen(const Node* root) {
    if (root == nullptr) { return; }
    bool isOperator = (root->left != nullptr); // leaves are numbers
    if (isOperator) { std::cout << "("; }
    printInOrderWParen(root->left);
    std::cout << (isOperator ? " " + root->token + " " : root->token);
    printInOrderWParen(root->right);
    if (isOperator) { std::cout << ")"; }
}
```

Rule. A number node prints just its number. An operator node prints (left-expression operator right-expression).

Build bottom up: `(1 + 2)` → `((1 + 2) * 4)` → `(5 + ((1 + 2) * 4))` → `((5 + ((1 + 2) * 4)) - 3)`.

In-order Printing with Parentheses

In-order traversal only says *when* to visit the node: left, node, right. Two versions: plain `inorder` gives `5 + 1 + 2 * 4 - 3`; `printInOrderWParen` gives `((5 + ((1 + 2) * 4)) - 3)`.

```
void printInOrderWParen(const Node* root) {
    if (root == nullptr) { return; }
    bool isOperator = (root->left != nullptr); // leaves are numbers
    if (isOperator) { std::cout << "("; }
    printInOrderWParen(root->left);
    std::cout << (isOperator ? " " + root->token + " " : root->token);
    printInOrderWParen(root->right);
    if (isOperator) { std::cout << ")"; }
}
```

Rule. A number node prints just its number. An operator node prints (left-expression operator right-expression).

Build bottom up: `(1 + 2)` → `((1 + 2) * 4)` → `(5 + ((1 + 2) * 4))` → `((5 + ((1 + 2) * 4)) - 3)`.

Connection to Make 24: Task B builds the same structure with `"(" + left + op + right + ")"`.

How the Parentheses Get Added

Idea. An operator node prints (**when its call starts** and) **just before its call ends**. Everything in between is its left subtree, the operator, and its right subtree.

Analogy. Walking into a room, you switch the light on at the door and switch it off when you leave. Rooms inside rooms give lights inside lights, so the parentheses **nest and match** automatically.

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	(

How the Parentheses Get Added

Idea. An operator node prints (**when its call starts** and) **just before its call ends**. Everything in between is its left subtree, the operator, and its right subtree.

Analogy. Walking into a room, you switch the light on at the door and switch it off when you leave. Rooms inside rooms give lights inside lights, so the parentheses **nest and match** automatically.

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	(
2	`printInOrderWParen(+)`	operator: print (	((

How the Parentheses Get Added

Idea. An operator node prints (**when its call starts** and) **just before its call ends**. Everything in between is its left subtree, the operator, and its right subtree.

Analogy. Walking into a room, you switch the light on at the door and switch it off when you leave. Rooms inside rooms give lights inside lights, so the parentheses **nest and match** automatically.

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	(
2	`printInOrderWParen(+)`	operator: print (	((
3	`printInOrderWParen(1)`	leaf: print 1	((1

How the Parentheses Get Added

Idea. An operator node prints (**when its call starts** and) **just before its call ends**. Everything in between is its left subtree, the operator, and its right subtree.

Analogy. Walking into a room, you switch the light on at the door and switch it off when you leave. Rooms inside rooms give lights inside lights, so the parentheses **nest and match** automatically.

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	(
2	`printInOrderWParen(+)`	operator: print (	((
3	`printInOrderWParen(1)`	leaf: print 1	((1
4	back in +	print +	((1 +

How the Parentheses Get Added

Idea. An operator node prints (**when its call starts** and) **just before its call ends**. Everything in between is its left subtree, the operator, and its right subtree.

Analogy. Walking into a room, you switch the light on at the door and switch it off when you leave. Rooms inside rooms give lights inside lights, so the parentheses **nest and match** automatically.

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	(
2	`printInOrderWParen(+)`	operator: print (	((
3	`printInOrderWParen(1)`	leaf: print 1	((1
4	back in +	print +	((1 +
5	`printInOrderWParen(2)`	leaf: print 2	((1 + 2

How the Parentheses Get Added

Idea. An operator node prints (**when its call starts** and) **just before its call ends**. Everything in between is its left subtree, the operator, and its right subtree.

Analogy. Walking into a room, you switch the light on at the door and switch it off when you leave. Rooms inside rooms give lights inside lights, so the parentheses **nest and match** automatically.

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	(
2	`printInOrderWParen(+)`	operator: print (	((
3	`printInOrderWParen(1)`	leaf: print 1	((1
4	back in +	print +	((1 +
5	`printInOrderWParen(2)`	leaf: print 2	((1 + 2
6	back in +	operator: print)	((1 + 2)

How the Parentheses Get Added

Idea. An operator node prints (**when its call starts** and) **just before its call ends**. Everything in between is its left subtree, the operator, and its right subtree.

Analogy. Walking into a room, you switch the light on at the door and switch it off when you leave. Rooms inside rooms give lights inside lights, so the parentheses **nest and match** automatically.

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	(
2	`printInOrderWParen(+)`	operator: print (	((
3	`printInOrderWParen(1)`	leaf: print 1	((1
4	back in +	print +	((1 +
5	`printInOrderWParen(2)`	leaf: print 2	((1 + 2
6	back in +	operator: print)	((1 + 2)
7	back in *	print *	((1 + 2) *

How the Parentheses Get Added

Idea. An operator node prints (**when its call starts** and) **just before its call ends**. Everything in between is its left subtree, the operator, and its right subtree.

Analogy. Walking into a room, you switch the light on at the door and switch it off when you leave. Rooms inside rooms give lights inside lights, so the parentheses **nest and match** automatically.

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	`(`
2	`printInOrderWParen(+)`	operator: print (	`((`
3	`printInOrderWParen(1)`	leaf: print 1	`((1`
4	back in +	print +	`((1 +`
5	`printInOrderWParen(2)`	leaf: print 2	`((1 + 2`
6	back in +	operator: print)	`((1 + 2)`
7	back in *	print *	`((1 + 2) *`
8	`printInOrderWParen(4)`	leaf: print 4	`((1 + 2) * 4`

How the Parentheses Get Added

Idea. An operator node prints (**when its call starts** and) **just before its call ends**. Everything in between is its left subtree, the operator, and its right subtree.

Analogy. Walking into a room, you switch the light on at the door and switch it off when you leave. Rooms inside rooms give lights inside lights, so the parentheses **nest and match** automatically.

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	`(`
2	`printInOrderWParen(+)`	operator: print (	`((`
3	`printInOrderWParen(1)`	leaf: print 1	`((1`
4	back in +	print +	`((1 +`
5	`printInOrderWParen(2)`	leaf: print 2	`((1 + 2`
6	back in +	operator: print)	`((1 + 2)`
7	back in *	print *	`((1 + 2) *`
8	`printInOrderWParen(4)`	leaf: print 4	`((1 + 2) * 4`
9	back in *	operator: print)	`((1 + 2) * 4)`

Question: Where Does) Go?

In `printInOrderWParen`, which line prints the) of an operator node?

(A) Before `printInOrderWParen(root->left);`

(B) Right after `std::cout << root->token;`

(C) After `printInOrderWParen(root->right);`, only if the node is an operator

(D) At the base case `if (root == nullptr)`

Question: Where Does) Go?

In `printInOrderWParen`, which line prints the) of an operator node?

(A) Before `printInOrderWParen(root->left);`

That is where (goes.

(B) Right after `std::cout << root->token;`

Too early: (1 +)2.

(C) **After `printInOrderWParen(root->right);`, only if the node is an operator**

Correct. Left, operator and right are all printed, so the group can close. Numbers print no parentheses.

(D) At the base case `if (root == nullptr)`

Parentheses around nothing.

Question: Where Does) Go?

In `printInOrderWParen`, which line prints the) of an operator node?

(A) Before `printInOrderWParen(root->left);`

That is where (goes.

(B) Right after `std::cout << root->token;`

Too early: (1 +)2.

(C) **After `printInOrderWParen(root->right);`, only if the node is an operator**

Correct. Left, operator and right are all printed, so the group can close. Numbers print no parentheses.

(D) At the base case `if (root == nullptr)`

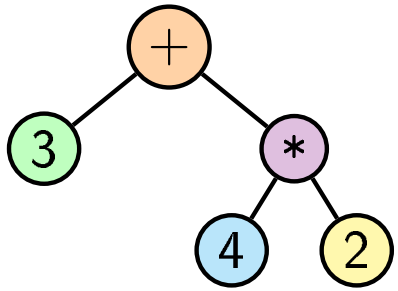
Parentheses around nothing.

Rule: (goes before the recursive calls,) after them.

Running `postorder` on `3 + 4 * 2`: the Call Tree

Every non-empty call makes two child calls, even for a leaf; a call on `nullptr` returns at once. **Red** = where a node is printed. **Same color** = **same node** of the expression tree (top right).

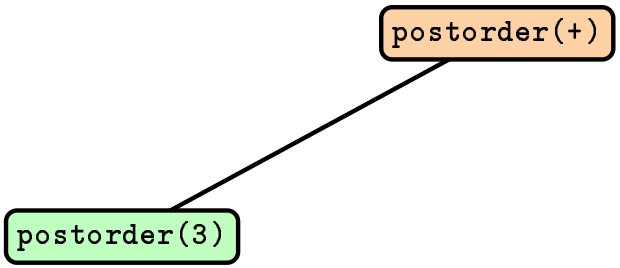
`postorder(+)`

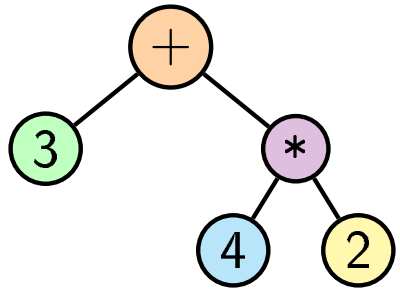


`Output:`

Running `postorder` on `3 + 4 * 2`: the Call Tree

Every non-empty call makes two child calls, even for a leaf; a call on `nullptr` returns at once. **Red** = where a node is printed. **Same color** = **same node** of the expression tree (top right).

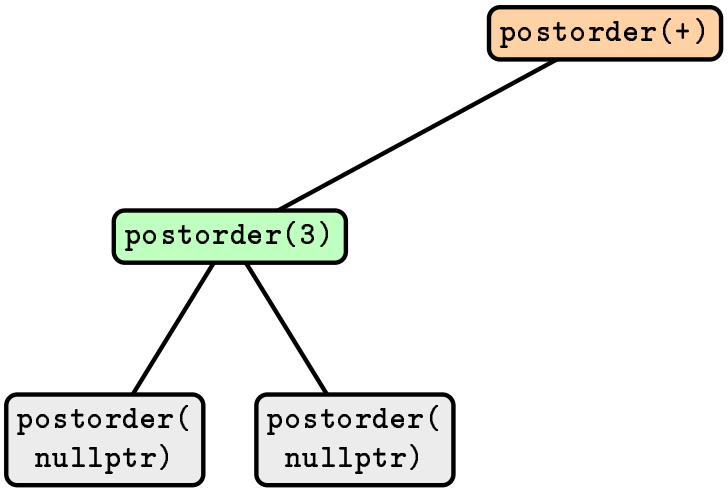


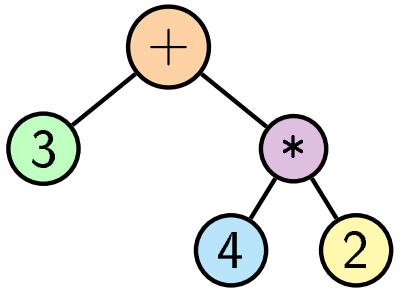


Output:

Running `postorder` on `3 + 4 * 2`: the Call Tree

Every non-empty call makes two child calls, even for a leaf; a call on `nullptr` returns at once. **Red** = where a node is printed. **Same color** = **same node** of the expression tree (top right).

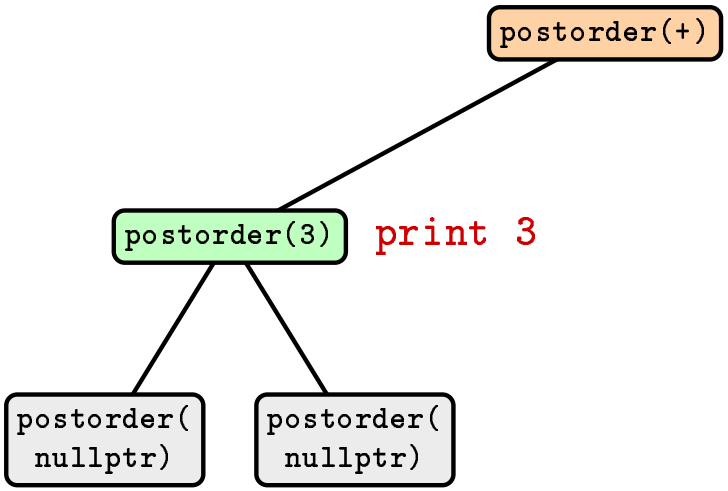


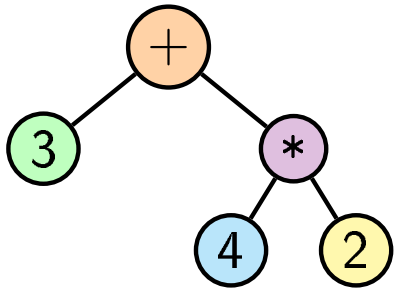


Output:

Running `postorder` on `3 + 4 * 2`: the Call Tree

Every non-empty call makes two child calls, even for a leaf; a call on `nullptr` returns at once. **Red** = where a node is printed. **Same color** = **same node** of the expression tree (top right).

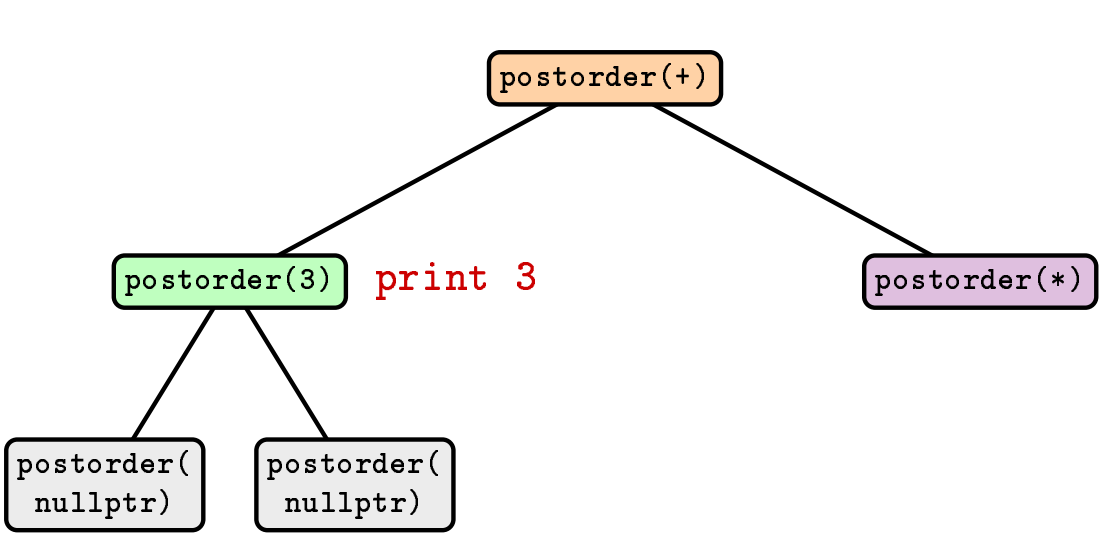


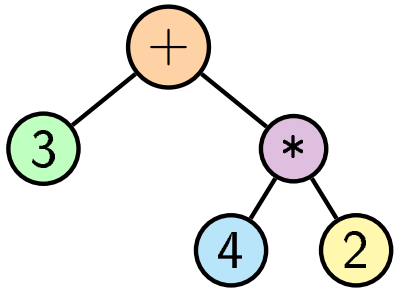



```
Output: 3
```

Running `postorder` on `3 + 4 * 2`: the Call Tree

Every non-empty call makes two child calls, even for a leaf; a call on `nullptr` returns at once. **Red** = where a node is printed. **Same color** = **same node** of the expression tree (top right).

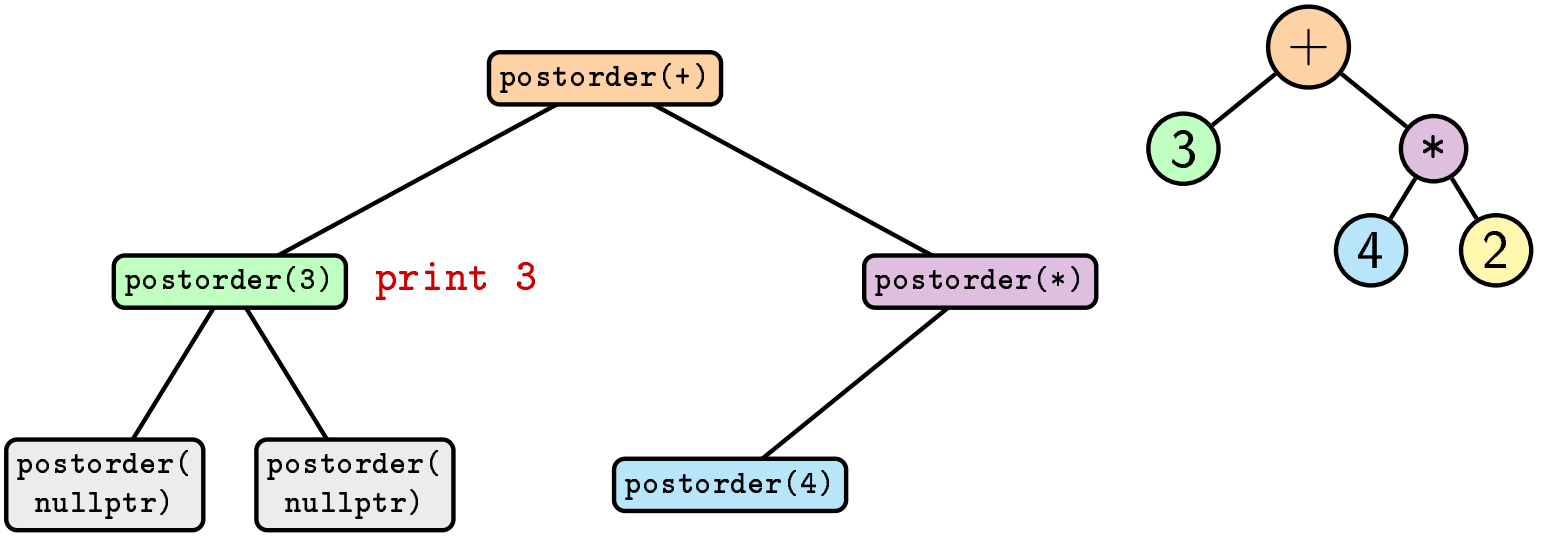





```
Output: 3
```

Running `postorder` on `3 + 4 * 2`: the Call Tree

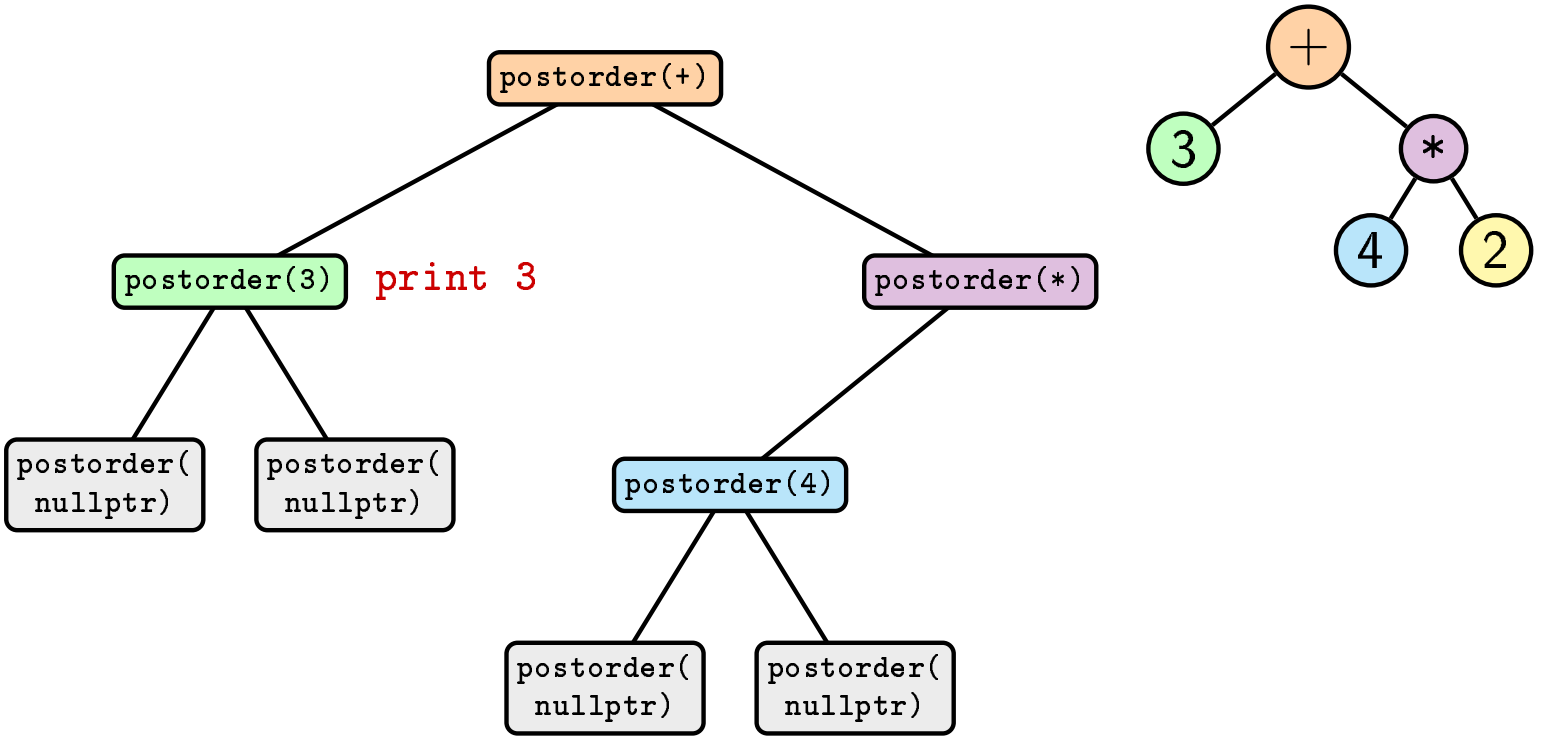
Every non-empty call makes two child calls, even for a leaf; a call on `nullptr` returns at once. **Red** = where a node is printed. **Same color** = **same node** of the expression tree (top right).




```
Output: 3
```

Running `postorder` on `3 + 4 * 2`: the Call Tree

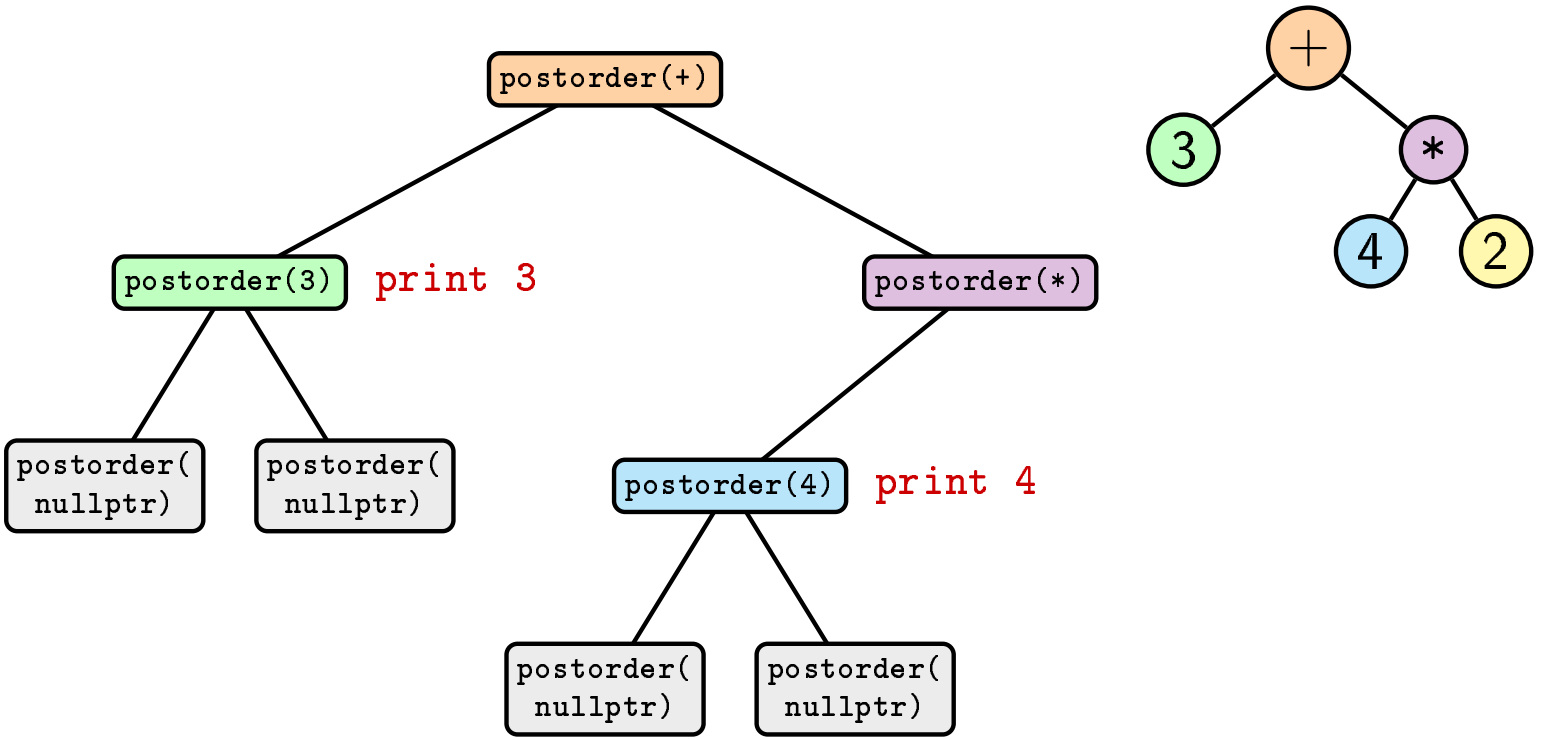
Every non-empty call makes two child calls, even for a leaf; a call on `nullptr` returns at once. **Red** = where a node is printed. **Same color** = **same node** of the expression tree (top right).




```
Output: 3
```

Running `postorder` on `3 + 4 * 2`: the Call Tree

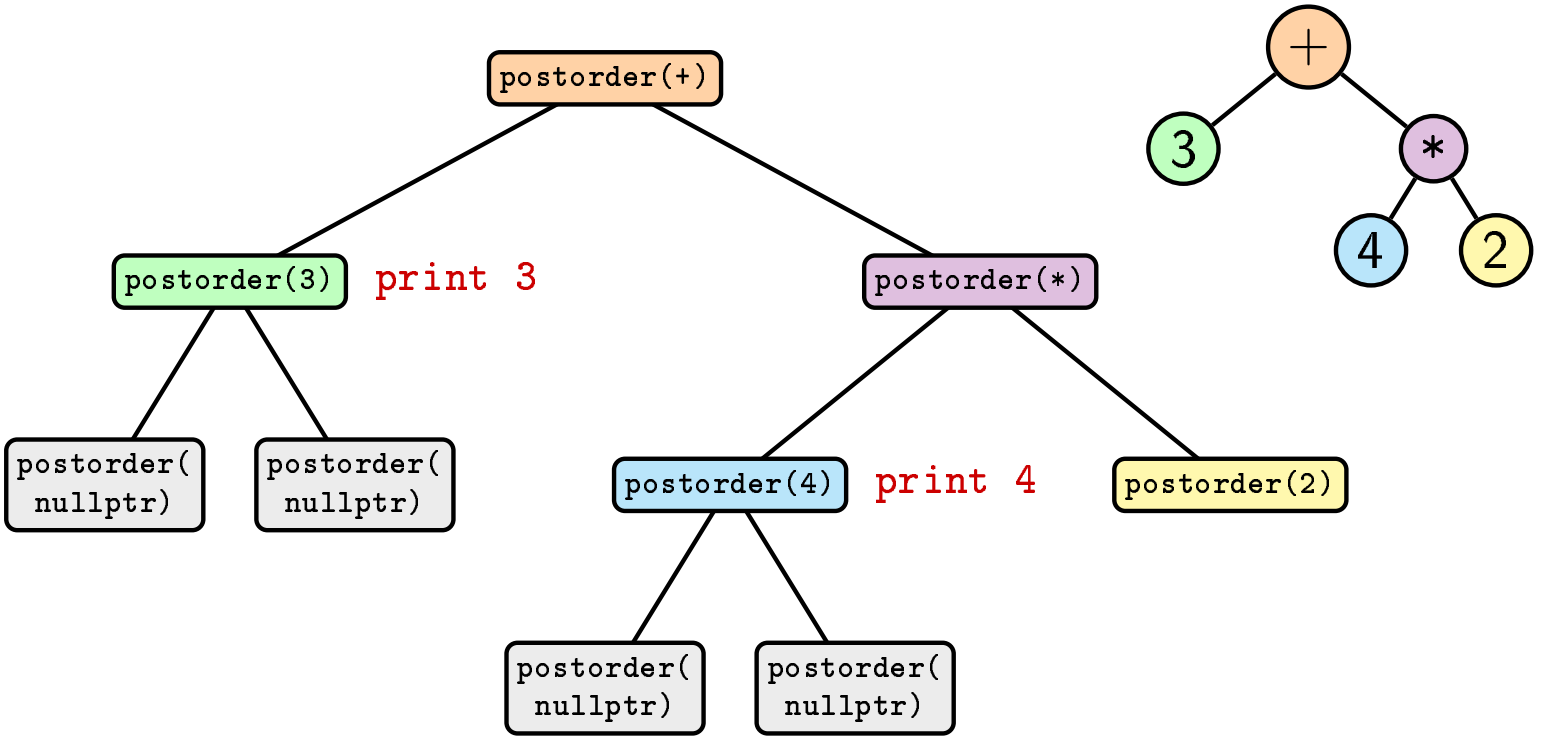
Every non-empty call makes two child calls, even for a leaf; a call on `nullptr` returns at once. **Red** = where a node is printed. **Same color** = **same node** of the expression tree (top right).




```
Output: 3 4
```

Running `postorder` on `3 + 4 * 2`: the Call Tree

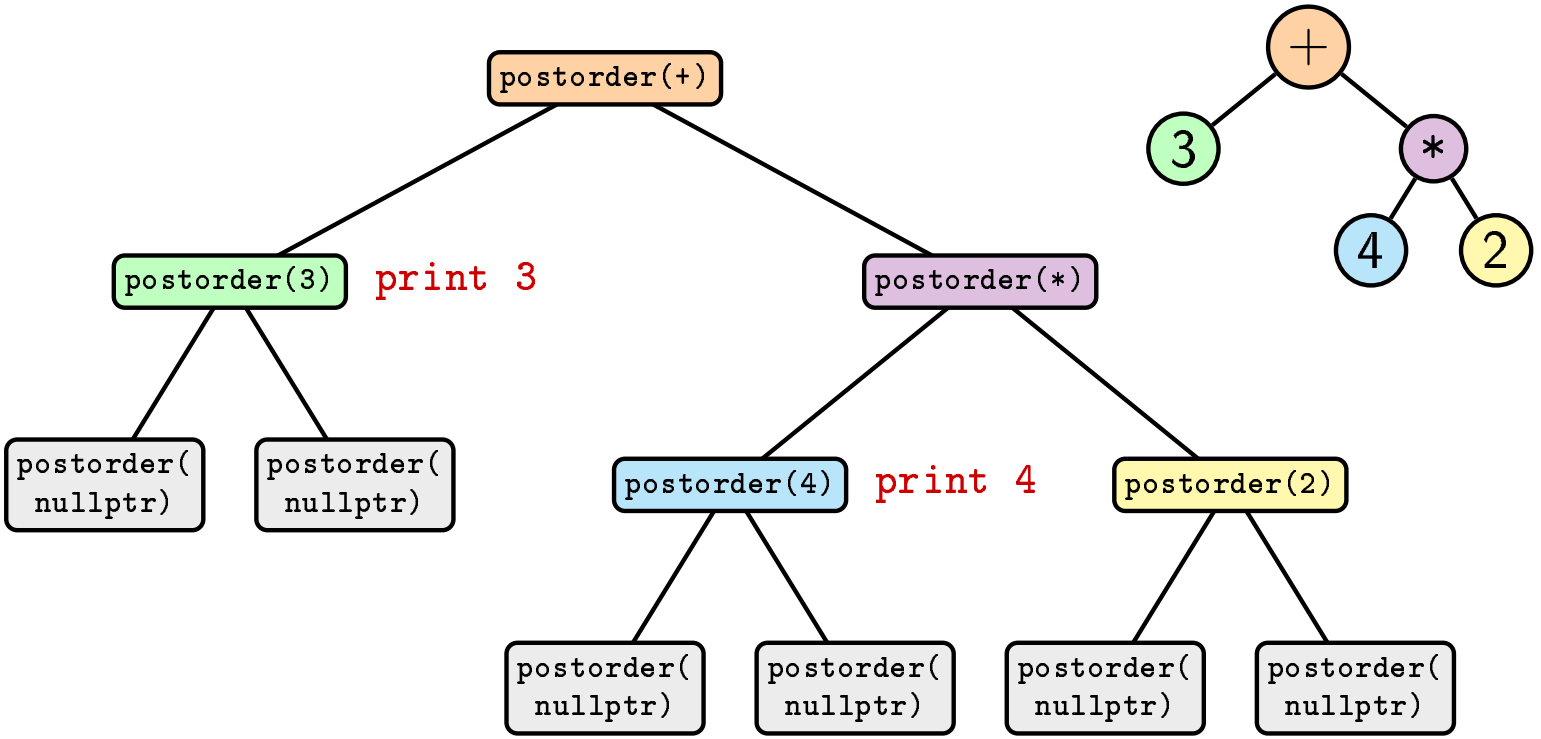
Every non-empty call makes two child calls, even for a leaf; a call on `nullptr` returns at once. **Red** = where a node is printed. **Same color** = **same node** of the expression tree (top right).




```
Output: 3 4
```

Running `postorder` on `3 + 4 * 2`: the Call Tree

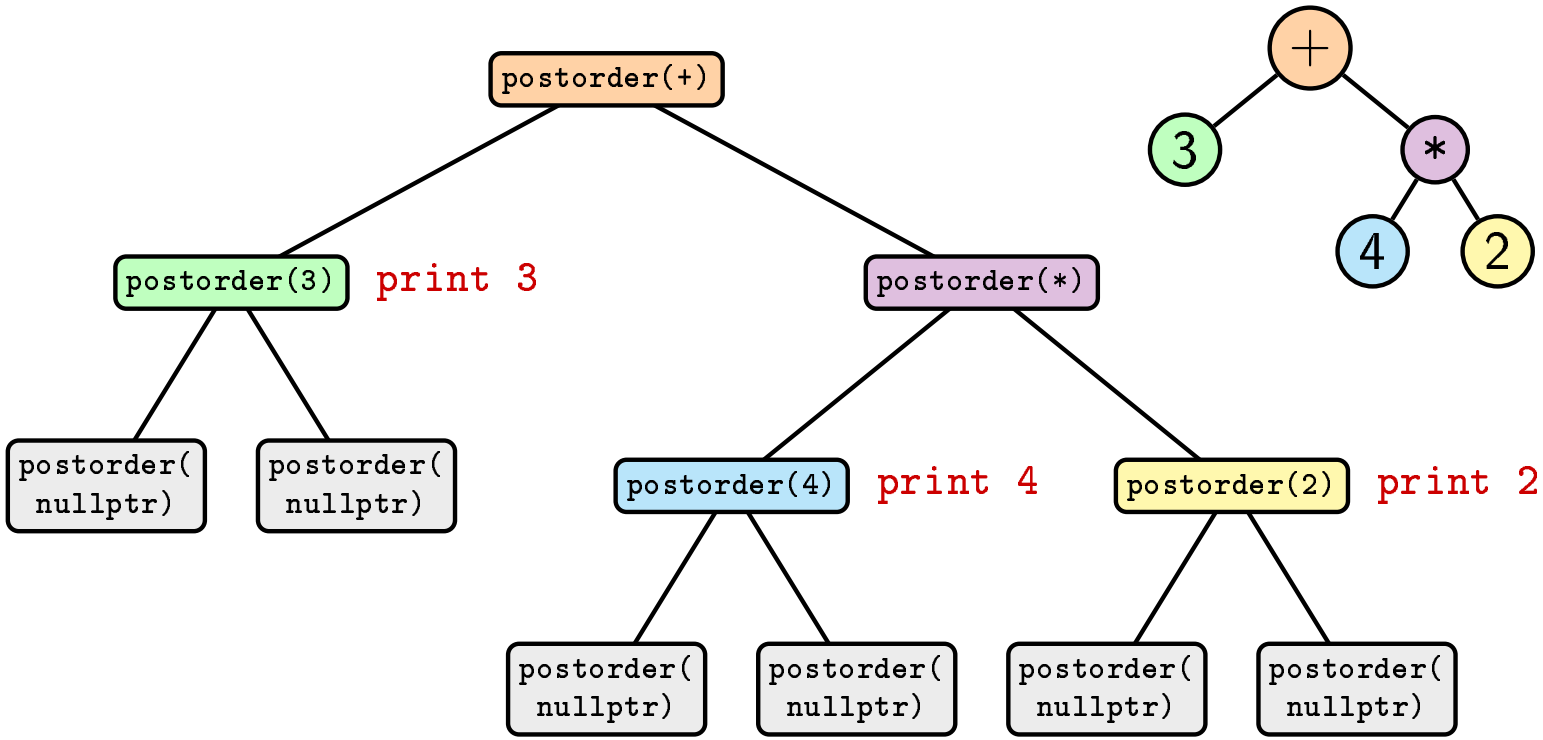
Every non-empty call makes two child calls, even for a leaf; a call on `nullptr` returns at once. **Red** = where a node is printed. **Same color** = **same node** of the expression tree (top right).




```
Output: 3 4
```

Running `postorder` on `3 + 4 * 2`: the Call Tree

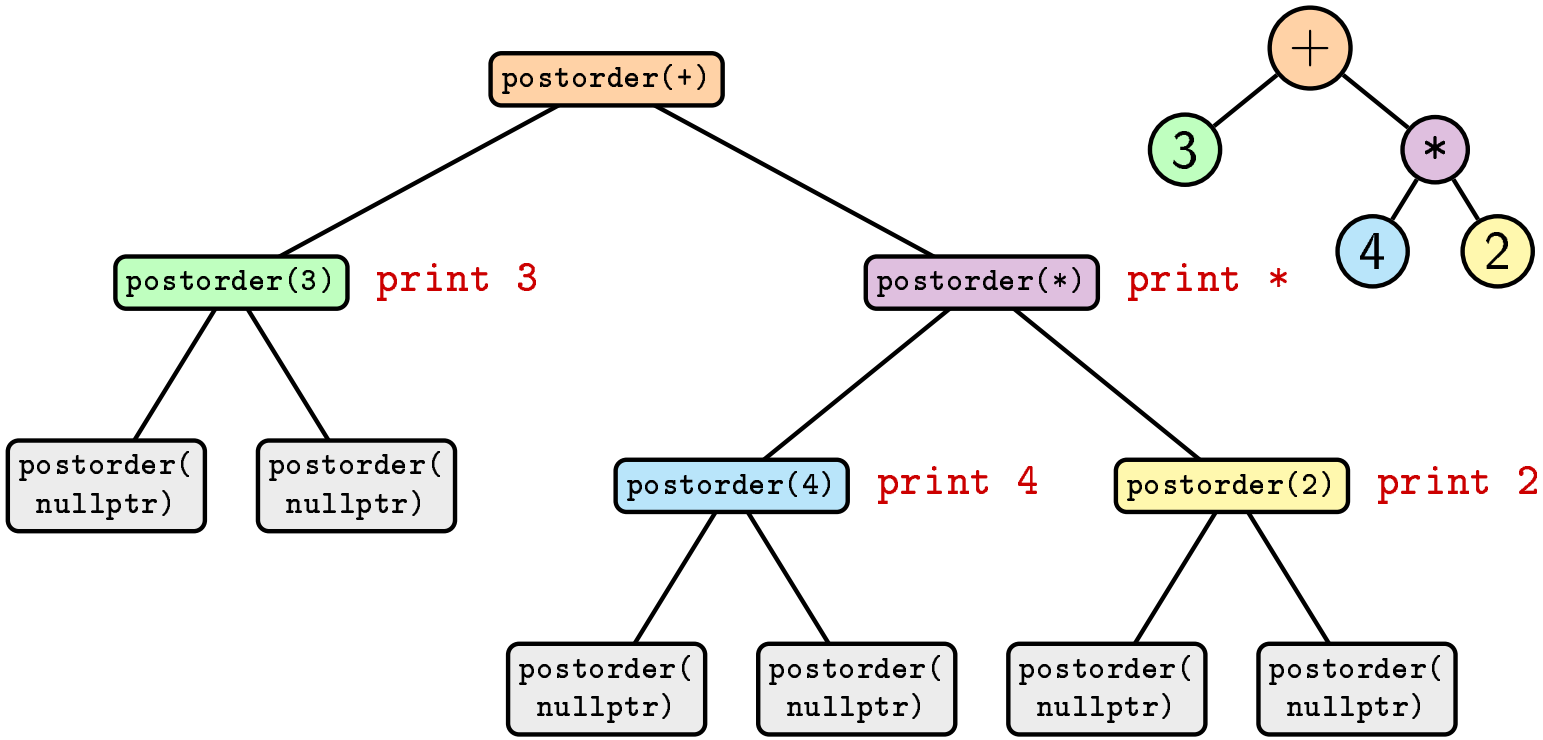
Every non-empty call makes two child calls, even for a leaf; a call on `nullptr` returns at once. **Red** = where a node is printed. **Same color** = **same node** of the expression tree (top right).



`Output: 3 4 2`

Running `postorder` on `3 + 4 * 2`: the Call Tree

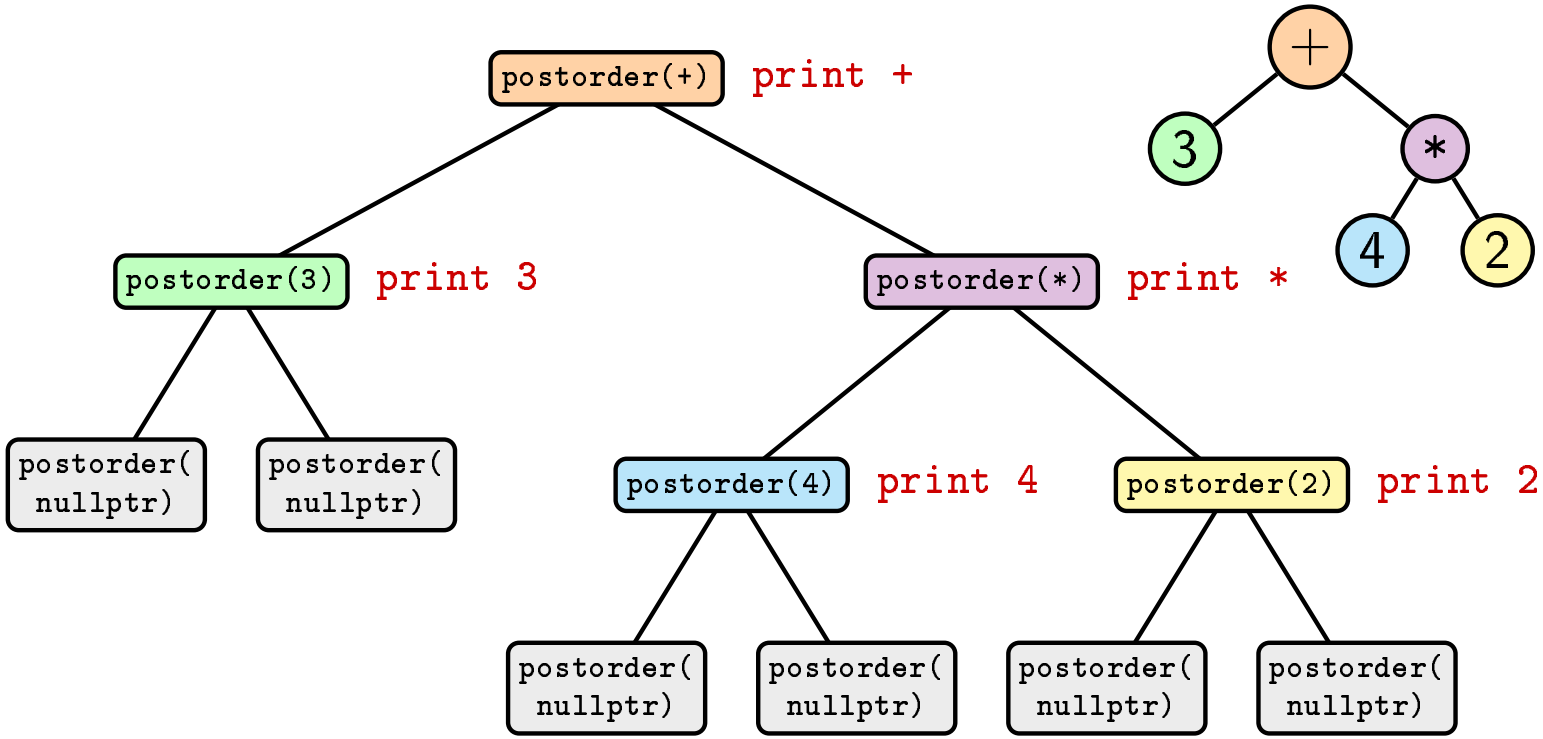
Every non-empty call makes two child calls, even for a leaf; a call on `nullptr` returns at once. **Red** = where a node is printed. **Same color** = **same node** of the expression tree (top right).




```
Output: 3 4 2 *
```

Running `postorder` on `3 + 4 * 2`: the Call Tree

Every non-empty call makes two child calls, even for a leaf; a call on `nullptr` returns at once. **Red** = where a node is printed. **Same color** = **same node** of the expression tree (top right).



`Output: 3 4 2 * +`

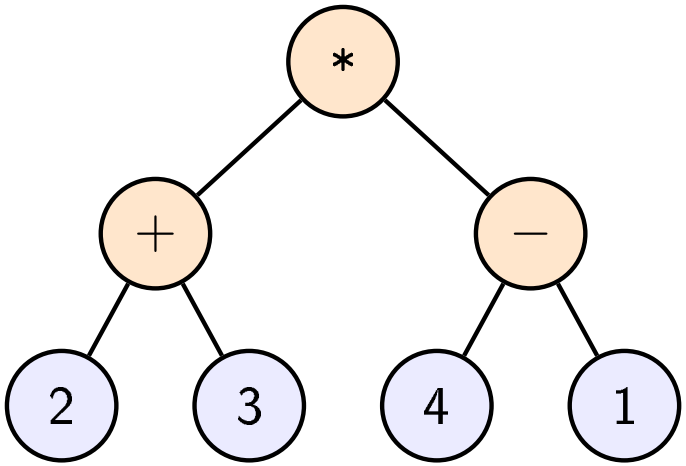
Reading the tree: a call on a leaf such as `postorder(3)` calls `postorder(nullptr)` twice (both return at once) and then prints 3. `preorder` would print each node *before* its two child calls; `inorder` between them.

Question: Your Turn

Draw the tree for `(2 + 3) * (4 - 1)`. Then write its pre-order, in-order, and post-order strings.

Question: Your Turn

Draw the tree for `(2 + 3) * (4 - 1)`. Then write its pre-order, in-order, and post-order strings.



Pre: `* + 2 3 - 4 1`

In (parens): `((2 + 3) * (4 - 1))`

Post: `2 3 + 4 1 - *`

Notice the in-order string **needs** its parentheses (our printer adds one pair per operator node, including the outermost, which is optional mathematically): without them, `2 + 3 * 4 - 1` is a different expression and a different tree. Pre- and post-order need **none**, which is why a stack can evaluate postfix without resolving precedence.

Summary and Preview

- **Operator combinations:** a search over a *shared string*; choose, explore, undo; undo with `resize`.
- **Unique printing:** a per-seat `std::set` skips a repeated letter; the same idea, with a seen set of hands, speeds up Make 24.
- **Task B:** carry each number with its text; combine texts in parentheses; the first non-empty string stops the search.
- **Expression trees:** the last operation is the root. Pre-, in-, and post-order differ only in *when the root is visited*; the left subtree is always first.

Coming up

Next class: how a computer evaluates postfix with one stack, and how the shunting-yard algorithm turns infix into postfix.
Project 3 is due Thursday, November 19.

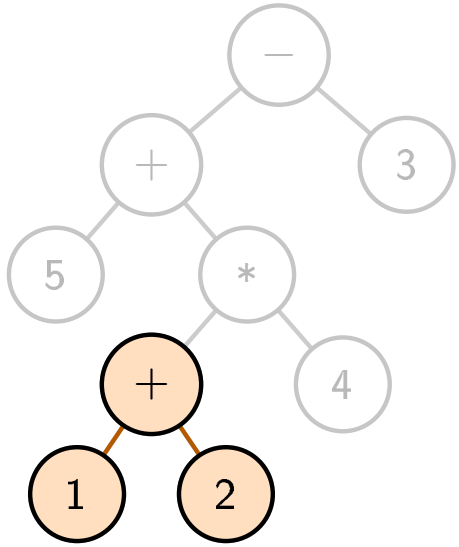
Backup Slides

Backup: a longer tree build and a worked traversal

In Slow Motion: Build `5 + (1 + 2) * 4 - 3`

Each step does the **next operation** and finishes one more **subtree**. The new piece is built from pieces that already exist, and the last piece is the root.

	Infix piece	**Postfix**
(1)	`1 + 2`	`1 2 +`

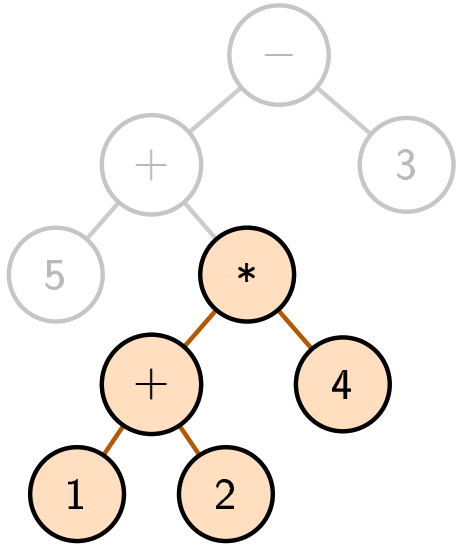


Parentheses first: $1 + 2$ **is its own subtree.**

In Slow Motion: Build `5 + (1 + 2) * 4 - 3`

Each step does the **next operation** and finishes one more **subtree**. The new piece is built from pieces that already exist, and the last piece is the root.

	Infix piece	Postfix
(1)	`1 + 2`	`1 2 +`
(2)	`(1 + 2) * 4`	`1 2 + 4 *`

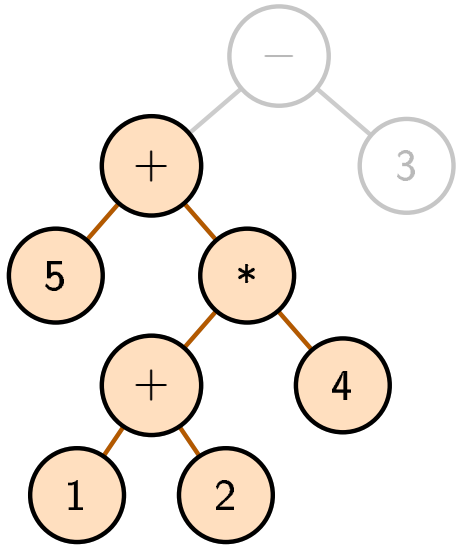


`*` binds tighter than the $+$ to its left, so the finished $(1+2)$ is multiplied by 4 first.

In Slow Motion: Build `5 + (1 + 2) * 4 - 3`

Each step does the **next operation** and finishes one more **subtree**. The new piece is built from pieces that already exist, and the last piece is the root.

	Infix piece	Postfix
(1)	`1 + 2`	`1 2 +`
(2)	`(1 + 2) * 4`	`1 2 + 4 *`
(3)	`5 + (1 + 2) * 4`	`5 1 2 + 4 * +`



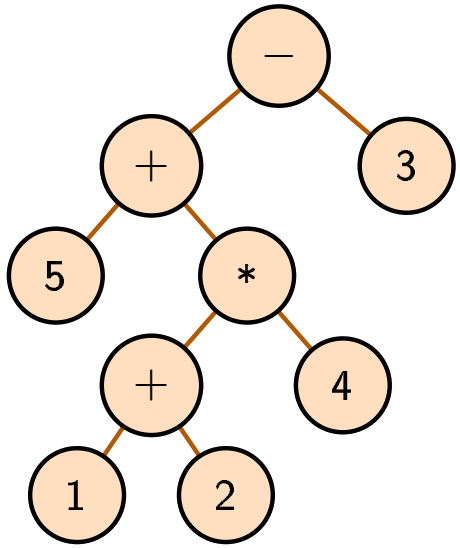
Now the leftover `5` can be added to that whole product.

In Slow Motion: Build `5 + (1 + 2) * 4 - 3`

Each step does the **next operation** and finishes one more **subtree**. The new piece is built from pieces that already exist, and the last piece is the root.

	Infix piece	Postfix
(1)	`1 + 2`	`1 2 +`
(2)	`(1 + 2) * 4`	`1 2 + 4 *`
(3)	`5 + (1 + 2) * 4`	`5 1 2 + 4 * +`
(4)	`5 + (1 + 2) * 4 - 3`	`5 1 2 + 4 * + 3 -`

The last operator, -3, goes at the root: left to right, it is applied to everything built so far.

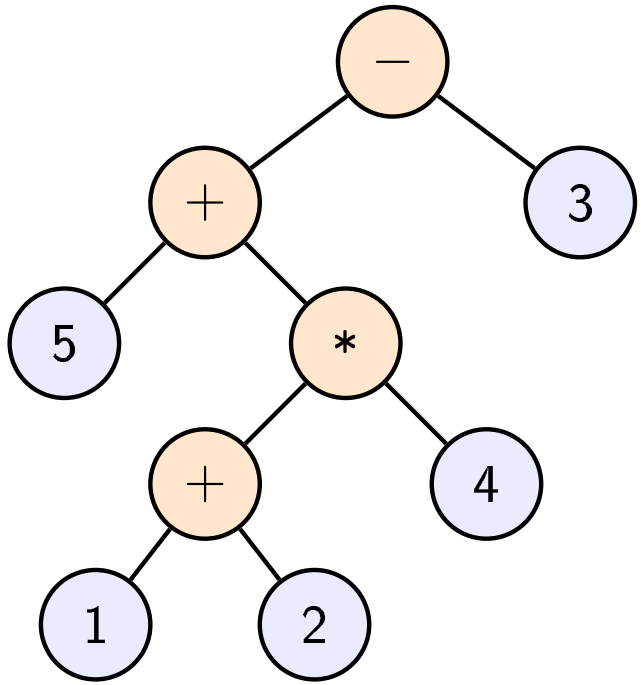


Worked Example: `5 + (1 + 2) * 4 - 3`

Tree. The final tree from the previous slide.

Read. At every node, recursively (left subtree before right):

- **post**(node) = post(left), post(right), node
- **pre**(node) = node, pre(left), pre(right)

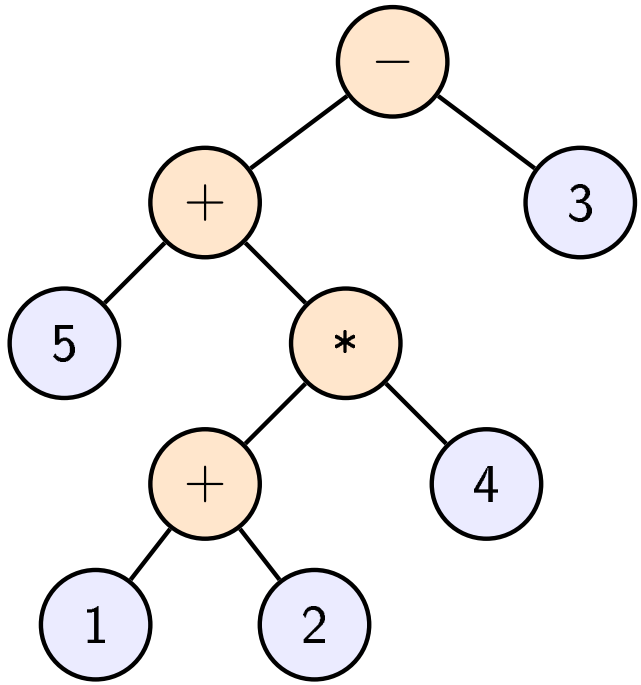


Worked Example: `5 + (1 + 2) * 4 - 3`

Tree. The final tree from the previous slide.

Read. At every node, recursively (left subtree before right):

- **post**(node) = post(left), post(right), node
- **pre**(node) = node, pre(left), pre(right)



Post-order: `5 1 2 + 4 * + 3 -`

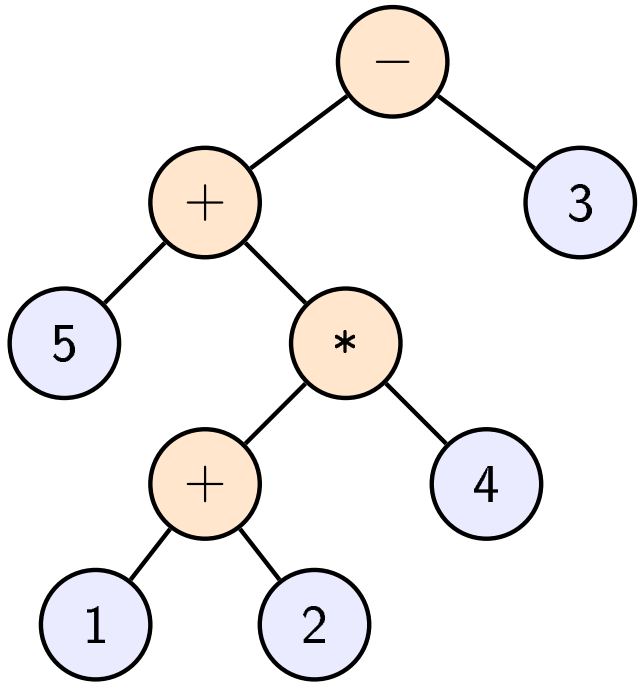
Check: the post-order string is the postfix that the shunting-yard algorithm (a later class) produces.

Worked Example: `5 + (1 + 2) * 4 - 3`

Tree. The final tree from the previous slide.

Read. At every node, recursively (left subtree before right):

- **post**(node) = post(left), post(right), node
- **pre**(node) = node, pre(left), pre(right)



Post-order: `5 1 2 + 4 * + 3 -`

Pre-order: `- + 5 * + 1 2 4 3`

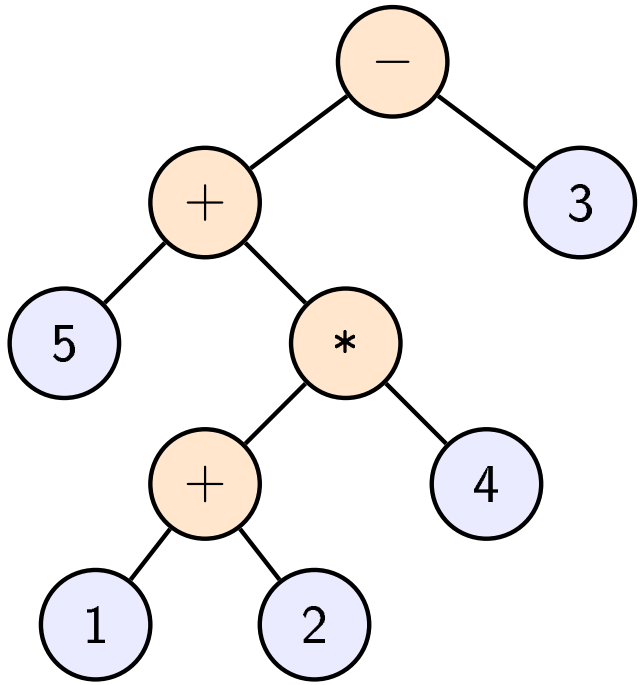
Check: the post-order string is the postfix that the shunting-yard algorithm (a later class) produces.

Worked Example: `5 + (1 + 2) * 4 - 3`

Tree. The final tree from the previous slide.

Read. At every node, recursively (left subtree before right):

- **post**(node) = post(left), post(right), node
- **pre**(node) = node, pre(left), pre(right)



Post-order: `5 1 2 + 4 * + 3 -`

Pre-order: `- + 5 * + 1 2 4 3`

In-order: `5 + 1 + 2 * 4 - 3` (reads as 11)

`((5 + ((1 + 2) * 4)) - 3)` (with parentheses: 14)

Check: the post-order string is the postfix that the shunting-yard algorithm (a later class) produces.